

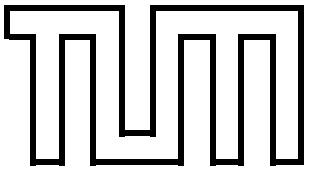
SCHOOL OF COMPUTATION, INFORMATION
AND TECHNOLOGY – INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Voblint: Towards a Verified Goblint-Style Analysis Pipeline in Isabelle/HOL

Manuel Lerchner



SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY – INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Voblint: Towards a Verified Goblint-Style Analysis Pipeline in Isabelle/HOL

Voblint: Auf dem Weg zu einer verifizierten Analysepipeline im Stil von Goblint in Isabelle/HOL

Author:	Manuel Lerchner
Supervisors:	Alexandra Graß, M.Sc. Prof. Dr. Helmut Seidl
Examiner:	Prof. Dr. Francisco Javier Esparza Estaun
Submission Date:	01.12.2026

I confirm that this master's thesis is my own work and I have documented all sources and material used.

Munich, 01.12.2026 Manuel Lerchner

Acknowledgements

I would like to thank my supervisors, Alexandra Graß and Helmut Seidl, for their guidance and support throughout this thesis. I am especially grateful to Alexandra for the many discussions, detailed feedback, and ideas that shaped the project, and I greatly appreciated the freedom to explore the topic in depth and to follow directions that emerged along the way. Javier Esparza kindly agreed to examine this thesis.

I am also grateful to Michael Petter for helping to scope the project and shape its initial direction, and to Julian Erhard for ideas and feedback that brought the formalization closer to Goblint. I also thank Sarah Tilscher for developing and maintaining the verified TD solver formalization on which this work builds.

I also acknowledge the many contributors to Goblint, whose work on the analyzer provided both a practical reference point and much of the inspiration for this thesis, as well as the developers of Isabelle and the open-source tooling around it. In particular, the Isabelle interfaces offered by AutoCorrode and Kevin Kappelmann's MCP server made it practical to integrate AI-assisted development into a formalization of this scale.

I am also grateful to the members of the Chair for Programming Languages, Compiler Construction and Specification Formalisms. I first came to know the chair through the course Functional Programming and Verification, and it became an important part of my time at TUM. Working there twice as a teaching assistant and once as a tutor gave me insight into both teaching and research, and showed me how much I enjoy teaching formal methods.

Finally, I would like to thank my family and friends. My family gave me the freedom to focus fully on my work and encouraged me through every stage of my studies. My friends provided encouragement, welcome distractions, and regular reminders that there is a world outside Isabelle, and their company made the more intense periods of work much easier. Their patience, friendship, and support have meant a great deal to me throughout my time at TUM.

Abstract

An abstract interpreter over-approximates a program's executions, so any property that holds for the over-approximation holds in every run. Goblint, an abstract interpreter for C, builds a control-flow graph of the program, translates the graph into a system of equations over abstract values, and solves that system with a generic fixpoint solver [1], [2]. Its guarantee therefore depends on four conditions: a control-flow graph that simulates the source program, equations that over-approximate its executions in every calling context, a solver result that is a post-solution of the equations, and verdicts that follow from that result. For Goblint's top-down solver, only the third condition had been verified [3], [4].

We present Voblint, an Isabelle/HOL formalization of a context-sensitive interprocedural analyzer. Its soundness proof covers all four conditions for VIMP, a small C-like language with fixed-width integer variables and recursive procedures with parameters and return values. A parsed program is translated into core VIMP, which makes every integer type and conversion explicit, as CIL does for Goblint [5]. The main theorem combines the proofs of the four conditions. It states that if the analyzer returns a report, the report covers every store that a finite execution of the translated program reaches from an initial store, and that every definite verdict at the corresponding program point holds for that store. We also prove that a check reported `DEAD` is unreachable and that an arithmetic obligation reported as an error fails wherever its point is reached. At a point without an arithmetic warning or error, no divisor is zero, no signed operation overflows, and no conversion changes its value. To our knowledge, Voblint is the first mechanized analyzer whose soundness proof connects a source semantics to a side-effecting constraint system and its verified solver.

The main difficulty is context sensitivity. A context-sensitive analyzer computes a separate invariant for each program point and calling context. The collecting semantics, however, merges the stores of all calls to a procedure, so it cannot say which stores a given context must cover. To solve this, we introduce activation traces, inspired by the local traces of Schwarz et al. [6]. Each trace follows one procedure activation, and a context policy decides to which contexts it belongs. If the policy admits every reachable call in some context, the union of the per-context sets of stores equals the collecting semantics, so splitting by context loses no execution.

The proof is modular and follows Goblint's architecture. Domains, context policies, analyses, and the solver each have their own proof obligations, and one soundness theorem combines them. A numeric domain only has to implement a shared interface and prove its abstract operations sound. The framework builds transfer functions from these operations and proves them sound once for all domains, as in Nipkow and Klein's generic abstract interpreter [7, §13.5]. The main theorem holds for every offered choice of analysis, context policy, placement of program globals, and treatment of signed overflow.

The analysis function is exported to OCaml, runs in a command-line interface and a browser playground, and is tested on 443 regression programs. Proofs by evaluation show that the main theorem is non-vacuous on concrete programs.

The main result is partial correctness: termination of the solver is not proved. We trust everything outside the formalization: the hand-written OCaml code, including the parser and the rendering, Isabelle's code generator, and the OCaml and WebAssembly toolchains. We document how VIMP differs from C11 [8].

Use of Generative AI

The Leiden Declaration on Artificial Intelligence and Mathematics recommends that authors transparently disclose the automated tools they use, including large language models [9]. This section follows that recommendation.

Substantial parts of this work were carried out with generative AI tools, primarily Anthropic's Claude, OpenAI's ChatGPT and Codex, and Cursor, under the author's direction and review. These tools contributed a large share of the implementation-level material. The author chose the research direction, selected and revised the designs, and approved the definitions and theorem statements.

In the Isabelle development, the tools generated many proofs, carried out refactorings across theories, and transformed tactic-style proofs and Sledgehammer output into structured Isar. They proposed designs and drafted planning documents for larger changes. For the thesis, they assisted with literature searches, checked citations and claims against cited sources, and drafted and revised the chapter structure, text, and figures.

Separately, Grammarly was used to check spelling and grammar in the thesis text. The Isabelle interfaces of AutoCorrode [10], used following the human-guided workflow of Kappelmann et al. [11], served as proof-development infrastructure.

The assistance also extended to design. To formalize calling contexts, the concrete semantics is based on traces instead of states, inspired by the local traces of Schwarz et al. [6]. The state-based collecting semantics of the prototype records the stores that reach a node, but not which procedure activation contains each store (Section 4.1). Language models helped adapt the thread-local setting of that work to procedure activations. The requirements for the activation traces of Section 4.2.1 were refined interactively with them: how traces are generated, that a callee trace begins only at a call, that a trace retains its call history, and that calling contexts are derived from traces.

For the analysis framework, agents inspected Goblint's OCaml sources to identify interfaces the formalization could approximate, such as local and global unknowns, unknowns indexed by node and context, and the enter/combine protocol at calls (Section 15.5). The source language and its compilation, the activation traces, the coverage contract, and the corresponding proofs have no direct counterpart in Goblint and were developed for this work. The formalization reuses the Isabelle/HOL formalization of Goblint's top-down solver and its partial-correctness proof by Tilscher et al. [4].

A major part of the development effort concerned finding suitable definitions, interfaces, invariants, and locale boundaries rather than discharging individual proof steps. During the first weeks, a small prototype mirrored Goblint's structure while leaving proof obligations as `sorry`. Once the definitions and locale boundaries had stabilized, many of the remaining proof obligations were short and were discharged with substantial agent assistance.

The assistants' output was not reliable on its own. Agents proposed incorrect lemmas and, in one case, introduced a premise into the end-to-end theorem that no program satisfied, making the theorem vacuous until the solver run was redesigned. Audits of theorem statements and full batch builds exposed these errors. The non-vacuity instances of Section 13.1 now demonstrate that the premises of the main theorems can be satisfied.

Agents also wrote much of the tooling around the formalization, such as the command-line interface, the website, and the regression suite with its ports of Goblint regression tests. They also wrote the checks that keep the theories, the generated material, the analyzer,

the website, and this thesis consistent (Section 14.1). These checks proved especially useful because they replaced repeated manual inspection with conditions that every change had to satisfy, and they exposed errors that manual reading had missed.

The table below quantifies the recorded part of this assistance, following the session-log analysis of Bryant et al. [12, Section 5]. The counts measure interaction with the tools, not authorship or the proportion of the work attributable to them.

	Claude Code		**Codex**	
	main	subagent	main	subagent
sessions	110	451	121	47
assistant messages	52,374	22,755	15,063	484
tool calls	56,474	27,952	25,235	2,402
shell	27,525	17,568	–	–
Isabelle batch builds	726	97	361	1
Isabelle server (I/Q, PIDE)	24,149	5,032	18,251	1,247
file edits and writes	850	1,340	600	22

Recorded automated-assistant activity during development, counted on 2026-09-25 from the local session logs (2026-08-24 to 2026-09-25 for Claude Code, 2026-06-27 to 2026-09-21 for Codex). No local logs are available for earlier Claude Code sessions, ChatGPT in the browser, or Cursor, so these are not counted. The author sent 2,434 messages to Claude Code and 1,227 to Codex. Shell edits count as shell calls. Interactions through the Isabelle server, including theory edits, count as server calls. Co-author lines in commit messages were not kept consistently and are not used as a measure.

Isabelle/HOL accepts every theorem in the development. Proofs by evaluation, used for some facts about fixed programs, additionally rely on Isabelle's code generator (Section 12.5). This establishes the stated propositions relative to their definitions and assumptions, but does not establish that those definitions faithfully capture the intended language or analysis. Definitions and theorem statements were therefore also checked against example programs, the regression suite, and the documented differences between VIMP and C11. Related-work claims were checked against the cited sources, often with agent assistance and subsequent author review.

Responsibility for the final claims, framing, and treatment of related work lies with the author alone.

Contents

Acknowledgements i

Abstract ii

Use of Generative AI iii

Part I: The Problem

1. Introduction 2

1.1. From executions to static guarantees . . . 2
1.2. State of the art . . . 4
1.3. Contributions . . . 7
1.4. Scope and limitations . . . 8
1.5. Outline . . . 9

2. Background 10

2.1. Executions and the collecting semantics . . . 10
2.2. Lattices and fixpoints . . . 11
2.3. Abstract interpretation . . . 12
2.4. Constraint systems . . . 14
2.5. Widening and narrowing . . . 15
2.6. Side-effecting constraint systems . . . 16
2.7. The verified top-down solver . . . 17

Part II: What Must Be Over-Approximated

3. Programs and Control-Flow Graphs 20

3.1. VIMP, and what it leaves out . . . 20
3.2. Machine integers and elaboration . . . 22
3.3. Executing a source program . . . 25
3.4. Scope and adequacy of the source semantics . . . 27
3.5. The procedure-aware control-flow graph . . . 29
3.6. Compilation . . . 31
3.7. Relating the two executions . . . 33

4. Traces and the Coverage Contract 36

4.1. Why contexts need traces . . . 36
4.2. Traces . . . 37
4.3. Calling contexts . . . 41
4.4. The coverage contract . . . 43

Part III: Analyses

5. Abstract Domains **47**

5.1. What an abstract value means . . . 47
5.2. Values at a machine kind . . . 48
5.3. From values to stores . . . 51
5.4. Evaluating expressions . . . 54
5.5. Answering queries . . . 54
5.6. Learning from a guard . . . 55
5.7. The complete domain interface . . . 57
5.8. A carrier the interface excludes . . . 57

6. What an Analysis Supplies **60**

6.1. Transfers at edges and calls . . . 60
6.2. Facts without a program point . . . 61
6.3. The analysis contract . . . 62
6.4. Program globals: flow-sensitive or flow-insensitive . . . 63

7. How Analyses Cooperate **65**

7.1. Questions and answers . . . 66
7.2. Proving an analysis against any channel . . . 66
7.3. Many analyses over one state . . . 67
7.4. What a new analysis has to prove . . . 69

Part IV: From Analyses to Verdicts

8. Equations, Contexts, and Routing **71**

8.1. Unknowns and right-hand sides . . . 71
8.2. Routing and context policies . . . 73
8.3. Soundness of the generated system . . . 75

9. Solving the Equations **77**

9.1. Adapting the equations to the solver . . . 77
9.2. A demand-driven solve . . . 80
9.3. What a terminating solve guarantees . . . 80
9.4. Merging contributions to global unknowns . . . 83
9.5. Making abstract states executable . . . 83
9.6. Why termination is not proved . . . 85

10. What a Report Guarantees **87**

10.1. From solution to report . . . 87
10.2. Verdicts . . . 88
10.3. The chain at one check . . . 89
10.4. The source-level theorem . . . 90
10.5. What a verdict says about executions . . . 92

Part V: Instances and Practice

11. Five Domains and an Order Analysis **95**

11.1. The implemented domains . . . 95
11.2. One loop, five answers . . . 96
11.3. The reduced product . . . 96
11.4. The order analysis . . . 98

12. From Theorem to Tool **100**

12.1. From theorem to code . . . 100
12.2. The generated code . . . 101
12.3. The unverified shell . . . 101
12.4. Tracing the exported solver . . . 102
12.5. The trust boundary . . . 102

13. Evaluation **105**

13.1. Non-vacuity . . . 105
13.2. Do the obligations exclude real errors? . . . 106
13.3. Precision on concrete programs . . . 107
13.4. Where precision ends . . . 109
13.5. Tests of the unverified parts . . . 113

14. Engineering and Tooling **115**

14.1. The Voblint repository . . . 115
14.2. `isar-tools`: source tooling for Isabelle . . . 116
14.3. Changes to the verified solver . . . 117

Part VI: Assessment

15. Related Work **120**

15.1. Verified analyzers: Verasco and the CompCert line . . . 120
15.2. Mechanized abstract interpretation . . . 121
15.3. Verified fixpoint solvers . . . 122
15.4. Context sensitivity and trace partitioning . . . 122
15.5. Goblint, local traces, and thread-modular analysis . . . 123
15.6. Where Voblint sits . . . 125

16. Conclusion **126**

16.1. Discussion . . . 126
16.2. Limitations . . . 127
16.3. Outlook . . . 128

Bibliography **131**

PART I

The Problem

1. Introduction

A static analyzer answers questions about every execution of a program at once: whether a divisor can be zero at some point, whether a signed operation can overflow, whether a conversion can change a value, whether a program point is reachable at all, or whether a check always holds. The analyzer is itself a program and can contain errors. Such an error can produce a confident answer that real executions contradict. In Goblint issue 1156, Goblint claimed that `c % 2 == 1` holds for $c \in \{-5, -7\}$ [13]. However, the truncating remainder of C11 [8, §6.5.5p6] gives -1 for both values. Pull request 1161 fixed the congruence domain, which had ignored the sign of the dividend [14]. Such bugs are particularly bad because a user who trusts the answer has no reason to look for the error. An imprecise answer, in contrast, only leaves the check undecided. In this thesis, we build Voblint, an executable analyzer whose answers are proved sound for every execution of the analyzed program, and we state exactly under which assumptions this proof holds.

Formal verification states such a claim as a proposition over explicit definitions, and a proof assistant checks its proof [7]. Isabelle/HOL constructs every theorem through a small trusted inference kernel, following the LCF approach [15], so the kernel accepts a proof regardless of who or what wrote it. This matters now that LLM coding agents produce large formalizations. Bryant et al. [12] report agents that wrote over 85,000 lines of Isabelle/HOL covering Munkres' general topology, with all 806 results proved. This thesis was developed with AI assistance as well (see Use of Generative AI), and its proofs pass the same check.

The kernel guarantees only that the formal statement follows from the definitions. Whether the definitions model the intended objects, and whether the statement expresses the intended claim, is left to human review. A review of parts of the Munkres formalization found definitions that are logically weaker than the textbook's. They are harmless only because each theorem assumes the missing constraints again [12, §8.1].

Verification projects therefore explicitly state what lies outside the proof. The seL4 proof relates the kernel's C implementation to an abstract specification in Isabelle/HOL and names the compiler, assembly code, boot code, cache management, and hardware as assumptions [16]. Voblint draws its boundary at the source language. Its main theorem starts from *source executions*, the executions of the elaborated program under the small-step semantics `pstep` of Voblint's input language VIMP. *Elaboration* (`elab_ptable`) translates the parsed program into core VIMP with explicit integer kinds. Every later representation is connected to source executions by proof, so only elaboration and `pstep` need an adequacy argument. That argument states which fragment of C they model, where they depart from C11 (Section 3.4), and why no existing verified semantics serves as the anchor.

1.1. From executions to static guarantees

Software is usually checked by testing. A failing test demonstrates a bug, but a passing test suite has observed only finitely many executions [17, §1.4.1]. A static analysis reasons about

all executions instead. At each program point, it computes an *abstract state*. An abstract state is a finite representation of a set of program states, for instance, an interval that contains every value a variable takes whenever execution reaches the point. The analysis is *sound* if the abstract state contains every state that occurs in a real run, although it may contain more. If a sound abstract state contains no state that violates a check, no execution violates it, tested or not (Figure 1.1). The converse does not hold. An abstract state can contain violating states that no execution reaches, and the analysis then cannot decide the check, even when every execution satisfies it.

Suppose the analysis computes $x \in [43, +\infty]$ at a point that divides by `x`. Every execution that reaches this point has $x \geq 43$, so the division is safe. If it computes only $x \in [0, 100]$, the abstract state contains $x = 0$, and the analysis cannot prove the division safe. However, every real execution may still have $x \geq 43$, for instance, because of a relation to another variable that an interval cannot express.

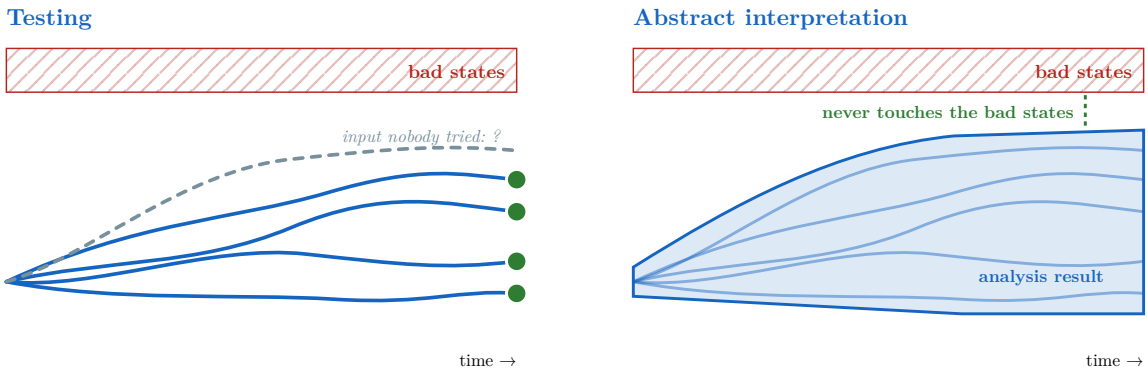


Figure 1.1: Testing and static analysis, schematically. Each curve is one execution, and the vertical axis stands for the program state over time. Left: tests observe only the runs they execute, and a run on an input nobody tried (dashed) stays unknown. Right: a sound analysis result (shaded) contains every run, tried or not, and may also contain states that no run reaches. Because it does not overlap the bad states, no execution can reach them.

For a general-purpose language, this incompleteness is unavoidable. By Rice's theorem, no algorithm decides a nontrivial semantic property for every program of a Turing-complete language [17, §1.3.3], [18]. A sound analysis may therefore raise false alarms and leave checks undecided, but it never makes a false claim. It is useful as long as it decides the checks of interest often enough [17, §1.3.5]. Since every sound analysis over-approximates the same executions, analyses can specialize. One tracks intervals, another parities, and another relations between variables. If any one of them proves a check, the check holds in every run. An analysis can also separate the activations of a procedure by their *calling context*, an abstraction of how the call was reached, such as the call site or the abstract state at entry. It then keeps one abstract state per program point and context. A richer domain or more contexts can decide more checks, but they also make the analysis more expensive. If there are many contexts, the analysis may not finish in practice.

Voblint answers every check that the user writes into the source program as a statement `__voblint_check(e);`. It also warns at every arithmetic operation where it cannot exclude a hazard: a zero divisor, a signed overflow, or a conversion that changes a value. Figure 1.2 shows a run in the browser playground together with what each answer guarantees. Only

`DEAD` makes a claim about reachability, and `REFUTED` is not a verified counterexample. Each run fixes an analysis configuration. It consists of the activated analyses (the domains Sign, Interval, Parity, Congruence, and the reduced-product domain Int, and an order analysis), the context policy that chooses the contexts, the rule that combines contributions to global values, the placement of the program's globals, and the treatment of signed overflow. The run shown uses intervals without calling contexts.



Line	Answer	What it guarantees
14	`PROVED`	the condition holds whenever a run reaches it
15	`REFUTED`	the condition fails whenever a run reaches it; not a counterexample
16	`UNKNOWN`	nothing
20	`DEAD`	no run reaches the check
18	`WARNING`	a zero divisor could not be excluded; no run is shown to divide by zero
17	none	no run reaching this point divides by zero, overflows, or changes a value by a conversion

Figure 1.2: One analyzer run on a clamp function, as the browser playground shows it, with the interval domain and no calling contexts. The analysis covers every value of `t` at once. The screenshot opens the run in the playground, as does the `VIMP` ↗ tag on every later VIMP listing (claim `intro-answers`, fixture `24-site-figures/precision/48-clamp_every_answer.vimp`).

1.2. State of the art

Most abstract interpreters in practical use have no machine-checked soundness proof and rely on testing instead. Livshits et al. observe that virtually all published whole-program analyses purposely make unsound choices [19, p. 44]. Such an analysis over-approximates most language features but under-approximates a few features that experts know well, for instance, reflection in Java, `eval` in JavaScript, or `setjmp`/`longjmp` in C. The authors call such analyses *soundy*. They treat this as an engineering choice for precision and scalability, and they ask authors to state clearly the nature and extent of the unsoundness, instead of mentioning it only in passing [19, p. 45].

Table 1.1 compares Voblint with six analyzers that have no machine-checked soundness proof. Five of them claim to be sound. Astrée, for instance, proves the absence of run-time errors in C programs of up to 132,000 lines [26]. Pulse-X is a bug finder instead, and its under-approximating soundness theorem is proved on paper [24, pp. 81:2–81:3]. The tools differ most in how they treat procedures. Four of them inline the callee at every call and therefore need no context policy [21, §5.3], [22, §5.1], [23, §3], [26, §5.3]. Inlining cannot handle unbounded recursion, and these tools exclude recursion, unroll it to a bounded depth, rely on user-written contracts, or assume that a recursive call may change any value [21, §6.3.9], [26, §5.3], [27, Limitations], [28, Analysis Assumptions]. Goblint, like Voblint, keeps one abstract state per program point and calling context and analyzes recursive programs. It is therefore the closest model for Voblint, but only its solver has a mechanized proof [3], [4].

Such tools are checked against executions. Cuoq et al. [29] compare Frama-C's value analysis with compiled execution on programs generated by Csmith and report fifty bugs found and fixed. By differential testing, which flags answers on which analyzers disagree, Klinger et al. [30] find soundness or precision issues in four of six analyzers, and Kaindlstorfer et al. [31] find 16 soundness issues in seven of eight, including MOPSA. Since its first participation in SV-COMP in 2021, Goblint has never produced a false negative on the competition's benchmark suite [2]. Still, its developers report soundness issues that the benchmarks missed [32, §3.4.6]. The Goblint defect at the start of this chapter is an error in one abstract operation. Voblint proves a soundness obligation for each of its abstract operations, which excludes this kind of error.

CompCert provides a precedent for a theorem about a delivered tool. It proves that its compiled code behaves as the source semantics specifies, and it lists the components that remain trusted [33]. Verasco carries the approach to a static analyzer for C [34]. For Goblint, local traces give each thread a concrete semantics against which its thread-modular analyses are proved sound on paper [6], [35], [36, Ch. 6].

Table 1.2 compares the mechanized abstract interpreters most closely related to Voblint. Their proofs cover small or intermediate languages, from While and IMP to C#minor. The differences that matter for Voblint lie in recursion, contexts, and the fixpoint computation. Verasco raises an alarm where a recursive call can occur [34], and three of the works have no procedures at all. No work in the table lets the context policy give one call several contexts, which an analysis needs when it splits one call into several alternatives. Only Cachera et al. compute the fixpoint with a verified solver, and that solver handles ordinary inequations, not side-effecting ones [38]. Only two works specify the constraints without computing them, and they have no executable analyzer [40], [41].

Tool	Language	Stance	Calls	Recursion	Relational	Combination	Mechanized
Astrée [20]	C subset	sound	inlining	not used	octagons	channels	none
Eva [21]	C	sound	inlining	contract	Apron	via values	none
MOPSA [22]	C, Python	sound	inlining	unsupported	Apron	products, queries	none
IKOS [23]	C/C++	sound	inlining	assumptions	Apron	fixed products	none
Pulse-X [24]	C	bug finding	summaries	unrolled	pure conditions	one domain	none
Goblint [25]	C	sound	per context	analyzed	Apron	products, queries	TD algorithm
Voblint	VIMP	proved sound	per context	covered	order analysis	products, queries	end to end

Table 1.1: Unverified analyzers and Voblint, as described by each tool's papers and documentation. *Stance* is the tool's own soundness claim. Under *calls*, inlining reanalyzes the callee at every call, a summary is computed once per procedure, and *per context* means one abstract state per program point and context. *Recursion* says how a recursive call is treated. *Analyzed* means the tool analyzes recursive calls, and *covered* means that Voblint's theorem includes them. *Relational* names the domains that relate variables. Apron is a library of relational numeric domains, and Voblint's order analysis tracks which variable is at most which. *Combination* names how domains exchange facts. Astrée's domains send each other constraints through reduction channels, Eva's domains communicate through the abstract values of expressions, a product runs domains side by side on one state, and a query lets one analysis ask another for a fact at the current point. *Mechanized* records what a proof assistant has checked. *End to end* means from source executions to the verdicts of the exported function, for every answer it returns (partial correctness) and with the trusted components of Section 12.5.

Voblint's architecture follows Goblint, an abstract interpreter for multithreaded C programs [1], [2]. Goblint defines analyses independently of the generic solvers that compute their results. The interface between the two is a side-effecting constraint system [2], [43], [44]. Such a system relates unknown abstract values by inequalities, and the inequality of one unknown may also contribute to other unknowns. Goblint's top-down solver, including its extension to side effects, has been formalized and proved partially correct in Isabelle/HOL [3], [4]. When it terminates, it returns values that satisfy the inequalities of the queried unknown and of every unknown it transitively reads (`part_post_solution`). Whether that system describes the program, and whether the verdicts read off its solution hold, lies outside the solver's theorem.

Calling contexts add another gap, since the per-context abstract states of an analyzer have no counterpart in an ordinary concrete semantics. Suppose `main` calls `bump(5)` and `bump(4)`, and `bump` returns its argument plus one. An analysis whose contexts record the argument at entry claims the result 6 in the first context and 5 in the second. The collecting semantics at the point where `bump` returns holds both results. It forgets how each store was reached, so it does not say which store belongs to which context. To state these claims, the concrete semantics must record the context of every *activation*, one execution of a procedure from its call to its return.

To our knowledge, no prior mechanized analyzer connects a source semantics to a side-effecting constraint system and its verified solver. Among the works of Table 1.2, Dabrowski and Pichardie are closest to Voblint in their treatment of contexts. Their context function is a parameter of the concrete semantics, but it gives each call exactly one context. Their analyses are constraints that no solver computes [40, §7]. Voblint's theorem combines four properties that no work in the table has together. It covers recursive procedures, takes

Work	Prover	Language	Recursion	Contexts	Fixpoint	Executable
Verasco [34]	Coq	C#minor	alarm	per call site	structural	yes
Blazy et al. [37]	Coq	Cminor CFG	covered	intraprocedural	checked	yes
Cachera et al. [38]	Coq	Java Card	covered	none	solver	yes
Cachera, Pichardie [39]	Coq	While	no procedures	none	structural	yes
Dabrowski, Pichardie [40]	Coq	Java-like	covered	object-sensitive	specified	no
Nipkow, Klein [7]	Isabelle	IMP	no procedures	none	iterator	in prover
Lammich, Müller-Olm [41]	Isabelle	flowgraphs	exact	summaries	specified	no
Franceschino et al. [42]	F*	IMP	no procedures	none	structural	yes
Voblint	Isabelle	VIMP	covered	set per call	TD solver	yes

Table 1.2: Mechanized abstract interpreters with machine-checked soundness proofs, as described in each work. *Prover* is the proof assistant and *Language* the analyzed language. *Recursion* says how a recursive call is treated: *alarm* raises an alarm, *covered* means the theorem includes recursive programs, *exact* interprets calls and returns exactly, and *no procedures* means the language has none. *Contexts* says how calls are distinguished: *per call site* reanalyzes a function at every call site, *object-sensitive* computes contexts by a function, *summaries* computes one summary per procedure, *set per call* lets the context policy return several contexts for one call, and *none* and *intraprocedural* mean that calls are not distinguished or not analyzed. *Fixpoint* says how the result is computed: *structural* iterates over the program's syntax, *iterator* over the whole annotated program, *checked* verifies the result of an untrusted iterator, *solver* and *TD solver* use a verified solver, and *specified* only states the constraints. *Executable* says whether the analyzer runs outside the prover. Chapter 15 gives the details.

the context policy as a parameter that may give several contexts to one call, is proved through a verified solver, and holds for the executable analyzer. It is also stated over source executions, whereas Verasco's theorem concerns C#minor, an intermediate language of CompCert. Voblint claims no better precision than any of these tools.

Voblint builds on prior work. The solver and the soundness proofs of its update rules come from the Isabelle formalization of Tilscher et al. [4]. Voblint uses a copy of it with changes made for this thesis (Section 14.3). The update rules come from Stemmler et al. [45], side-effecting constraint systems from Seidl et al. [43] and Apinis et al. [44], and the local/global analysis architecture from Goblint. Widening, narrowing, and the reduced product are well studied [46], [47]. The derivation of transfer functions from sound value operations follows Nipkow and Klein [7, Sects. 13.5, 13.7], and the activation traces adapt the local traces of Schwarz et al. [6] to procedure activations. Isabelle's code generator [48] produces the OCaml code the tools run.

1.3. Contributions

The contribution is the Isabelle/HOL formalization of Voblint and its machine-checked soundness proof, available at `github.com/ManuelLerchner/Voblint-Verification-Master-Thesis` and archived on Zenodo [49]. Figure 1.3 shows the chain of inclusions that the proof establishes at one program point, from source executions to the reported verdicts. Each step may add states that no execution reaches, which costs precision, but must not lose a reachable state. Errors such as a compiler that drops an edge or a return to the wrong caller would lose such a state, and the proof excludes them.



Figure 1.3: Soundness at a program point v as nested sets. A valid trace records, besides the stores of a graph run, which call created each activation, and every graph run is represented by one. Splitting the node collecting semantics by context loses no store. The source-level theorem chains these inclusions for every report the analyzer returns (`run_voblint_source_sound`, Chapter 10).

The formalization makes four contributions. The first is the end-to-end theorem itself. The other three make this theorem possible or show that it is not vacuous.

- *End-to-end soundness.* Whenever the analysis function `run_voblint` returns a report, its definite verdicts (`PROVED` and `REFUTED`) hold for every source execution from an initial store with zeroed globals that reaches the corresponding program point, under every analysis configuration the function offers (`run_voblint_source_sound`, Section 10.4). Companion theorems justify `DEAD` and the absence of arithmetic warnings (Chapter 10).
- *A concrete semantics of calling contexts.* An activation trace records the path of one activation together with the activation that called it. Therefore, the contexts that a

policy assigns to a call can be read off the concrete execution. A policy may assign several contexts to one call. Assume that an analysis claim meets the coverage contract. This contract includes that every call from a store the claim contains gets at least one context. Then the stores collected per context together are exactly the stores of the node collecting semantics, which ignores contexts (`node_collect_eq_Union_activation_collect`, Section 4.4).

- *Separately verified components.* Domains, context policies, analyses, and the solver are each verified against their own interface, and generic theorems compose them for every configuration. A numeric domain proves only its primitive operations sound, and the transfer functions derived from them are sound for every such domain (`nonrel_spec_sound`). A context policy proves only that its contexts agree with the concrete semantics and that every call from a store that the claim contains gets a context. Analyses that ask one another for facts can be combined under conditions on their answers (Chapter 7). The solver enters the proof only through proved facts about its result (Section 9.3.3).
- *Needed assumptions and nonempty results.* For several conditions of the proof, a theorem shows that dropping or weakening the condition admits an answer that a concrete execution contradicts. Examples are an analysis result that misses a store that some run reaches, or an abstract remainder that excludes the concrete one. Therefore, none of these conditions can simply be dropped (Section 13.2). One example is dropping the requirement that every call from a store the claim contains gets a context (`total_dropped_unsound`). Since a soundness theorem would also hold for an analyzer that answers `UNKNOWN` everywhere, further theorems show definite verdicts on concrete programs (Section 13.1) and that one context policy is strictly more precise than another on one program (Section 13.3).

1.4. Scope and limitations

Voblint is a Goblint-style analyzer with several context policies, over a small language with parameters, return values, and recursive procedures. It isolates the parts of Goblint's architecture that the proof is about: calling contexts, side-effecting constraint systems, configurable domains, analyses that answer one another's queries, and the top-down solver. It is not a verification of Goblint itself. *End-to-end* means that the proof reaches from source executions to the result that the analysis function returns. Figure 1.4 places each stage of the analyzer relative to the proof.

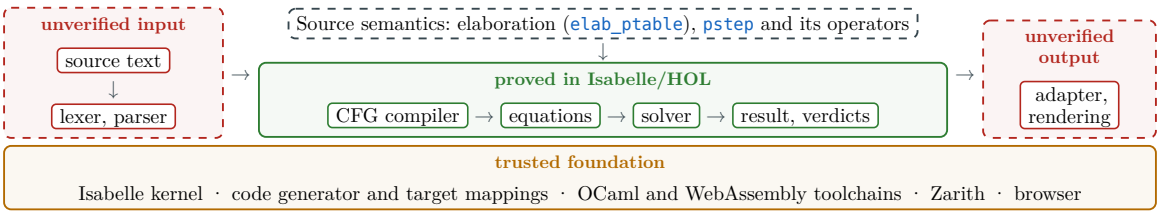


Figure 1.4: Where the proof starts and stops, for accepted programs whose solve terminates. The adequacy of elaboration and `pstep` is argued (Section 3.4). The handwritten adapters connect the generated code to the command line and the browser, Zarith is OCaml's arbitrary-precision integer library, and Section 12.5 gives the exact boundary.

The theorem has the following limits (Section 16.2).

- *Language and properties.* VIMP has no pointers, heap, or threads. Its integers have fixed widths and wrap around. Every operation is defined, including division by zero, and a local declared without an initializer holds some value of its kind. Therefore, a verdict does not always hold for a C program with the same text (Section 3.4). Voblint checks assertions and arithmetic hazards, while Goblint also checks data races, memory safety, memory leaks, and program termination [32, §3.3].
- *Adequacy and trusted components.* We argue, but do not prove, that elaboration and the core semantics `pstep` model the intended language (Section 3.4). The parser, the code generator, and the target toolchains lie outside the proof (Section 12.5).
- *Partial correctness.* The theorem covers every report the analyzer returns. Termination of the solve is not proved for every program (Section 9.6).
- *Precision and cost.* No completeness or general precision ordering is proved (Section 13.3). The executable uses the same data representations as the proofs, and we report no time or memory measurements (Section 16.2).
- *Goblint and evidence.* The correspondence to Goblint is architectural, and no theorem transfers to Goblint's implementation (Section 15.5). The regression corpus is small and was written or adapted for this work (Section 13.5).

1.5. Outline

After Chapter 2, the thesis follows the nested sets of Figure 1.3 from the inside out. Part II fixes what an analysis must over-approximate: VIMP and its compilation to a control-flow graph (Chapter 3), and activation traces indexed by calling context with the coverage contract that analyses must meet (Chapter 4). Part III develops what each analysis contributes, without contexts or a solver: the abstract domains (Chapter 5), the interface through which an analysis supplies its transfer functions (Chapter 6), and the combination of analyses that query one another (Chapter 7).

Part IV turns any analysis that meets this interface into a sound result. Chapter 8 generates the equations of a program, Chapter 9 computes their solution with the verified solver, and Chapter 10 composes these parts into the source-level theorem. Part V puts the theorem to use. Chapter 11 instantiates it for five domains and an order analysis. Chapter 12 follows `run_voblint` to the delivered tools and their trust boundary. Chapter 13 tests the premises and obligations, the precision, and the unverified code, and Chapter 14 describes the tooling. Part VI compares Voblint with prior work (Chapter 15) and closes with limitations and future work (Chapter 16).

2. Background

Chapter 1 described soundness informally: an analysis result must contain every state that a real run reaches, and it may contain more. Such a result is an *over-approximation*. To prove that Voblint computes one, we need precise definitions of the runs, of the abstract states, and of how an analyzer computes them from a system of constraints. We introduce them using one small program, which we follow from its executions to the result of the solver.

2.1. Executions and the collecting semantics

An analysis describes the states a program can be in at each of its points, so we first make the points explicit. A *control-flow graph* has the program points as nodes and an edge for every way control can pass from one point to the next, labeled with the assignment or condition executed on the way. Figure 2.1 shows the graph of a loop that counts `i` from 0 to 5, our running example. The loop becomes a cycle through h, b, and t, and its *loop head* h is the node through which control enters and exits the cycle.

```
1 i32 i = 0;                                    VIMP ↗
2 while (i < 5) {
3   i = i + 1;
4 }
```

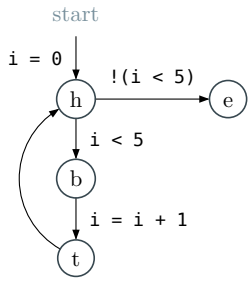


Figure 2.1: The counting loop and its control-flow graph (schematic). Nodes are program points: h is the loop head, b the start of the body, t the point after the increment, and e the point after the loop. `i32` declares a 32-bit signed integer. Edges carry the assignment or the condition that leads from one point to the next. The edge from t back to h has no effect.

The state of an execution is a *store*, which maps each variable to its value, here the value of i. An execution starts at the entry of the graph in an initial store and follows a path, updating the store at every edge. The counting loop reaches h once before the first iteration and again after each run of the body, so i takes exactly the values $0, 1, \ldots, 5$ there. Collecting, for every program point, the stores of all executions that reach it gives the *collecting semantics* $\mathcal{C}$ [46, §4], [50, §3.4], which maps every program point v to the set $\mathcal{C}(v)$ of stores reached there. Here $\mathcal{C}(h)$ holds the stores with $i \in \{0, \ldots, 5\}$. For programs with procedures, Chapter 4 defines Voblint's version over execution traces (`node_collect`).

The collecting semantics is the exact answer an analysis approximates. An analysis is *sound* if, at every program point, its result describes every store that the collecting semantics gives there. It may describe more stores, which costs precision, but never fewer.

Soundness thus relates three levels: the individual executions, the collecting semantics that gathers the stores they reach, and the abstract states that an analysis computes. Figure 1.3 refines these levels for Voblint.

2.2. Lattices and fixpoints

At h, the intervals $[0, 5]$ and $[0, 10]$ both contain every value i takes there (the collecting semantics $\{0, ..., 5\}$), but $[0, 10]$ also admits $i = 7$, which no execution reaches. A description is *more precise* than another if it admits fewer states. Ordered sets make this comparison formal. In an ordered set $(\mathbb{D}, \leq)$ (`order`), $a \leq b$ means that a is at least as precise as b [7, Def. 10.23]. For sets of concrete states, the order is inclusion.

Two descriptions reaching a point along different paths must be combined into one that covers both. Their *join* $a \sqcup b$ is the least upper bound, the most precise element above both (`semilattice_sup`). At the loop head, the value before the loop and the value after the body are combined this way. Dually, the *meet* $a \sqcap b$ is the greatest lower bound. An analysis uses it at a branch to restrict the incoming value to the values the condition admits. If the analysis knew only $[0, 10]$ at h, entering the body would restrict it to $[0, 10] \sqcap [-\infty, 4] = [0, 4]$. Soundness does not need this refinement, since $[0, 10]$ itself is sound at b, but it makes the result more precise, and Goblint and Voblint both refine at branches. The least element $\bot$ lies below every element, and a top element $\top$, if one exists, lies above every element. In abstract interpretation, $\bot$ describes no states and $\top$ describes all of them. A *complete lattice* (`complete_lattice`) has a join $\bigsqcup X$ and a meet $\bigsqcap X$ for every subset X [7, Def. 13.1], [50, Def. 2.5].

Voblint's solver, however, requires only a join semilattice with a least element (`bounded_semilattice_sup_bot`) that carries the widening and narrowing operators of Section 2.5, so joins need to exist only for pairs. A numeric domain adds a top element. Refinement at a branch adds a pairwise intersection that keeps every value both operands admit (`meet_ik_sound`, Section 5.6).

An analysis computes the description at a point from the descriptions at its predecessors, by a function f on $\mathbb{D}$. The classical theory requires f to be *monotone* (`mono`), so that $a \leq b$ implies $f(a) \leq f(b)$ [7, Def. 10.26], [50, Def. 2.9] and more precise input never yields less precise output. For a monotone f, the iterates $\bot, f(\bot), f(f(\bot)), ...$ form an ascending chain. Each of them lies below every fixpoint. The verified solver that Voblint reuses, however, does not assume monotonicity.

A *fixpoint* of f is an element with $f(d) = d$, and a *post-fixpoint* is an element with $f(d) \leq d$, so applying f once more yields nothing above d. By the Knaster–Tarski theorem, a monotone f on a complete lattice has a least fixpoint $\operatorname{lfp} f$, the meet of all post-fixpoints [51, Thm. 1] (`lfp`, `lfp_unfold`), so every post-fixpoint lies above $\operatorname{lfp} f$. For a monotone f on a finite-height order, the iteration from $\bot$ stabilizes after finitely many steps [52, §1.5], and an iterate that no longer changes is the least fixpoint (`lfp_Kleene_iter`).

An analysis does not have to reach $\operatorname{lfp} f$. Every post-fixpoint of an f that includes the initial states and approximates each step soundly already contains the collecting semantics (Section 2.3).

2.3. Abstract interpretation

The correctness of an abstract result needs a meaning for its elements. Let U be a set of concrete objects, such as integers or stores (`store`), and A an *abstract domain*, an ordered set of *domain elements* whose underlying set is its *carrier*. A domain element is an *abstract value* when it describes the values of one variable, such as an interval for one integer, and an *abstract state* when it describes stores. A concretization $\gamma : A \to \mathcal{P}(U)$ gives each domain element its meaning. It is monotone, so a less precise element admits at least the same objects: $a \leq b$ implies $\gamma(a) \subseteq \gamma(b)$ (`gamma_mono`). Soundness at a program point v (Section 2.1) then becomes an inclusion. If a is the analysis result at v, then

$$\mathcal{C}(v) \subseteq \gamma(a),$$

the consistency requirement between an abstract and a concrete interpretation in [46, §6], [50, Def. 2.11].

The running example uses the *interval domain* [46, §9.2], [50, §4.5]. Its elements are $\bot$ and the intervals $[l, u]$ with $l \in \mathbb{Z} \cup \{-\infty\}$, $u \in \mathbb{Z} \cup \{+\infty\}$ and $l \leq u$, and an interval denotes the integers between its bounds:

$$\gamma([l, u]) = \{n \in \mathbb{Z} \mid l \leq n \leq u\}, \quad \gamma(\bot) = \emptyset.$$

The order is inclusion of these sets, so $[l, u] \leq [l', u']$ holds exactly when $l' \leq l$ and $u \leq u'$. The join $[l, u] \sqcup [l', u'] = [\min(l, l'), \max(u, u')]$ is the smallest interval containing both. An interval cannot have holes, so the join may contain values that neither operand contains: $[0, 0] \sqcup [5, 5] = [0, 5]$. The meet intersects the bounds. This is the textbook domain. Voblint's representation `ivl` admits every pair of bounds, so $\bot$ is one of several empty pairs (Section 5.1).

At the loop head, both $[0, 5]$ and $[0, 10]$ are sound (Figure 2.2), as is every interval that contains $[0, 5]$. An analysis aims for a precise one, because a more precise element admits fewer values and so decides more checks: $[0, 5]$ proves `i <= 5` at the loop head, and $[0, 10]$ does not.

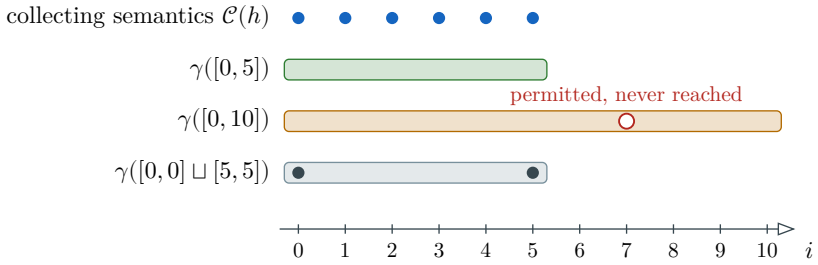


Figure 2.2: Concretization of intervals at the loop head. Both $[0, 5]$ and $[0, 10]$ contain the collecting semantics $\mathcal{C}(h)$, and $[0, 10]$ also permits $i = 7$. The join of $[0, 0]$ and $[5, 5]$ adds the values in between.

Much of abstract interpretation also assumes an abstraction function $\alpha : \mathcal{P}(U) \to A$ that forms a *Galois connection* with γ and gives every set of states a best abstraction [47, §5.3], the most precise domain element whose concretization contains the set. Not every useful domain has such best abstractions, and Cousot and Cousot treat a Galois connection as the ideal case that efficient representations do not always meet [53, §5]. Voblint uses γ alone, like the verified analyzers of Jourdan et al. and of Nipkow [34, §2], [54, §5.2], since its soundness theorem claims coverage and nothing about optimal precision. The

concretization exists only in the logic. Since its result is in general an infinite set, it is never executed, and the analyzer itself is defined from executable operations only, so that it can be exported and run as a standalone program.

The analysis replaces each concrete operation by an abstract one, a *transfer function* $f^\sharp : A \to A$, which must not lose any state that the concrete operation can produce. In general, a program's steps form a relation $\to$ on U, and one state may have several successors, even infinitely many, for example, when the next value is read from user input. The *post-image* $\operatorname{post}(X) = \{s' \mid \exists s \in X. s \to s'\}$ collects the successors of all states in X. The transfer function is sound if, for every a,

$$\operatorname{post}(\gamma(a)) \subseteq \gamma\big(f^\sharp(a)\big),$$

so stepping from any state that a describes leads to a state that $f^\sharp(a)$ describes [17, §4.2.2], [46, §6], [50, Def. 2.15]. The square of Figure 2.3 thus commutes only up to inclusion. In the counting loop, the edge from b to t executes `i = i + 1`. Over $\mathbb{Z}$, the interval transfer $[l, u] \mapsto [l + 1, u + 1]$ is sound for it, and applied to $[0, 4]$ it yields $[1, 5]$, which contains every successor $i = 1, \ldots, 5$. In VIMP, `i` is a 32-bit integer whose increment wraps around at the largest value. Voblint's transfer accounts for this. If a result leaves the range, it returns the whole range of the type. If the configuration lets the type wrap and the wrapped bounds still form one interval, it returns the wrapped interval instead. Soundness composes along paths [50, Thm. 2.6], and at control-flow joins the upper-bound laws of the join keep the states from both predecessors (`gamma_sup_ub1`, `gamma_sup_ub2`).

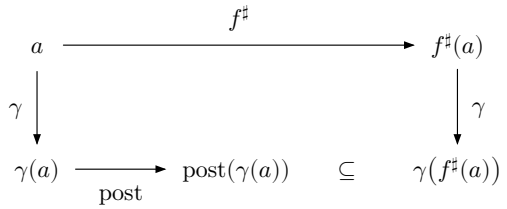


Figure 2.3: Soundness of a transfer function $f^\sharp$: the square commutes up to inclusion.

Sound transfer functions make every post-fixpoint that contains the initial states a sound result [50, Thm. 2.8]. The standard argument uses one domain element that describes the whole program. It works with *configurations*, each a program point together with a store, so that one relation $\to$ describes every step of the program, each moving an execution along one edge. The constraint systems of Section 2.4 then split this element into one abstract state per program point. In the counting loop, a configuration is a pair (p, n) of a program point and the value of i, so $U = \{\text{start}, h, b, t, e\} \times \mathbb{Z}$. With the set I of initial configurations, the reachable configurations form the least fixpoint of

$$F(X) = I \cup \operatorname{post}(X)$$

[17, §4.1.1, Thm. 4.1]. A set with $F(X) \subseteq X$ contains the initial configurations and the successors of all its elements, so no execution can leave it, and $\operatorname{lfp} F$ is the smallest such set. In the counting loop, executions begin at the node `start`, with any value of i, so $I = \{(\text{start}, n) \mid n \in \mathbb{Z}\}$. Iterating F from $\emptyset$ adds one step at a time: $(h, 0)$, then $(b, 0)$, $(t, 1)$, $(h, 1)$, and so on, up to $(h, 5)$ and finally $(e, 5)$. The least fixpoint contains (h, n) exactly for $n \in \{0, \ldots, 5\}$, which is $\mathcal{C}(h)$, since $\mathcal{C}$ groups the reachable configurations by their program point.

Now take a domain element a with $I \subseteq \gamma(a)$ and $f^\sharp(a) \leq a$, for a sound $f^\sharp$. Soundness of $f^\sharp$ and monotonicity of γ give $F(\gamma(a)) \subseteq I \cup \gamma\big(f^\sharp(a)\big) \subseteq \gamma(a)$. So $\gamma(a)$ is a post-fixpoint of F and, by Knaster–Tarski, contains $\operatorname{lfp} F$, every reachable configuration. Here F works on sets of configurations and $f^\sharp$ on domain elements, and the soundness of $f^\sharp$ relates the two through γ.

Voblint's proof uses the same principle on a set of execution traces defined inductively as a least fixpoint, and proceeds by induction over its rules. Analyzers such as Goblint keep one abstract state per program point [17, §4.2.1] and state one inequality per point, whose solution plays the role of a.

2.4. Constraint systems

An analysis needs an abstract state at every program point, and the state at one point depends on the states at its predecessors. The analysis therefore states its result as a system of constraints. Each *unknown* of the system stands for the abstract state at one program point, and each constraint says what the unknown must cover.

The counting loop has one variable, so an abstract state is an interval for i. (With several variables, a *non-relational* state maps each variable to its own abstract value.) With one unknown per program point of Figure 2.1, each unknown must cover what the edges entering its point produce:

$$h \geq [0,0] \sqcup t, \qquad b \geq h \sqcap [-\infty, 4],$$
$$t \geq b +^\sharp [1,1], \quad e \geq h \sqcap [5, +\infty].$$

The assignment `i = 0` yields $[0,0]$ at h, the condition `i < 5` restricts the branch into the body, and its negation restricts the exit. Substituting b and t into the first inequality gives one for the loop head, $h \geq f(h)$, with

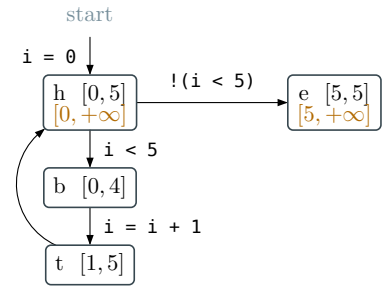
$$f(d) = [0,0] \sqcup \big((d \sqcap [-\infty, 4]) +^\sharp [1,1]\big),$$

whose post-fixpoints appear in Figure 2.4b. A solution assigns an interval to every unknown such that all four inequalities hold, and Figure 2.4a shows two. Both contain the collecting semantics ($\{0, \ldots, 5\}$ at h), and the second is less precise at h and e, so a solution need not be the least one.

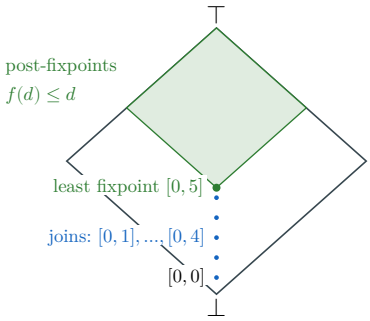
In general, let $\mathcal{X}$ be a set of unknowns and $\sigma : \mathcal{X} \to A$ a *valuation*, which assigns a domain element to every unknown. Valuations are ordered pointwise: $\sigma \leq \sigma'$ if $\sigma(x) \leq \sigma'(x)$ for every unknown x. Each unknown x has a *right-hand side* rhs_x, a function of the valuation, and σ is a *post-solution* if

$$rhs_x(\sigma) \leq \sigma(x) \quad \text{for every unknown } x$$

(`post_solution`), what the constraint-system literature calls a solution [32, §2.4.4]. For the loop head, $rhs_h(\sigma) = [0,0] \sqcup \sigma(t)$. A post-solution is a post-fixpoint of all right-hand sides at once, so the argument of Section 2.3 applies at every program point, and a post-solution bounds the reachable states everywhere. We use the term *constraint system* for the general concept and the term *equation system* for the system that Voblint generates and the solver receives, although soundness only requires inequalities.



(a) two solutions on the graph



(b) post-fixpoints of $h \geq f(h)$

Figure 2.4: The counting loop's inequalities, solved by hand. (a) The least solution at each node of Figure 2.1 and, in orange where it differs, a less precise solution. (b) Post-fixpoints of the loop-head constraint (schematic, shaded) and its least fixpoint $[0, 5]$.

The formulation goes back to data-flow analysis, where Kildall computes one value per node of a program graph by iteration [55, §3].

Programs with procedures add a choice. A procedure that is called from several places can be analyzed once for all its calls, with one unknown per program point, or separately per *calling context*, an abstraction of how an *activation* was reached. An activation is one execution of a procedure from its call to its return [56, §7.2.1]. Sharir and Pnueli distinguish the call-string approach, whose contexts record the most recent call sites, from the functional approach, which relates the abstract state at entry to the state at the return [57]. In the constraint-system formulation of Apinis et al., a context-sensitive analysis has one unknown $[v, c]$ per program point v and context c [44, §3]. Recall the calls `bump(5)` and `bump(4)` of Chapter 1. With contexts that record the argument at entry, the unknowns at the point where `bump` returns can claim 6 and 5 separately, whereas a single unknown there must cover both results.

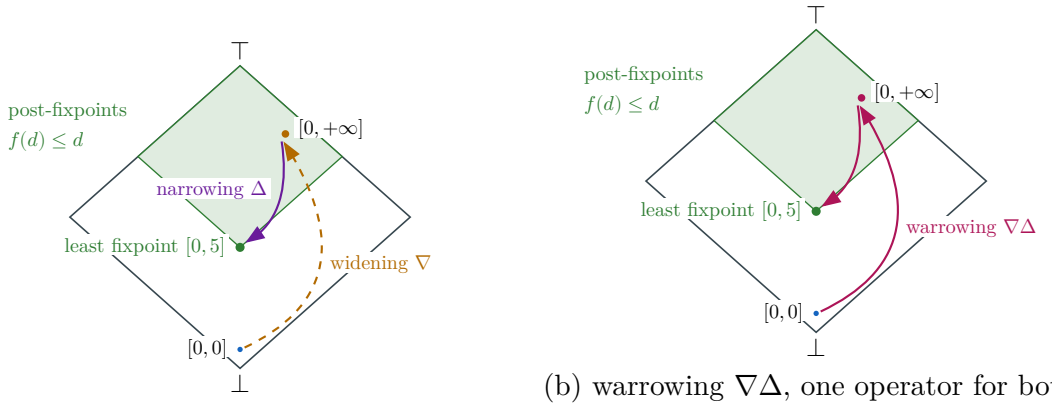
2.5. Widening and narrowing

An analysis should terminate in reasonable time, even on a program whose executions do not terminate. With joins alone, successive abstract iterates can mirror successive loop iterations. At the example's loop head, joins take five increases after $[0, 0]$. For `while (true) { i = i + 1; }`, the iteration over intervals with bounds in $\mathbb{Z}$ never stabilizes, because these intervals have infinite ascending chains such as

$$[0, 0] < [0, 1] < [0, 2] < \ldots.$$

With signed 32-bit bounds, a corresponding chain is finite, but it can still have more than two billion elements.

A *widening* $a \nabla b$ replaces the update from the old value a to the new value b by a guess that lies above both, which keeps the result sound (`widening`). A classical widening is also chosen so that repeated widening stabilizes after finitely many steps [46, §9.1.3], [53, §4], which the upper-bound law alone does not ensure. A common strategy widens at loop heads. The standard interval widening replaces a growing bound by $+\infty$ [46, §9.2], [50, §4.5.2], so widening at h turns the first change there, from $[0, 0]$ to $[0, 1]$, into $[0, +\infty]$ at once. Propagated through the loop, this value yields the less precise solution of Figure 2.4a, which loses the bound 5 of the loop condition `i < 5`.



(a) widening, then narrowing

(b) warrowing $\nabla\Delta$, one operator for both steps

Figure 2.5: Extrapolation on the lattice of Figure 2.4b. (a) Widening jumps from $[0, 0]$ to the post-fixpoint $[0, +\infty]$, and narrowing then recovers the least fixpoint $[0, 5]$. (b) Warrowing takes the same two steps with one operator, which widens while the candidate grows and narrows once it lies below the current value. Here both reach $[0, 5]$.

The branch into the body restricts $[0, +\infty]$ to $[0, 4]$, the body yields $[1, 5]$, and evaluating the head again gives $[0, 0] \sqcup [1, 5] = [0, 5]$. A widened iteration only grows and would not take this smaller value. A *narrowing* lets it descend. Both steps appear in Figure 2.5a, and Figure 2.5b shows one update operator that combines them, widening or narrowing depending on the candidate. The example uses intervals over $\mathbb{Z}$. For VIMP's fixed-width integers, Voblint's widening moves a growing bound to the type's limit instead of $+\infty$, which loses the bound 5 in the same way and needs narrowing just as much.

A narrowing $a\Delta b$ combines the current value a with a candidate $b \leq a$ and returns a value between the two [46, §9.3.4], [53, §4]:

$$b \leq a\Delta b \leq a.$$

These are the *bracket laws* of the solver's class `narrowing`. A classical narrowing must also make repeated narrowing stabilize. The class omits this law, since termination of the solver is not proved. In the counting loop, the interval narrowing reaches $[0, 5]$, but in general, narrowing need not recover the least fixpoint.

Soundness of the result does not depend on how the solver widens and narrows. The verified TD solver applies both operators in a single update, *warrowing* (`warrow`). It narrows when the candidate lies below the current value and widens otherwise [58], [59, §3], [60], so each update lies above the candidate (`warrowing_properties`). An unknown whose value no longer changes under this update therefore satisfies its inequality. The solver's correctness theorem uses only this and holds whether or not the right-hand sides are monotone (Section 2.7). Monotonicity is thus needed for the classical theory of least fixpoints, but not to certify that a returned valuation is a post-solution. As long as the operators satisfy their laws, the choice of widening and narrowing affects only precision and termination.

2.6. Side-effecting constraint systems

The analyses so far are *flow-sensitive*: they respect the order in which statements execute and therefore compute a separate abstract state for every program point. For some facts,

one value for the whole program is enough. A *flow-insensitive* analysis ignores execution order and keeps one abstract value for an aspect of the state across the program, for example, one range for a global variable, and mixed analyses choose between the two per aspect [2].

A flow-insensitive fact needs an unknown of its own that belongs to no program point, and every point that changes the fact has to feed its new value into that unknown. An ordinary right-hand side for the unknown would have to list all these points in advance, which is not always possible. In a context-sensitive analysis, for example, an unknown that writes a global is a pair of a program point and a calling context, and the contexts are discovered only while the system is solved.

Side-effecting constraint systems avoid that list. Seidl, Vene, and Müller-Olm introduced them for multithreaded programs [43, §3], and Apinis et al. use them for context-sensitive analysis [44]. While the right-hand side of one unknown is evaluated, it may *publish* a *contribution* to another unknown as a *side effect*. We follow the verified solver and let side effects target only *global unknowns* $\mathcal{X}_G$, which have no right-hand side and receive only contributions. From here on, $\mathcal{X}$ denotes the *local* unknowns, which keep their right-hand sides, and a valuation σ assigns values to both kinds. A global unknown need not correspond to a global variable of the program. A right-hand side then returns a pair [32, §2.4.5], $rhs_x : (\mathcal{X} \cup \mathcal{X}_G \to A) \to (\mathcal{X}_G \rightharpoonup A) \times A$, of a partial map ρ of the contributions it publishes, joined per global unknown, and its own value d. If $rhs_x(\sigma) = (\rho, d)$, a post-solution must bound both, $d \leq \sigma(x)$ and $\rho(g) \leq \sigma(g)$ for every $g \in \mathrm{dom}(\rho)$. If the assignments `g = 5` and `g = 4` publish $[5, 5]$ and $[4, 4]$ to the unknown of a flow-insensitive global `g`, that unknown must bound their join $[4, 5]$. This describes what a right-hand side computes. The verified solver represents its evaluation by a strategy tree (Section 2.7), so that it discovers reads and side effects while it solves.

With contexts as indices, a system can have infinitely many unknowns, of which only those that influence the *query*, the unknown whose value is asked for, matter. A *local* solver therefore explores the system from the query and solves only the unknowns that it meets [61]. Its result comes with a *certificate*, the property `part_post_solution`, which requires the bounds above only on a set of unknowns that contains the query and is closed under the local reads of its right-hand sides.

2.7. The verified top-down solver

A straightforward implementation of Kleene iteration recomputes every unknown in every round, although most values no longer change. Practical solvers therefore update one unknown at a time. They work in rounds or with a worklist of unknowns whose inequalities may be violated [52, §§1.5, 1.12], in orders chosen for widening [62], or locally, on demand from a query, as the solver RLD does [63]. Voblint reuses the verified local top-down solver (TD) of [3], [4].

TD starts from the query and evaluates right-hand sides on demand. When the right-hand side of x reads an unknown y, TD first solves y and records that x depends on y. An unknown is *stable* if it has been evaluated and no unknown that it read has changed since then. When the value of y changes later, every unknown that read it is *destabilized* and evaluated again. The unknowns at which a read closes a cycle are the solver's *widening points* [61]. TD detects them while it solves [59, §4], so they need not coincide with the

loop heads of the control-flow graph. At a widening point, it combines the old and the new value with warrowing. The playground replays this run on the counting loop step by step. Contributions published to a global unknown are merged into its value by an *update rule* (`update_rule`), which keeps one record per *origin*, the unknown whose right-hand side published the contribution (Section 9.4). The run ends once every unknown the query transitively reads is stable, and it returns this *stable set* V together with the valuation σ.

To solve on demand, TD has to discover which unknowns a right-hand side reads, so that it can solve each of them first. In the counting loop, the reads are fixed, since the right-hand side of h always reads t. In general, a right-hand side may choose the next unknown from a value it has already read. At a procedure call, for instance, the callee context to read is known only once the caller's value has been read. The verified solver therefore receives every right-hand side as a *strategy tree* (`strategy_tree`), a small program that reads one unknown at a time and makes every read and every side effect explicit:

$$\tau \coloneqq \mathsf{Answer}(d) \mid \mathsf{QueryL}(y, k) \mid \mathsf{QueryG}(g, k) \mid \mathsf{Side}(g, d, \tau)$$

with a local unknown $y \in \mathcal{X}$, a global unknown $g \in \mathcal{X}_G$, a domain element $d \in A$ and a continuation $k : A \to \tau$ that receives the value read. $\mathsf{Answer}(d)$ returns d as the value of the right-hand side, QueryL and QueryG read a local or a global unknown, and $\mathsf{Side}(g, d, \tau)$ publishes d to g before continuing with τ. Figure 2.6 shows the trees of the counting loop and of a flow-insensitive assignment.

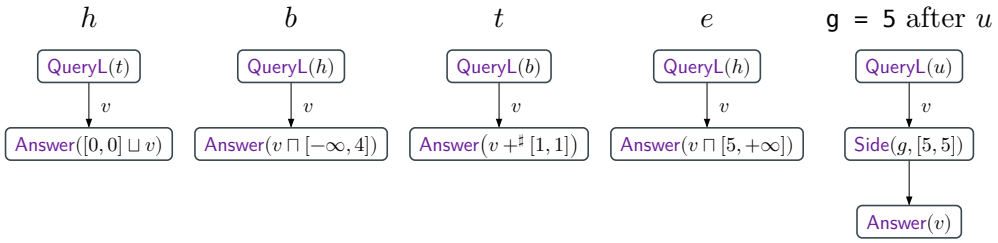


Figure 2.6: Right-hand sides as strategy trees. The first four are the inequalities of the counting loop (Section 2.4): each reads one unknown, passes its value v on, and answers. The last publishes the value of a flow-insensitive global `g` (Section 2.6) before answering.

The solver's main theorem, `partial_post_solution`, establishes the certificate `part_post_solution` of Section 2.6 and states partial correctness. If the run terminates on the query x and returns its stable set V and the valuation σ, then V contains x, is closed under the local reads of its right-hand sides, and on V the valuation is a post-solution, published contributions included. The theorem needs no monotonicity of the right-hand sides and says nothing about unknowns outside V. We call the set on which this certificate holds the *solved set*. Termination of the solver is not proved.

HOL functions are total, but Isabelle's `function` package also accepts a recursive definition without a termination proof. Its defining equations then hold only on a *domain*, the arguments on which the recursion terminates. The solver is defined this way, with the domain `solve_dom`, and its correctness theorem assumes membership in it. Voblint proves that every completed run of the executable form `solve_c` lies in the domain (`solve_dom_of_solve_c`, Section 9.6). The end-to-end theorem therefore needs no termination premise. It applies whenever `run_voblint` returns a report.

PART II

What Must Be Over-Approximated

3. Programs and Control-Flow Graphs

Chapter 2 analyzed a single loop through its control-flow graph. The soundness theorem, however, should talk about the programs a user writes, and these have procedures and recursion. This chapter defines two views of such a program and connects them. The source semantics executes a program directly (Section 3.3). An analysis cannot generate its equations from it, because its equations need a fixed, finite set of program points, which contexts may then refine (Section 2.4), and under recursion the commands that remain to run nest arbitrarily deep (Section 3.5.1). Voblint therefore compiles the elaborated program to a procedure-aware control-flow graph and generates its equations from the graph (Figure 3.1). A soundness proof for the graph alone would leave the compiler outside the theorem. This chapter therefore proves the first link of the soundness chain: every source run of a well-formed program is matched by a graph run that holds the same store (Section 3.7). The program of Figure 3.1, in which `main` calls `bump(5)` and `bump(4)`, is the running example of the remaining chapters.

```
1  fun i32 bump(i32 n) {  VIMP
2    return n + 1;
3  }
4
5  fun main() {
6    i32 a = bump(5);
7    i32 b = bump(4);
8    __voblint_check(a == 6);
9    __voblint_check(b == 5);
10 }
```

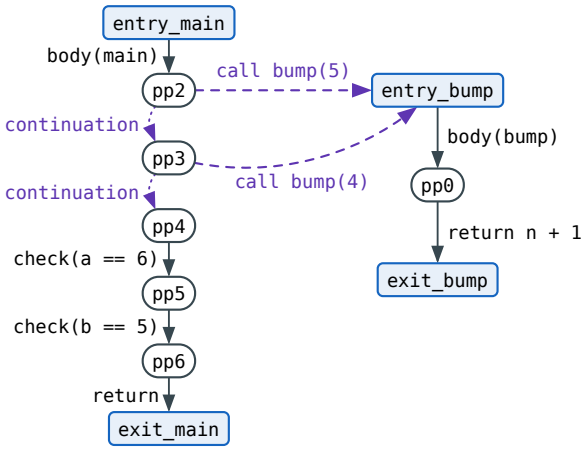


Figure 3.1: The running example and its procedure-aware graph, with node and edge labels as `voblint --dot` prints them. Solid arrows are local edges and dashed arrows are call edges. Dotted connectors are not edges. They show the continuation of a call edge, that is, the node where the caller resumes. The edge `return` into the result node of `main` is the return without a value by which `main` falls through (Section 3.6.1). The drawing omits the corresponding node `pp1` of `bump` and its edge, which no run reaches because `bump` always returns explicitly.

3.1. VIMP, and what it leaves out

VIMP (Figure 3.2) is the small imperative language with C-like syntax that the formalization analyzes. It is kept small because every construct needs a semantics, compiler clauses, and a transfer function proved sound in every abstract domain. It has integer variables

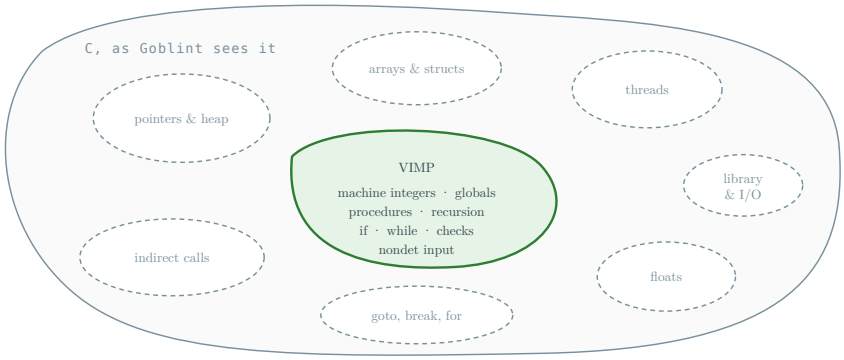


Figure 3.2: Constructs represented in VIMP relative to Goblint's C input, taken from the explainer page. Dashed regions are constructs that VIMP omits. Sizes and positions are schematic. The figure compares only syntax. Section 3.4 lists where shared constructs have different meanings.

of C's machine integer types, locals and globals, procedures with value parameters, direct calls with an optional result, recursion, assignment, sequencing, conditionals, `while` loops, nondeterministic input, and checks. It omits pointers and the heap, arrays and structs, indirect calls, threads, floating point, `goto`, `break`, `continue`, and exceptions. Because VIMP has no pointers, concrete memory is a store that maps variables to integers.

Expressions are integer-valued. As in C11, comparisons and logical operators yield 0 or 1 (§6.5.8p6, §6.5.9p3, §6.5.3.3p5, §6.5.13p3, §6.5.14p3), and a condition holds when it is nonzero (`truthy`, §6.8.4.1p2, §6.8.5p4).

From here on, we show definitions and theorems in Isabelle/HOL [7], a typed logic of total functions. We write $x :: \tau$ for "x has type τ", $\tau_1 \Rightarrow \tau_2$ for the type of functions from τ_1 to τ_2, `'a` for a type variable, and $\lambda x.t$ for the function that maps x to t. A *datatype* declares a type by its constructors and comes with case distinction and structural induction, and a *record* is a tuple with named fields. A theorem declares its variables after `fixes`, local abbreviations after `defines`, its premises after `assumes`, and its conclusion after `shows`. Inside a proposition, $A \Longrightarrow B$ is an implication. Subscripts, superscripts, and symbols such as $\vdash$ are notation for a constant applied to its arguments, and the text names the constant at first use.

Definition 3.1 (Surface expressions, `datatype src_exp`)**.**

$$\begin{aligned} e ::= {} & n \mid x \mid e + e \mid e - e \mid e * e \mid e \,/\, e \mid e \,\%\, e \\ & \mid e < e \mid e <= e \mid e > e \mid e >= e \mid e \,!= e \mid e == e \\ & \mid\, !e \mid e \,\&\&\, e \mid e \,||\, e \mid (k)\, e \end{aligned}$$

Here k is a C integer type such as `u8` or `i32`, and $(k)\, e$ converts e to it explicitly.

Each operator has its own constructor, because an abstract domain chooses its transformer by the operator, and the executable analyzer makes this choice by inspecting the syntax. The two AFP languages with procedures that we considered do not allow this. IMP2 [64] represents a binary operator by a HOL function of type `int ⇒ int ⇒ int`, which executable code cannot compare with another one, so it cannot tell `+` from `*`. Its semantics is also deterministic, without input, and its calls take no arguments and return no value. Schirmer's Simpl [65] is more expressive, but its basic commands and conditions are

likewise HOL functions and sets of states. Both languages target program verification, where semantic objects are convenient. An analyzer needs syntax that it can inspect, so VIMP is defined from scratch with such a syntax, which code generation can also export.

Besides the structured statements, commands include calls, which may store a result, and two markers that appear only during execution.

Definition 3.2 (Commands, `datatype gcom`)**.**

$$c ::= \textbf{skip} \mid x := e \mid \textbf{check}\ (e) \mid c;\ c \mid \textbf{if}\ (e)\{c\}\ \textbf{else}\ \{c\} \mid \textbf{while}\ (e)\{c\}$$
$$\mid [x :=]\ p(e, ..., e) \mid \textbf{return}\ e? \mid \text{Restore} \mid \text{Unwind}$$

Every constructor except sequencing also carries an annotation i, omitted above. The semantics never reads it, and parsed and elaborated programs carry the trivial annotation $()$. A step copies the annotation of the command it reduces onto what it leaves behind, so the Restore marker of a call and the **skip** that the finished call leaves carry the annotation of that call. The simulation of Section 3.7 uses this to recognize a finished call.

The command type `gcom` is generic in the variables and expressions it contains, which the prefix g marks, and two instances matter. A parsed program is a `src_com`, with names as variables and surface expressions. A translation called elaboration turns it into a core command (`com`), in which every operation carries its integer type.

In concrete syntax, an assignment is written `x = e;` and a check `__voblint_check(e);`. The parser lowers `true`, `false`, and unary minus to the expression grammar above. A check states a condition for the analyzer to decide. It behaves like **skip** whatever value e has, so it neither stops the run nor refines the store. It also carries its source position as a label, which the semantics ignores and the report uses to name the check.

A call of a declared procedure starts an *activation*, one execution of the procedure body from its call to its return. VIMP also has three built-in *library calls*, `__voblint_nondet_int`, `min`, and `max`, which a program calls without declaring. A library call finishes in one step without starting an activation. It stores a value the call may return (`special_result`), any integer for `__voblint_nondet_int` and the minimum or maximum of the arguments for `min` and `max`, converted to the destination's integer type. Restore and Unwind are not source syntax. They appear only during execution, where they implement calls and returns (Section 3.3).

A program (`imp_prog`) is a list of procedure declarations, each with formal parameters and a body, plus a list of global variables. Its procedure table (`proc_table`) looks up a name in that list. The entry procedure is the declaration named `main`, and `main_body` is its body.

3.2. Machine integers and elaboration thy

Like a C program, a VIMP program declares its variables with integer types and leaves most conversions implicit, so the result of an operation depends on the width at which it is computed. In the program below, `x` and `y` are 8-bit unsigned variables.

```
fun main() {                                                    VIMP
  u8 x = 200;
  u8 y = 100;
  u8 z = x + y;
  __voblint_check(x + y < 250);
  __voblint_check(z < 250);
}
```

Read over mathematical integers, both checks compare 300 with 250 and fail. Read at the variables' own width, both compare $(200 + 100) \bmod 2^8 = 44$ and hold. C answers differently for the two checks. It first converts both 8-bit operands to `int` (the integer promotions) and adds at that width, so the first check compares $300 < 250$ and fails. The assignment to `z` then converts 300 back to 8 bits, so `z` holds 44 and the second check holds. The analyzer reports the same, the first check as `REFUTED` and the second as `PROVED`, and it reports an arithmetic diagnostic for the conversion of 300 to `u8`, of severity error because every run that reaches the assignment overflows (Section 10.5). Had the program declared all three variables `i32`, `z` would hold 300 and the second check would be `REFUTED` (claim `kinds-i32`). A verdict is therefore only meaningful together with the width at which each operation computes.

Machine kinds. A variable has one of eight *machine kinds* (`ikind`): the signed and unsigned integers of 8, 16, 32 and 64 bits, written `i8`, `u8`, `i16`, `u16`, `i32`, `u32`, `i64` and `u64`. A kind k of width w fixes a range, $[-2^{w-1}, 2^{w-1} - 1]$ if k is signed and $[0, 2^w - 1]$ if it is unsigned. Values remain mathematical integers, and a kind restricts which of them a variable may hold. Wraparound maps every integer n into the range of k by two's complement. With m the least value of the range,

$$\texttt{ik_norm}\, k\, n = ((n - m) \bmod 2^w) + m$$

(`ik_norm_mod`). For example, 300 wraps to 44 at `u8`. Wraparound applies to signed kinds too, whose overflow C leaves undefined (§6.5p5).

A program declares kinds in C's style, for instance `global u8 g;` for a global, `fun u16 f(u8 n)` for a formal and a return value, and `i32 x = 0;` for a local, which may appear anywhere in a body and whose scope ends with its enclosing block. Every variable and every procedure that returns a value declares its kind, and the parser rejects a name that no declaration in scope introduces.

Identities and stores. A name alone does not determine a kind, because two procedures may both have a formal `n`, one of kind `i8` and one of kind `u32`. The semantics therefore stores values under *identities* (`var_id`), which qualify a formal or local by its procedure and leave a global unqualified. Two further identities cannot be named by any program. The *return slot* ReturnVar carries a returned value back to the caller. The identity ReturnId p holds no value. The kind environment uses it to record the kind that p returns. Two procedures share an identity only for a global (`resolve_var_eq_iff`). Block scopes never reach the formal language, because the parser renames every shadowing declaration to a fresh name such as `x#2` and resolves every use to the innermost declaration in scope (`26-machine-kinds/15-block_scopes.vimp`). The parser's scope resolution and the kinds it declares are trusted, since no theorem covers the parser (Section 12.5). A store s (`store`) is a total function from identities to integers, and a kind environment κ (`kenv`) gives each identity its kind.

Elaboration. The parser builds *surface* VIMP, the expressions of Section 3.1 over names, with implicit conversions. The semantics executes *core* VIMP (`exp`), in which every

operation carries the kind it computes at, every variable read names an identity, and every conversion inside an expression is explicit. Only a call's result has no conversion expression, and the semantics converts it to the kind of the call's destination (Section 3.3). *Elaboration* (`elaborate`) translates surface into core VIMP by C's rules (§6.3.1.1p2, §6.3.1.8p1). An arithmetic operator computes at the common kind that the integer promotions and the usual arithmetic conversions determine, and promotion turns every kind narrower than 32 bits into `i32`. A comparison converts both operands to their common kind and yields 0 or 1 at `i32`, and a logical operator tests each operand at its own kind. A conversion is inserted wherever a value meets a context of another kind and omitted where the kinds agree, because even a conversion that changes no value costs precision in a domain that records only signs. assignment to `z` above thus becomes `z := (u8) ((i32) x + (i32) y)`.

Goblint's front end CIL makes the same translation for C [5] (`doBinOp`, `mkCastT`). It differs in lowering `&&` and `||` into branches, where VIMP evaluates both operands, and in being unverified OCaml.

A literal takes the kind of its context, so `u8 z = 300;` stores 44. The literal keeps its value 300 in the core expression, so the analyzer still reports the conversion as an arithmetic error. Where a literal decides the kind of an operation, it counts as `i32` or, if it does not fit, as `i64` (§6.4.4.1p5). Commands are elaborated in the same way. A written value is converted to the kind of its target, the variable at an assignment, the formal at an argument, and the return kind at a **return**. A condition or a check is evaluated at its own kind, and the arguments of `min` and `max` convert to their common kind, as the operands of C's `MIN` macro do. Figure 3.3 shows each rule on one program. Elaborating every body gives the program's core procedure table $\Pi = \texttt{elab_ptable}\ p\ ds$, in which every operation receives operands of its own kind and every written value has the kind of its target (`wt_com_elab_com`).

source text

```
1  fun u16 f(u8 n) {                VIMP ↗
2    return n + 1;
3  }
4
5  fun main() {
6    u8 x = 200;
7    u8 z = x + 100;
8    i32 s = -1;
9    u32 u = 1;
10   i32 b = s < u;
11   u16 r = f(x);
12   __voblint_check(b == 0);
13 }
```

surface VIMP (parser)

```
f:
  return n + 1
main:
  x := 200
  z := x + 100
  s := -1
  u := 1
  b := s < u
  r := f(x)
  check(b == 0)
```

```
kinds:
f: return u16, n u8
main: x u8, z u8
      s i32, u u32
      b i32, r u16
```

core VIMP (elaboration)

```
f:
  return (u16) ((i32) n + 1)
main:
  x := 200
  z := (u8) ((i32) x + 100)
  s := -1
  u := 1
  b := (u32) s < u
  r := f(x)
  check(b == 0)
```

Figure 3.3: One program in its three forms. The trusted parser lowers each declaration to an assignment, `u8 x;` to `x = __voblint_nondet_int();`, and collects the declared kinds into `kind_decls`. Elaboration computes every operation at its C kind and inserts each conversion, so `x + 100` wraps to 44, `s < u` compares $2^{32} - 1$ with 1, and the check `b == 0` is `PROVED`. The surface and core columns are the analyzer's `--ast` and `--dot` output (claims `elab-tour*`).

Evaluation. Since every kind is written in the core expression, evaluation `eval` needs no kind environment. An operation computes its mathematical result and wraps it into its kind with `ik_norm`, a conversion wraps its operand, and a variable read returns the stored value. Division and remainder truncate toward zero and give 0 and the dividend for a zero divisor (`c_div`, `c_mod`). Every expression therefore has a value and no effect, so a transfer function may evaluate a guard more than once. Input enters only through the statement `x = __voblint_nondet_int();`, never inside an expression.

To check elaboration, `src_eval` gives a surface expression its meaning directly, given a kind for every name, in the shape the C standard describes. It converts the operands to their common kind, computes there with wraparound, and converts the result to the kind the context expects. Elaboration agrees with it on every store whose values lie in their kinds (`all_in_kinds`).

In the statement below, ρ resolves the names of a procedure to identities, $t \circ \rho$ is the procedure's view of a core store t, $\kappa \circ \rho$ gives each name its kind, and ik is the kind that the context expects of the expression's value.

Theorem 3.3 (Elaboration preserves meaning, `eval_elaborate`)**.**

```
(* src/Program_Model/VIMP/Core_Exp.thy *)                                    Isabelle
theorem eval_elaborate:
  assumes "all_in_kinds κ t"
  shows "eval (elaborate ϱ κ ik e) t = src_eval (κ ∘ ϱ) ik e (t ∘ ϱ)"
```

3.3. Executing a source program thy

A *small-step* operational semantics defines the meaning of a program by a step relation from one configuration to the next, and a run is a sequence of steps [66, §3.2]. VIMP's *source semantics* `pstep` is of this kind, and so is the semantics of the control-flow graph.

Without calls, a configuration is a pair (c, s) of the command that remains to run and the store, as in the semantics of IMP by Nipkow and Klein [7, §7.3]. An assignment updates the store and leaves **skip**, and a check leaves the store unchanged. A sequence steps its first command until it is **skip**. A conditional selects a branch by `truthy`, and a loop unfolds into a conditional that runs the body and then the loop again. The command that remains after a step, which describes the remainder of the execution [66, §3.2], is the *residual*.

While a callee runs, the caller's remaining work has to wait, and a **return** has to skip the rest of the callee's body. VIMP keeps the waiting work inside the residual and saves only data on a stack. A call continues with the callee's body followed by Restore, a runtime marker that hands control back to the caller once the body reaches its end without a **return**, that is, once it *falls through*. A **return** becomes the marker Unwind, which discards the commands after it until it meets the Restore of its call. Each suspended activation keeps its data in a *frame*.

Isabelle defines step relations such as this one inductively. An *inductive definition* is the least relation closed under its rules, and its *rule induction* proves a property of every element with one case per rule.

Definition 3.4 (Source execution, `inductive pstep`)**.** A source configuration is a triple (c, s, frs) of the command that remains to run, a store s, and a list frs of frames (`frame`) for the suspended activations. Each frame records the caller's store and the variable, if any, that receives the result. The step relation `pstep` of Figure 3.4 relates a configuration of a core program to its successor.

```
(* src/Program_Model/VIMP/VIMP_Semantics.thy *)                                                    Isabelle
inductive pstep :: "kenv ⇒ ('a, var_id, exp) gptable
    ⇒ 'a com × store × frame list ⇒ 'a com × store × frame list ⇒ bool"
    ("(_,_ ⊢/ _ →ₚ/ _)" [51, 51, 51, 51] 50)
  for κ :: kenv and Π :: "('a, var_id, exp) gptable"
where
  Assign:  "κ,Π ⊢ (Assign i x a, s, frs) →ₚ (SKIP i, s(x := kind_conv κ (Some x) (eval a s)), frs)"
| Check:   "κ,Π ⊢ (Check i l c, s, frs) →ₚ (SKIP i, s, frs)"
| Seq1:    "κ,Π ⊢ (Seq (SKIP i) c2, s, frs) →ₚ (c2, s, frs)"
| Seq2:    "κ,Π ⊢ (c1, s, frs) →ₚ (c1', s', frs')
             ⟹ κ,Π ⊢ (Seq c1 c2, s, frs) →ₚ (Seq c1' c2, s', frs')"
| IfTrue:  "truthy (eval b s) ⟹ κ,Π ⊢ (If i b c1 c2, s, frs) →ₚ (c1, s, frs)"
| IfFalse: "¬ truthy (eval b s) ⟹ κ,Π ⊢ (If i b c1 c2, s, frs) →ₚ (c2, s, frs)"
| While:   "κ,Π ⊢ (While i b c, s, frs) →ₚ (If i b (Seq c (While i b c)) (SKIP i), s, frs)"
| Call:    "Π p = Some decl
             ⟹ length actuals = length (formals decl)
             ⟹ distinct (formals decl)
             ⟹ vals = map (λe. eval e s) actuals
             ⟹ callee = bind_formals (formals decl)
                          (map2 (λx. kind_conv κ (Some x)) (formals decl) vals)
                          (enter_state is_global s)
             ⟹ κ,Π ⊢ (Call i dst p actuals, s, frs)
                 →ₚ (Seq (body decl) (Restore i), callee, Frame s dst # frs)"
| Special: "special_table p = Some desc
             ⟹ classify_special desc actuals = Some sc
             ⟹ special_result eval sc s v
             ⟹ κ,Π ⊢ (Call i (Some x) p actuals, s, frs) →ₚ (SKIP i, s(x := kind_conv κ (Some x) v), frs)"
| RestoreStep:
    "κ,Π ⊢ (Restore i, s, Frame fr dst # frs)
       →ₚ (SKIP i, combine_assign dst (kind_conv κ dst (s ReturnVar)) (combine_env is_global fr s), frs)"
| ReturnSome:
    "κ,Π ⊢ (Return i (Some e), s, frs) →ₚ (Unwind i, s(ReturnVar := eval e s), frs)"
| ReturnNone:
    "κ,Π ⊢ (Return i None, s, frs) →ₚ (Unwind i, s, frs)"
| UnwindDead:
    "¬ is_Restore c2 ⟹ κ,Π ⊢ (Seq (Unwind i) c2, s, frs) →ₚ (Unwind i, s, frs)"
| UnwindAct:
    "κ,Π ⊢ (Seq (Unwind j) (Restore i), s, Frame fr dst # frs)
       →ₚ (SKIP i, combine_assign dst (kind_conv κ dst (s ReturnVar)) (combine_env is_global fr s), frs)"
```

Figure 3.4: The declaration of the step relation `pstep`, lifted from the theory.

Expressions are evaluated by `eval`, and every written value is converted to the kind that a kind environment κ assigns to its target. Evaluation needs no kinds, but this conversion does, so for a procedure table Π and configurations c, c' a step is written $\kappa,\ \Pi \vdash$ `c` $\rightarrow_{\mathtt{p}}$ `c'`, and $\rightarrow_{\mathtt{p}}^{*}$ is its closure.[1]

A call evaluates the actuals in the caller's store, resets every local of the callee to 0 (`enter_state`), binds the formals, each converted to its formal's kind, and pushes a frame. A `return e` writes the value of e to the return slot. Popping the frame merges the caller's locals with the callee's globals (`combine_env`) and writes the result, converted to the kind of the call's destination, to that destination (`combine_assign`). The remaining helpers of Figure 3.4 recognize a library call and its arguments and test for the Restore marker.

[1]The explainer page animates the source execution, the graph execution and the activation trace of Chapter 4 side by side for one program.

The root activation has no Restore, since `main` has no caller to return to, and a bare Unwind has no step (`pstep_Unwind_stuck`), so `main` must contain no **return**. A source run of a program p with declarations ds is a run of `pstep` on its elaboration Π = `elab_ptable` p ds from the configuration (`main_body` $\Pi, s_0, []$). Its initial store s_0 lies in `init_stores` κ, where every global is 0, as C's static objects are, and every other identity holds some value of its kind. The semantics converts every value it writes to the kind of its target, so every store a run reaches keeps each variable within its kind (`run_from_init_in_kinds`).

The *source contract* `wf_source_program` excludes programs whose calls cannot step or whose results are undefined. It requires that

- every call names a declared procedure with matching arity or a library function with suitable arguments and a destination, and no declared name is a library name,
- the formals of a procedure are distinct and none of them is a global or the return slot, and `main` is declared, takes no arguments and contains no **return** (`no_return`),
- bodies contain no Restore or Unwind,
- the name `ret_var` of the return slot is reserved, so no global, source expression, assignment, or call destination names it,
- a call that stores a result names a *value-providing* callee (`value_providing`), whose body cannot fall through, cannot reach a **return** without a value, and contains a **return** with a value.

C11 makes using the value of a call that falls through undefined (§6.9.1p12). Without the last condition, VIMP would silently store the 0 to which a call resets the return slot on entry. A value-providing callee may still diverge.

Just as Nipkow and Klein show that well-typed programs do not get stuck [7, §9.1.2], we show that well-formed VIMP programs do not, since claims about the points after a stuck run would hold vacuously. For a program satisfying `wf_source_program`, every reachable configuration has either finished, as **skip** with an empty frame stack, or can step. This progress result does not imply termination, since a run may still diverge:

```
(* src/Program_Model/Compile/Simulation/Source_Progress.thy *)                Isabelle
theorem source_progress:
  assumes wf: "gwf_source_program vars ReturnVar is_global Π"
      and run: "κ,Π ⊢ (main_body Π, s0, []) →ₚ* (c, s, frs)"
  shows "(is_SKIP c ∧ frs = []) ∨ (∃cfg'. κ,Π ⊢ (c, s, frs) →ₚ cfg')"
```

The compiler of Section 3.6 also reads a list of the procedures other than `main`, and its input contract `wf_compile_input` is `wf_source_program` plus the condition that this list contains each of these procedures exactly once. Elaboration preserves this contract (`elab_wf_compile_input`), so the theorem covers every run of an elaborated program.

3.4. Scope and adequacy of the source semantics

Every source-level guarantee in this thesis is interpreted against the executions of `pstep` on the elaborated program, from a store in `init_stores` (Section 3.3). Whether these are the executions the programmer intended cannot be proved from the same definitions, so this section argues it. A surface program has no semantics of its own, so the argument covers every definition that gives it a meaning: elaboration, the evaluation of core expressions, wraparound, the conversions on writes, the initial stores and `pstep` itself. The one proved consistency check, `eval_elaborate`, relates two of these definitions to each other and

neither of them to C. Table 3.1 compares VIMP with C11 [8] (committee draft N1570) wherever both languages have a construct but model it differently.

	VIMP	**C11**
integers	every result wraps at its kind	signed overflow undefined (§6.5p5) unsigned arithmetic wraps (§6.2.5p9)
conversions	wrap into the target kind	to unsigned: modulo 2^w (§6.3.1.3p2) to signed: implementation-defined (§6.3.1.3p3)
literals	a leading - belongs to the literal `i32` if $\lvert n\rvert \le 2^{31}-1$, else `i64` -2^{63} is an `i64`	- is unary minus (§6.4.4.1p2, §6.5.3.3p3) first type of its list that fits (§6.4.4.1p5) 2^{63} has no standard type (§6.4.4.1p6)
`a / 0`, `a % 0`	0 and a	undefined (§6.5.5p5)
`/`, `%`, $b \neq 0$	truncated; $(a \;/\; b)\cdot b + a\%b = a$	the same where $a \;/\; b$ is representable (§6.5.5p6)
`&&`, `||`	both operands evaluated	second operand only if needed (§6.5.13p4, §6.5.14p4)
initial values	globals 0, others arbitrary	static 0, automatic indeterminate (§6.7.9p10)
`u8 x;`	any value of its kind, each time	indeterminate (§6.7.9p10)
callee locals	0 on entry, never read	indeterminate (§6.7.9p10)
checks	never stop the run, never refine	`assert` aborts on 0 unless `NDEBUG` (§7.2p1, §7.2.1.1)
`min`, `max`	library calls (`special_table`)	no counterpart
`main`	no explicit `return`	`return` gives the exit status (§5.1.2.2.3)

Table 3.1: Constructs that both languages have but model differently. The VIMP rows restate `ik_norm`, `kind_conv`, `ik_of_lit`, `lits_fit`, `c_div`, `c_mod`, `eval`, `init_stores`, `pstep_declare`, `enter_state` and `pstep`.

Each row is a place where a statement true of a VIMP program does not necessarily hold for the corresponding C program. Wrapping signed overflow, wrapping conversions and defined division by zero can flip a verdict. After `z = 5 / 0;` the check `z == 0` holds in VIMP, so a `PROVED` verdict on a program that divides by zero says nothing about C. The analyzer therefore reports possible arithmetic hazards as separate diagnostics (Section 10.5).

The parser folds a leading minus sign into the literal and chooses its kind by magnitude, so `-9223372036854775808`, the one literal on which VIMP and C differ, is an `i64`. In C, it is unary minus applied to `9223372036854775808`, which no standard type represents when `long long` has 64 bits (§6.4.4.1p6). Division and remainder truncate toward zero as in C11. Goblint once got the remainder wrong, and issue 1156 [13] reports that it claimed `c % 2 == 1` for $c \in \{-5, -7\}$. Because expressions are total and have no effects, evaluating both operands of `&&` gives the short-circuit value. Only the arithmetic diagnostics differ, because they cover both operands. The analyzer warns about the divisor in `x != 0 && 10 / x > 1` (claim `and-divisor-warning`).

VIMP fixes initial values that C leaves open. Every source-level theorem assumes an initial store from `init_stores`, in which the locals of `main` hold some value of their kinds. The parser lowers a declaration without an initializer, such as `u8 x;`, to the library call `x = __voblint_nondet_int();`, which stores some value of the kind of `x` whatever `x` held before (`pstep_declare`, `pstep_declare_forgets`). A declared local thus starts at some value of its kind each time its declaration executes, and that the parser lowers every declaration this way is trusted. A call also resets the callee's locals to 0 (`enter_state`). However, no program the parser accepts observes the reset, because the parser rejects a use before the declaration and the declaration writes the local before any read. No shipped analysis relies on the reset either, because every abstract entry assumes nothing about callee locals beyond their kinds (`nonrel_enter_frame` for the analyses that track each variable separately). Checks never prune executions, so the verdict for one check does not

depend on an earlier one. Goblint's `__goblint_check` behaves the same way, and its library table classifies it as checked and not used for refinement (`libraryFunctions.ml`).

No second semantics. No external semantics cross-checks `pstep`. For C, Isabelle offers the C parser of AutoCorres2 [67], which translates a subset of C into Simpl programs [65], and Coq has the Clight semantics of CompCert [33]. Relating VIMP to either of them, or to IMP2 or Simpl directly, would need a translation and its own adequacy argument, which is left as future work.

3.5. The procedure-aware control-flow graph

3.5.1. Why a graph

An analysis attaches abstract states to program points and transfer functions to transitions. `pstep` offers neither in a usable form. Its only notion of location is the residual. Without recursion, only finitely many residuals arise, because a loop's unfolding revisits the same command forms, but a recursive call can nest another Restore wrapper at every call depth. An assignment is also not a transition between fixed program points, because inside a sequence, it steps through the rule `pstep.Seq2`, which rebuilds the surrounding residual. Finally, nothing names the entry of a procedure, its result, or the point where a caller resumes. The schematic control-flow graph of Figure 2.1 solves the first two problems, with finitely many nodes and one transfer per edge. The procedure-aware graph solves the third. It adds an entry node and a result node per procedure, and call edges that record their continuations. The entry node holds the state a callee starts from, the result node the state it returns with, and the continuation names the node where the caller resumes. A call can therefore be analyzed by connecting these nodes instead of by following the callee's body.

A *syntax-directed* analyzer, such as that of Nipkow and Klein [7, Ch. 13], needs no graph, because it recurses over the command structure. Voblint instead follows Goblint and is *constraint-based* [44]. It turns the program into a system of inequalities over unknowns indexed by the graph's nodes and leaves the solution to a generic solver that knows nothing about programs. Calls need no special treatment by the solver, because a call contributes ordinary right-hand sides and side effects (Section 2.6) to the same system. This choice requires the compiler and simulation proof of this chapter, and a semantics of traces that says what the inequalities mean (Chapter 4). The solver is the verified top-down solver of Section 2.7.

3.5.2. Nodes and edges thy

Definition 3.5 (Control-flow graph, `record gcfg`)**.** A graph g is a record of the type below. The graph of a program p with declarations ds, `elab_cfg` p ds, is its instance at identities and core expressions:

```
(* src/Program_Model/CFG/CFG_Def.thy *)                                    Isabelle
record ('v, 'e) gcfg =
  intra     :: "(cfg_node × ('v, 'e) gedge_action × cfg_node) set"
  calls     :: "(cfg_node × ('v, 'e) gcall_action × cfg_node × cfg_node) set"
  cfg_entry :: cfg_node
```

The field `intra` holds the *local edges* (u, a, v), each labeled by a `gedge_action` a. The field `calls` holds the *call edges* $(u, ca, \mathsf{FunctionEntry}\ q, k)$. Such an edge consists of the call site

VIMP	transition and its effect on the store s	Goblint
`EA_Assign` x e	assignment, sets x to the converted value of e	`Assign`
`EA_Assume` e	guard, keeps s if e holds, else no successor	`Test`
`EA_AssumeNot` e	guard, keeps s if e fails, else no successor	`Test`
`EA_Special`	`min`, `max` or input, stores a possible result	`Proc`
`EA_Check`	check, keeps s	`Proc`
`EA_Body` p	enters the body of p, keeps s	`Entry`
`EA_Ret`	return, writes a returned value to the return slot	`Ret`
`EA_Nop`	no-op, keeps s	`Skip`
call edge	call of a declared procedure	`Proc`
none	inline assembly, in-place declaration	`ASM`, `VDecl`

Table 3.2: Transitions of the graph and their counterparts among the edges of Goblint's CFG at the pinned revision (`edge.ml`). All rows except the last two are `gedge_action` constructors on `intra` edges, with the effect that `edge_step` gives them. Goblint routes library calls and `__goblint_check` through `Proc`. It skips empty statements and uses `Skip` only for the self-loop of an empty loop.

u, the call information ca, the entry node of the callee (by `wf_cfg`), and the node k where the caller resumes. Nodes (`cfg_node`) are Statement n, FunctionEntry p or FunctionResult p. A check is a local edge labeled `EA_Check` with its condition.

The call information CallEdge *dst formals args* (`gcall_action`) copies the callee's formals onto the edge, so the graph semantics binds actuals to formals without the procedure table. VIMP has no function pointers (Section 3.1), so every call names its callee, and the compiler fixes the target of each call edge. Indirect calls would require discovering call targets during the analysis and are left for future work. The node kinds match Goblint's (`node0.ml`), which calls our FunctionResult node `Function`.

Each edge action a denotes a transfer `edge_step` a, which maps a store to the set of stores after the edge. The set is empty when a guard fails and has more than one element for nondeterministic input. `EA_Body` labels the edge from the entry node of a procedure into its body, and `EA_Nop` labels a **skip**. The compiler also emits `EA_Nop` for a library call that stores no result or whose arguments do not fit the library function, but the source contract excludes such calls (Section 3.3).

A fragment of Goblint's input. Goblint analyzes C after its CIL front end [5] has normalized it, so that expressions have no side effects and a call is a separate instruction with an optional destination (`cil.mli` at the revision Goblint pins). VIMP makes the same choice, and every edge kind of Goblint's CFG, except inline assembly and in-place declarations, has a VIMP counterpart (Table 3.2). No translation from C to VIMP is formalized.

The graph keeps local edges and call edges in separate fields, because a local edge changes the store within one activation, while a call edge starts a new one. No edge action denotes a call, so a call can never be taken as a local step.

Returns, in contrast, are ordinary local edges. A **return** in p leads to the result node FunctionResult p, which all callers of p share, so recursion needs no copies of nodes. A shared result node has no single continuation, and in a compiled graph no edge leaves it. The call edge records the continuation k instead. After a return, execution resumes at the continuation saved in the frame, and an analysis connects the callee's result to the correct caller through the call edge. The saved frame identifies the matching call, so the graph needs

no return edges to individual callers. A completed program ends at FunctionResult `main` (`cfg_exit`). Figure 3.5 shows these properties on a recursive program.

```
1  fun i32 dec(i32 x)                    VIMP ↗
   {
2    return x - 1;
3  }
4
5  fun i32 sum(i32 n) {
6    if (n < 1) {
7      return 0;
8    }
9    i32 m = dec(n);
10   i32 r = sum(m);
11   return r + n;
12 }
13
14 fun main() {
15   i32 x = sum(2);
16   __voblint_check(x == 3);
17 }
```

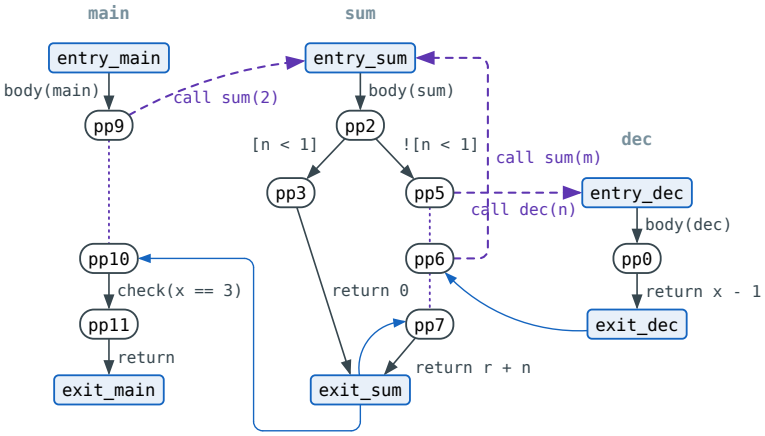


Figure 3.5: A recursive program and its compiled graph, labeled as `voblint --dot` prints it. Here `pp` n is Statement n, and entry_p, exit_p are FunctionEntry p, FunctionResult p. As in Figure 3.1, solid gray arrows are `intra` edges, and dashed purple arrows are `calls` tuples. Dotted connectors and thin blue arrows are not edges. Dotted connectors show the continuation of a call tuple. Thin blue arrows show where execution continues after a result node. Each procedure has one copy of its nodes, whatever the recursion depth. The drawing omits the fall-through nodes `pp` 1 of `dec` and `pp` 8 of `sum` and their return edges, which no run reaches because both bodies always return explicitly. The number `pp` 4 belongs to the empty `else` branch of the conditional, which receives no edge (Section 3.6.1).

3.6. Compilation

The compiler turns a VIMP program into a procedure-aware graph. Its design decides which program points exist and where the code of each command lies in the graph.

3.6.1. Compiling a command thy

To emit the edges of a command, a compiler has to know where control goes after it. Compilers commonly pass this target down, as the inherited attribute S.next of the dragon book [56, §6.6.3]. Voblint's *continuation-passing* compiler does the same. It takes the command and the node k at which control continues afterward, and emits the command's edges so that they end at k (`compile`, `compile_prog`). Nodes are numbered consecutively. A command c occupies a block of `csize` c numbers, a count computed from its syntax alone, and it is entered at the first of them, entry(c). The compiler therefore knows the entry of c_2 in c_1; c_2 before it compiles c_2. Schematically, writing compile(c, k) for the edges of c against the continuation k, n for entry(c) and h for the entry of a loop,

$$\begin{aligned}
\text{compile}(x := e, k) &= \left\{ n \xrightarrow{x:=e} k \right\}, \\
\text{compile}(p(e_1, \ldots, e_m), k) &= \left\{ n \xrightarrow{\text{call}} \text{FunctionEntry}\, p \text{ with continuation } k \right\}, \\
\text{compile}(c_1;\, c_2, k) &= \text{compile}(c_1, \text{entry}(c_2)) \cup \text{compile}(c_2, k), \\
\text{compile}(\textbf{if } (e)\{c_1\} \textbf{ else } \{c_2\}, k) &= \left\{ n \xrightarrow{e} \text{entry}(c_1), n \xrightarrow{\neg e} \text{entry}(c_2) \right\} \\
&\quad \cup \text{compile}(c_1, k) \cup \text{compile}(c_2, k), \\
\text{compile}(\textbf{while } (e)\{c\}, k) &= \left\{ h \xrightarrow{e} \text{entry}(c), h \xrightarrow{\neg e} k \right\} \cup \text{compile}(c, h), \\
\text{compile}(\textbf{return } e, k) &= \left\{ n \xrightarrow{\textbf{return } e} \text{FunctionResult}\, p \right\}.
\end{aligned}$$

The alternative would give every fragment an exit node of its own, and sequencing two fragments would need a no-op edge between two nodes that denote the same program point. With continuations, a node is a program point between two transfers, as in Goblint's CFG. A conditional needs no join node, because both branches end at the same k, and a loop compiles its body against its own head, so the compiled counting loop of Figure 2.1 has no node t and no edge $t \to h$. A **return** ignores its continuation. An empty branch **skip** of a conditional keeps its number but receives no edge, because the guard edge leads directly to k.

A call to a declared procedure becomes a call edge from the call site to the callee's entry, with k as its continuation, and adds no local edge. A **return** becomes an `EA_Ret` edge to the result node of its procedure. A body that falls through needs such an edge as well, so `compile_proc` compiles every body against one extra node, the *fall-through node*, from which a return edge without a value leads to the result node (Figure 3.6). Compiling the body directly against the result node would let a final assignment end there, and the extra node keeps every edge into FunctionResult p a return edge. In a body that always returns, the fall-through node and its edge are dead code.

```
(* src/Program_Model/Compile/VIMP_Proc_to_CFG.thy *)                    Isabelle
definition compile_proc ::
  "('a, 'v, 'e) gptable ⇒ pname ⇒ ('a, 'v, 'e) gproc_decl ⇒ nat
   ⇒ nat × (cfg_node × ('v, 'e) gedge_action × cfg_node) set
        × (cfg_node × ('v, 'e) gcall_action × cfg_node × cfg_node) set"
where
  "compile_proc Π p decl n =
     (let r = n + csize (body decl);
          (n', ben, E, K) = compile Π p (body decl) (Statement r) n
      in (Suc r,
          insert (FunctionEntry p, EA_Body p, ben)
            (insert (Statement r, EA_Ret None p, FunctionResult p) E),
          K))"
```

Figure 3.6: The declaration of `compile_proc`, lifted from the theory.

These rules give every compiled graph a fixed shape. Call edges always enter a procedure entry, no local edge enters one, and every return edge ends at the result node of its own procedure. The structural contract `wf_cfg` collects these properties, and every compiled graph satisfies it without premises (`compile_prog_wf`). The graph's entry is FunctionEntry `main`, and its edge sets are finite (`compile_prog_finite`), so the executable analyzer can enumerate them.

3.6.2. Graph execution thy

Definition 3.6 (Graph execution, `inductive cstep`)**.** A graph configuration (`cconf`) is a triple of a node, a store, and a stack of suspended callers (`cframe`). Each entry records where to resume, which variable receives the result, and the caller's store. The step relation `cstep` is the instance at the same parameters as `pstep`, written κ, g ⊢ c $\rightarrow_c$ c', and $\rightarrow_c^*$ is its closure.

```
(* src/Program_Model/CFG/CFG_Exec.thy *)                                    Isabelle
inductive cstep :: "kenv ⇒ (var_id, exp) gcfg ⇒ cconf ⇒ cconf ⇒ bool"
    ("(_,_ ⊢/ _ →c/ _)" [51, 51, 51, 51] 50) for κ g where
  Intra [intro]:
    "(u, a, v) ∈ intra g ⟹ s' ∈ edge_step κ a s ⟹ κ,g ⊢ (u, s, stk) →c (v, s', stk)"
| Call [intro]:
    "(u, CallEdge dst pars actuals, FunctionEntry q, cont) ∈ calls g ⟹
     κ,g ⊢ (u, s, stk)
       →c (FunctionEntry q, call_enter κ (CallEdge dst pars actuals) s, (cont, dst, s) # stk)"
| Return [intro]:
    "κ,g ⊢ (FunctionResult q, t, (cont, dst, caller) # stk)
       →c (cont, combine_collect κ dst caller t, stk)"
```

Figure 3.7: The declaration of the graph step `cstep`, lifted from the theory.

Figure 3.7 lists the rules, one per kind of step. A local edge applies its transfer `edge_step`, a call edge enters the callee and pushes a frame, and a result node pops the top frame. In the running example (Figure 3.1), a run of `main` takes the call edge of `bump(5)` to the entry of `bump` with $n = 5$ and pushes a frame that names the continuation. It follows the local edges of `bump` to its result node, pops the frame, and resumes at the continuation, where `a` holds 6. The entry and exit transfers (`call_enter`, `combine_collect`) are built from the functions that `pstep` uses, so both executions can hold the same store. `cstep` is defined for an arbitrary graph, not only for compiled ones.

3.7. Relating the two executions

The soundness theorem is stated over `pstep`, but the analysis solves equations generated from the graph. Every finite source run must therefore be matched by a run of the compiled graph that ends with the same store at a related node. We prove this with a *forward simulation*, a relation between source and graph configurations such that every source step can be matched by zero or more graph steps that preserve it.

The source keeps the remaining work of a suspended caller inside the running command, behind a Restore marker, while the graph keeps it on the frame stack as the continuation node of the call. The two also step at different places. Unfolding a loop, discarding a finished **skip**, and propagating Unwind have no corresponding graph step, so the graph answers one source step with zero or more steps. We prove only this direction, because soundness needs no converse.

The relation has to say where a residual is in the graph and how the two stacks correspond. The first part is `control_at`. It relates a residual of a procedure body to the node of the body's compiled code at which the graph stands when the source has that residual left to run. A residual that starts with a command is at the entry node of that command, and a command that has just finished, leaving **skip**, is at its continuation. For the stacks, consider the running example while `bump(5)` runs. The source command has the form $(r;\ \text{Restore}\ i);\ a_1;\ ...;\ a_m$. Here r is the residual of the body of `bump`, i is the annotation of the call, and $a_1, ..., a_m$ are the commands of `main` still to run after the call,

written *afters* in Figure 3.8. The graph is at a node of `bump`, and its stack holds one frame $(cont, dst, caller)$ with the continuation node of the call. The invariant connecting the two is that the commands after the Restore are exactly the code that the graph runs from *cont* on. Once the call has finished, it leaves **skip** i; a_1; ...; a_m, and this residual is at *cont*.

Definition 3.7 (Simulation relation, `inductive csim`)**.** `csim`, written `Π, g, p ⊢ (c, s, frs) ≈ (v, s, stk)`, relates a source configuration to a graph configuration holding the same store, where p is the procedure of the outermost activation, `main` for a run of the program.

```
(* src/Program_Model/Compile/Simulation/Simulation_Relation.thy *)                    Isabelle
inductive csim :: "('a, var_id, exp) gptable ⇒ (var_id, exp) gcfg ⇒ pname
                    ⇒ ('a, var_id, exp) gcom × store × frame list
                    ⇒ cfg_node × store × cframe list ⇒ bool"
    ("(_,_,_ ⊢/ _ ≈/ _)" [51, 51, 51, 51, 51] 50) for Π g where
  Base:
    "control_at Π p body_p k n c v ⟹ compiled_at Π g p body_p k n ⟹
     Π, g, p ⊢ (c, s, []) ≈ (v, s, [])"
| Nested:
    "Π, g, q ⊢ (inner, s, frs) ≈ (v, s, stk) ⟹
     control_at Π pc body_pc kc nc (seq_after (SKIP i) afters) cont ⟹
     control_at Π pc body_pc kc nc (seq_after (Call i dst q args) afters) site ⟹
     compiled_at Π g pc body_pc kc nc ⟹
     Π, g, pc ⊢ (seq_after (Seq inner (Restore i)) afters, s, frs @ [Frame caller dst])
       ≈ (v, s, stk @ [(cont, dst, caller)])"
| Returning:
    "pop_ready w ⟹ pop_ann w = Some i ⟹
     control_at Π pc body_pc kc nc (seq_after (SKIP i) afters) cont ⟹
     control_at Π pc body_pc kc nc (seq_after (Call i dst q args) afters) site ⟹
     compiled_at Π g pc body_pc kc nc ⟹
     Π, g, pc ⊢ (seq_after w afters, callee, [Frame caller dst])
       ≈ (FunctionResult q, callee, [(cont, dst, caller)])"
```

Figure 3.8: The declaration of `csim`, lifted from the theory.

In Figure 3.8, `compiled_at` states that *body_p* is the body of p and that its compiled edges, together with its entry edge and its fall-through edge, lie in g. As in Section 3.6.1, k is the continuation against which the body is compiled, here its fall-through node, and n is the first node number of the body. The suffix c in *kc*, *nc* and *body_pc* marks the caller. `seq_after` c *afters* appends the list *afters* to c by sequencing. Base relates a single activation with both stacks empty. Nested adds one suspended caller beneath a related callee configuration and states the invariant above with two `control_at` premises for the same *afters*. The call with annotation i, followed by *afters*, is at the call site, and the finished call **skip** i; *afters* is at *cont*. The list is the same in both because a call does not change what its caller does afterwards, and the shared annotation i marks the **skip** as the remainder of this call. Returning covers the moment after the callee has finished. The graph is already at the callee's result node, while the source still holds the Restore of the call, possibly behind a propagating Unwind (`pop_ready`). The condition `pop_ann` $w = \text{Some } i$ says that this Restore belongs to the call with annotation i.

Equal commands can occur at different places, so the relation does not determine the node. In `if (b) { x = 1; } else { x = 1; }`, the residual `x = 1` is related to both branch nodes. The simulation theorems therefore state that some related graph configuration exists.

Theorem 3.8 (One step is matched, `csim_step`)**.** *If a source configuration and a graph configuration are related by* `csim` *and the source takes one step, the graph takes zero or more steps to a configuration related to the new source configuration.*

```
(* src/Program_Model/Compile/Simulation/Simulation_Preservation.thy *)
theorem csim_step:
  assumes SIM: "Π, g, ϱ ⊢ (c, s, frs) ≈ (v, t, stk)"
      and PC: "procs_embedded Π g"
      and WF: "return_safe c"
      and STEP: "κ,Π ⊢ (c, s, frs) →p src'"
  shows "∃cfg'. (κ,g ⊢ (v, t, stk) →c* cfg') ∧ Π, g, ϱ ⊢ src' ≈ cfg'"
```

Isabelle

The theorem also assumes that every declared body lies in the graph with its entry and return edges, and that the running command returns only inside an activation that a call opened. Both hold for every well-formed program, and every source step preserves the second.

The proof distinguishes calls, returns, return propagation and all other steps. The hard cases are the ones that change the stacks. At a call, the graph takes the call edge, and the new Nested layer gets its location of the finished call from the location of the call itself, since the compiler gave the call edge the continuation node of the call. The return is the hardest case. The source writes the return slot, and the graph follows along the return edge to the result node. The source then propagates Unwind through the dead code of the callee, and the graph stays at the result node, which Returning allows. The final pop of the frame is matched by the graph's return to *cont*, and there the caller's residual **skip** i; $afters$ is related to *cont* because the Nested layer recorded this location at the call. By induction on the run, `csim_star` extends the theorem to finite source runs, as in CompCert's simulation diagrams [33, §4.3].

The initial source configuration is related to the node after the `EA_Body` edge of `main` (`compile_prog_main_base`). Therefore, if a finite source run of a well-formed program reaches (c, s, frs), a graph run from the entry reaches some (v, s, stk) that `csim` relates to it. The program points of a source configuration are the nodes v related to it in this way. At each of them, the store s of the source execution is a store of a graph run, which is the first inclusion of the soundness chain of Figure 1.3. A check is found in the graph by its label, the source position it carries. If the residual is about to run a check, every node related to it has an `EA_Check` edge with the same label and condition (`csim_next_check_edge`). Flat runs, however, do not record which activation a store belongs to, and a context-sensitive analysis needs this. Chapter 4 therefore regroups graph runs into activation traces.

4. Traces and the Coverage Contract

Chapter 3 matched every source run with a run of the compiled graph. For an analysis that computes one abstract state per node, this suffices. A context-sensitive analysis, however, computes a separate state for each node and calling context, so to justify its result we need to know which executions each context represents. The collecting semantics of Chapter 2, which gathers the stores reached at each node, does not record this. In the running example, both calls of `bump` reach the same result node, but a context-sensitive analysis may distinguish the activation entered with 5 from the one entered with 4.

Each call of a procedure starts a new *activation* of it, which lasts until the call returns [56, §7.2.1]. We represent executions by *activation traces*. An activation trace records the path of one activation through the graph, together with the trace of the caller that created it and, after a call returns, the finished callee. The calling contexts of an activation are then derived from its trace. An analysis result is judged through its *claim*, which assigns each node and context a set of stores. For the analyzer, the claim at a node and context is the concretization of the abstract value it computed there. On the trace semantics we build a local *coverage contract*, a set of conditions on the claim that mention only concrete stores. Once they hold, the claim for a node and a context covers the stores that executions in that context reach there, and the claims of all contexts together cover every store that a run reaches at the node.

4.1. Why contexts need traces

The collecting semantics of Chapter 2 assigns each node v of a control-flow graph the set of stores that some run holds on reaching v [46, §4]. Chapter 2 over-approximated it with one unknown per node, which we now write $[v]$ [44, §2]. A context-sensitive analysis instead splits $[v]$ into unknowns $[v, c]$, one per calling context c, following the value tables of Sharir and Pnueli [57] in the constraint-system formulation of Apinis et al. [44, §3]. Seidl et al. state that the invariant for $[u, c]$ "only need[s] to take into account executions reaching u that satisfy the restriction imposed by context c" [2, §4]. The claim for $[v, c]$ therefore has to describe only a subset of the stores that the claim for $[v]$ describes, and it can be correspondingly more precise.

In the running example (Figure 3.1), `bump` returns its argument plus one. A context-insensitive analysis has one claim for the result node of `bump`, which must describe both results, 6 and 5. An analysis that distinguishes the two calls by their argument can instead claim 6 for the call with argument 5 and 5 for the call with argument 4. To justify these claims, we must know which store at the result node belongs to which call, and the collecting semantics does not record it.

The frame stack of the graph execution (Section 3.6.2) records the callers that wait for the running activation. It holds, however, only what execution needs to resume them, and it forgets a call once the call has returned. The contexts of a callee, in contrast, depend on

the caller's context and store at the call, and after the return the caller continues in the contexts it had before. Nor does the run itself make this explicit. A flat record of the run, the list of node-store pairs (v, s) that it passes through (Figure 4.1, top), interleaves the steps of all activations. In the program of Figure 3.5, `sum(2)` calls `dec` and then itself, so one run visits the nodes of `sum` in three activations, and finding the contexts of a step would mean recovering from the list which call created its activation and which return resumes which caller.

Grouping the same run by activation makes these relationships explicit (Figure 4.1, bottom). A call creates a new activation attached to its caller, and a return composes the finished callee back into that caller. The construction is inspired by the *local traces* of Schwarz et al. [6], which record one thread's view of a concurrent execution. An activation trace records one activation's view of a sequential run.

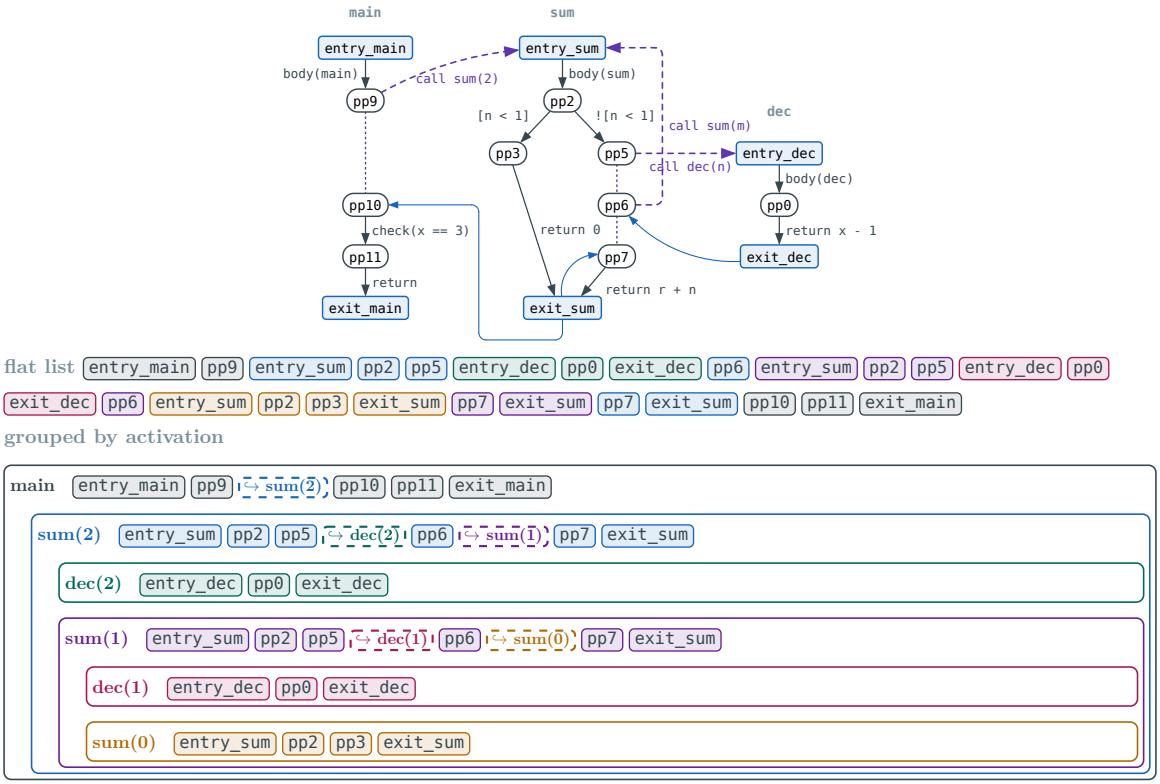


Figure 4.1: The compiled graph of the program in Figure 3.5 (top) and one run on it, as the flat list of its steps and grouped by activation. Colors mark each step's activation, which the flat list itself does not record. A dashed tag ↪ marks a call of the named callee. The run is written out by hand, and stores are omitted.

4.2. Traces

4.2.1. Activation traces thy

We turn the grouping of Figure 4.1 into a datatype.

Definition 4.1 (Activation trace, `datatype activation_trace`)**.** A trace has one constructor per way an activation begins or continues (Figure 4.2).

```
(* src/Program_Model/CFG/Collecting/Activation_Trace_Def.thy *)
type_synonym activation_path = "(cfg_node × store) list"
```

```
(* src/Program_Model/CFG/Collecting/Activation_Trace_Def.thy *)
datatype activation_trace =
    Root "activation_path"
  | Call (activation_trace_caller: "activation_trace") "activation_path"
  | Resume (activation_trace_current: "activation_trace")
    (activation_trace_callee: "activation_trace") "activation_path"
```

Figure 4.2: The declarations of `activation_path` and `activation_trace`, lifted from the theory. An `activation_path` is the path of one activation, a list of pairs of a CFG node and a store.

Root π is the initial activation of the program with activation path π, and a run starts with Root $[(entry, s_0)]$ at the entry of `main` with an initial store s_0. Call $t'\ \pi$ is a callee created by the caller t', whose activation path begins at the callee's entered store. Resume $t'\ t''\ \pi$ is the activation t' continued past the call that produced the finished callee t''. Its path π begins with the whole path of t', followed by the continuation, so a call starts a fresh path and a return extends the caller's. We write `path_of` t for the path of t, and `extend` $t\ (v, s)$ for t with the step (v, s) appended to its path. `sink_node` t and `sink_store` t are the node and store of the last entry of the path, and `caller_of` t is the caller that created the activation. A Root has no caller, a Call stores its caller, and a Resume keeps the caller of the activation it continues, `caller_of` (Resume $t'\ t''\ \pi$) = `caller_of` t'.

Each trace represents the history of one activation up to a program point, so on well-formed compiled programs its activation path stays inside the nodes of one procedure (`valid_activation_trace_frag_callers`). A whole run is a family of traces linked by their caller and callee fields. Because a Call stores its caller exactly, frozen at the call node, a finished callee composes back into the activation that created it, without a search for a compatible stack frame. A return therefore continues exactly the caller that created the callee.

The datatype stores no calling context. Which information a context records depends on the analysis, and for contexts built from abstract entry states it even depends on the analysis's own result, so a trace that stored its context would change the traces with every analysis. Instead, the traces and the stores they collect are fixed once, independently of any analysis. Each analysis supplies for every concrete call the set of callee contexts it admits, and the contexts of traces are derived from these sets (Section 4.3). The soundness theorems are therefore proved once for all such choices.

4.2.2. Valid traces thy

The datatype also admits terms that no execution produces. A path may jump from the entry of `sum` directly to its result node, along no edge of the graph, and a Resume may continue `main` after its call `sum(2)` with the finished inner activation `sum(1)`, whose caller is `sum(2)`. Validity rules out such terms.

Definition 4.2 (Valid traces, `inductive_set valid_activation_trace`)**.** For a graph g and a set S of initial stores, `valid_activation_trace g S` is the least set of traces closed under the four rules of Figure 4.3.

```
(* src/Program_Model/CFG/Collecting/Activation_Trace_Def.thy *)                Isabelle
inductive_set valid_activation_trace ::
    "kenv ⇒ (var_id, exp) gcfg ⇒ store set ⇒ activation_trace set"
    ("𝒯_,_,_")
  for κ g and S where
  root:
    "s ∈ S
     ⟹ Root [(cfg_entry g, s)] ∈ 𝒯κ,g,s"
| intra:
    "t ∈ 𝒯κ,g,s
     ⟹ (sink_node t, a, v) ∈ intra g
     ⟹ s' ∈ edge_step κ a (sink_store t)
     ⟹ extend t (v, s') ∈ 𝒯κ,g,s"
| call:
    "caller ∈ 𝒯κ,g,s
     ⟹ (sink_node caller, CallEdge dst pars args, FunctionEntry p, cont)
          ∈ calls g
     ⟹ Call caller
          [(FunctionEntry p,
            call_enter κ (CallEdge dst pars args) (sink_store caller))]
        ∈ 𝒯κ,g,s"
| ret:
    "callee ∈ 𝒯κ,g,s
     ⟹ caller_of callee = Some caller
     ⟹ sink_node callee = FunctionResult p
     ⟹ (sink_node caller, CallEdge dst pars args, FunctionEntry p, cont)
          ∈ calls g
     ⟹ Resume caller callee
          (path_of caller
             @ [(cont, combine_collect κ dst
                         (sink_store caller) (sink_store callee))])
        ∈ 𝒯κ,g,s"
```

Figure 4.3: The four rules of `valid_activation_trace`: root, intra, call, and ret. Each reads the graph relation for its own kind of step: `intra` for local flow, and `calls` both for entering a callee and for recovering the continuation *cont* at a return.

The root rule corresponds to the initial graph configuration, and the other three correspond to the rules of `cstep`. Intra appends a step along a local edge. Call starts a new trace that holds the caller, where `cstep` pushes a frame, and ret builds a trace that holds caller and callee, where `cstep` pops the frame.

The ret rule applies when a callee trace has reached a result node `FunctionResult p`. No edge leaves that node, since it serves every caller of p, so the rule finds the caller through `caller_of callee = Some caller` and takes the `calls` edge from its call node into p, the only such edge on compiled programs. That edge names the continuation *cont*, where the caller continues with the store that `combine_collect` builds from both final stores.

4.2.3. Graph runs are valid traces thy

The traces describe the runs of the compiled graph, and through Section 3.7 those of VIMP programs, keeping the history that the frame stack discards.

Definition 4.3 (Trace representation, `definition activation_trace_repr`)**.** A valid trace represents a graph configuration if it ends at the configuration's node and store, and its chain of creating callers matches the runtime frame stack.

For a well-formed compiled program (wf_compile_input), the correspondence is lockstep. The initial configuration is represented by a Root trace, a local graph step extends the current trace, a call creates a Call trace, and a return a Resume trace (cstep_preserves_activation_trace_repr). Hence every configuration that a graph run of such a program reaches has a representing valid trace (csteps_preserve_located_activation_trace). The converse is not proved, and soundness does not need it. With the simulation of Chapter 3, this result extends to VIMP programs.

Theorem 4.4 (Source runs are traces, source_run_has_activation_trace)**.** *For a well-formed compiled program, every finite source execution from an initial store in S has a matching graph configuration and a valid trace ending at the same node with the same store, whose caller chain represents the graph's frame stack.*

```
(* src/Program_Model/Compile/Source_To_Trace.thy *)                    Isabelle
theorem compile_prog_run_has_activation_trace:
  assumes wf: "gwf_compile_input vars ReturnVar is_global Π ps"
    and s0: "s0 ∈ S"
    and run: "κ,Π ⊢ (main_body Π, s0, []) →ₚ* (residual, s, frs)"
  shows "∃v stk t. Π, compile_prog Π ps, prog_main_name ⊢ (residual, s, frs) ≈ (v, s, stk)
                  ∧ activation_trace_repr κ (compile_prog Π ps) S (v, s, stk) t"
```

4.2.4. The node collecting semantics thy

Collecting the final stores of the valid traces per node gives the *node collecting semantics*. It plays the role of the collecting semantics of Chapter 2 for programs with procedures, with valid traces in place of runs.

Definition 4.5 (Node collecting semantics, definition node_collect)**.**

```
(* src/Program_Model/CFG/Collecting/Activation_Trace_Collect.thy *)    Isabelle
definition node_collect ::
    "kenv ⇒ (var_id, exp) gcfg ⇒ store set ⇒ cfg_node ⇒ store set"
    ("𝒞_,_,_") where
  "node_collect κ g S v =
     {sink_store t | t. t ∈ 𝒯_κ,g,S ∧ sink_node t = v}"
```

Its core VIMP instance is $\mathcal{C}_{\kappa,\mathtt{g},\mathtt{S}}$ (node_collect), and a store is in $\mathcal{C}_{\kappa,\mathtt{g},\mathtt{S}}$ v exactly when some valid trace ends at v with this store. From here on κ, g, and S are the kind environment, graph, and initial stores of the program at hand. A procedure's result is an ordinary collected node, and the stores in which a run of the whole program completes are those collected at FunctionResult *main*. Every store that a finite source execution of a well-formed program reaches therefore lies in the node collecting semantics at a corresponding node (source_reaches_node_collect).

The node collecting semantics forgets the traces, so a claim stated over it alone is context-insensitive. The traces themselves keep their structure, so the same set of traces can also be split by context.

4.3. Calling contexts thy

A *context* is an index under which an activation is analyzed, and one activation may carry several. At a fixed node v, activations with the same context c contribute to the same abstract unknown $[v, c]$. Contexts change only at calls. A local step keeps the contexts of the trace (`activation_context_rel_extend`), and after a return the caller continues in the contexts it had before the call.

Which information a context records is the analysis's *context policy* [17, §8.4.1], [52, §2.6], [52, §2.9], [57]. The call-string approach records the call history, in practice bounded to the k most recent call sites, since a recursive procedure otherwise has infinitely many contexts. The functional approach instead computes caller-independent procedure summaries that each call site instantiates. Goblint uses partial tabulation, a variant of the functional approach that tabulates summaries only for the calling contexts the analysis meets [32, §3.3], [36, §5, p. 110], and obtains a callee's context by applying each analysis's `context` function to its abstract entry state. Erhard et al. treat full entry states, their projections, and call strings in one framework [68, §4]. Voblint offers bounded call strings, whose length zero gives the context-insensitive analysis, and entry-state contexts, whose context is the list of abstract values of the callee's formals in the entry state and whose number nothing bounds.

These selectors choose one callee context per call. The *entry operation* of an analysis, Goblint's `enter`, turns an abstract caller state at a call into abstract entry states of the callee. In Goblint it may return several alternatives for one call, each a pair of a caller state and a callee entry state with its own context. Several alternatives let an analysis split a call into cases. Suppose, hypothetically, that a caller passes an argument that is either negative or positive, and that its state keeps the two cases apart, for instance as two path-sensitive states, which no shipped Voblint analysis has. With a single entry state, a sign analysis analyzes the callee once for the join of both cases, which it can only describe by $\top$, any sign. With one alternative per case, it analyzes the callee once in a context for negative and once for positive arguments and keeps the sign apart. If the cases overlap, one concrete call belongs to several contexts at once, which a context function of the call cannot express. A Voblint policy therefore returns, for each concrete call, a set of callee contexts. The shipped analyses return one alternative per call, so for them the set has at most one element for a fixed caller context. An entry operation that returns two overlapping alternatives admits one concrete call in two contexts (`ov_two_contexts_admitted`, evaluated for a hand-built Interval specification).

The set may also be empty. Under entry-state contexts it is empty for every caller store that the analysis's state at the call site does not describe, for instance at a call that the analysis considers unreachable. There an empty set loses nothing, because the claim contains no such store. At a call that the program actually makes, however, an entry operation that returned no alternative would leave the callee unanalyzed and the continuation at $\bot$. In the counterexample program, which has no branches, every run reaches the continuation (`ov_empty_no_callee_context`, `ov_empty_continuation_bot`). The coverage contract therefore has to require a context at every call whose caller store the claim contains.

Definition 4.6 (Context policy, `type_synonym call_context_policy`)**.** A context policy maps a concrete call to the set of callee contexts it admits (Figure 4.4).

```
(* src/Program_Model/CFG/Collecting/Activation_Trace_Context.thy *)                Isabelle
type_synonym 'c call_context_policy =
  "cfg_node ⇒ 'c ⇒ call_info ⇒ store ⇒ store ⇒ 'c set"
```

Figure 4.4: The declaration of `call_context_policy`, lifted from the theory.

A context policy is a function on concrete calls and is never executed. It names, for each concrete call, the contexts in which a claim has to cover the callee. Each argument of a policy *adm* serves some policy. The call site lets call strings record where a call happened, so with call strings of length one, `bump(5)` enters the context that consists of its call site in `main`. The caller's context lets call strings extend the caller's history and lets entry-state contexts read the abstract state computed for the caller in that context, and the call information is what the entry operation needs to compute an abstract entry state. The two stores serve entry-state contexts, whose abstract entry state must describe the entered store, the callee's store after the formals have been bound (`bump(5)` enters with $n = 5$), and whose alternative must describe the caller store that the return later combines with the callee's result. On the graph, the policy is applied to the call edges, with the entered store that the edge's entry transfer `call_enter` computes.

Definition 4.7 (Context of a trace, `inductive activation_context_rel`)**.** The relation `activation_context_rel` relates each trace to the contexts its activation may carry, which can be none, one, or several. The root activation carries the initial context c_0. A called activation carries a context c' if its caller carries some context c and c' lies in the set that *adm* admits for the call from c. A resumed activation carries the contexts that the caller had before the call.

The contexts of an activation are thus fixed when it is created.[2] Since a return continues exactly the caller that created the callee, the result of a callee that carries c' flows back only to the callers whose calls admitted c'. A claim for a context therefore needs to cover only the executions in that context. The node collecting semantics can now be split by context. A store belongs to context c at node v if some valid trace ending at v with that store carries c.

Definition 4.8 (Activation collecting semantics, `definition activation_collect`)**.**

```
(* src/Program_Model/CFG/Collecting/Activation_Trace_Context.thy *)                Isabelle
definition activation_collect ::
  "kenv ⇒ 'c call_context_policy ⇒ 'c ⇒ (var_id, exp) gcfg
     ⇒ store set ⇒ cfg_node ⇒ 'c ⇒ store set"
    ("𝒜_,_,_,_,_") where
  "activation_collect κ adm c₀ g S v c =
     {sink_store t | t. t ∈ 𝒯_κ,g,S ∧ sink_node t = v
                        ∧ activation_context_rel κ adm c₀ g t c}"
```

Its core VIMP instance $\mathcal{A}_{\kappa,\mathtt{adm},c_0,g,S}$ `v c` (`activation_collect`) is the *activation collecting semantics* at v and c, and the claim for the unknown $[v, c]$ must contain it. In the running example under interval entry-state contexts, a context is the list of abstract values of the formals, here only n. Thus `bump(5)` gets the context $[[5, 5]]$ and `bump(4)` the context $[[4, 4]]$. At `bump`'s result node, the activation collecting semantics of $[[5, 5]]$ holds the final store

[2] The explainer page draws, for each policy, the contexts that the analyzer creates for one program.

with result 6, that of $[[4, 4]]$ the final store with result 5, and the node collecting semantics both. Stores reached in two contexts lie in both sets.

The split makes the claims more precise. A single claim for the result node must contain both stores, so an interval analysis can at best bound the result by $[5, 6]$, while a claim per context only has to contain 6 in $[[5, 5]]$ and 5 in $[[4, 4]]$. No store is lost as long as every valid trace carries at least one context, which the coverage contract of the next section guarantees.

Under entry-state contexts, the policy reads the abstract state that the analysis computed for the caller, so the activation collecting semantics depends on the analysis it is meant to justify. This dependence is harmless. The valid traces and the node collecting semantics do not mention any policy. The policy only decides how their stores are distributed over contexts, and under the totality condition of the coverage contract it cannot drop a store. The union of the activation collecting semantics over all contexts then equals the node collecting semantics for every such policy (Section 4.4, `node_collect_eq_Union_activation_collect`). The analyzer itself never evaluates a policy. It chooses callee contexts from the abstract entry states it computes, a step called routing, and Section 8.2 proves that this choice agrees with the policy.

4.4. The coverage contract thy

The goal is a claim `cover`, with a set of stores `cover v c` for each node v and context c, that contains $\mathcal{A}_{\kappa,\mathtt{adm},c_0,\mathtt{g},\mathtt{S}}$ `v c` everywhere. For the analyzer, `cover v c` is the concretization of the abstract value computed for $[v, c]$. The coverage contract reduces the goal to local conditions on `cover` that mention only stores, and it is therefore the interface between the analyzer and the concrete semantics.

Valid traces grow by four rules, so it is enough to check the claim locally against the same four cases. There is one obligation per rule: INIT for the root activation, INTRA for a local edge, CALL for the entry into a callee, and RETURN for the continuation after a callee finishes. Each obligation other than INIT looks at one step of a run and requires the claim to contain the store after the step whenever it contains the stores before it. Figure 4.5 shows these obligations at one call, together with a fifth, TOTAL.

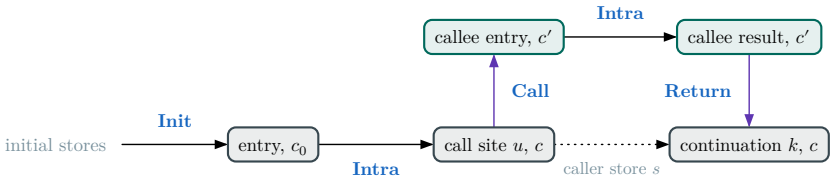


Figure 4.5: The obligations at one call (schematic). Boxes are (node, context) pairs, and solid arrows show where each obligation applies, INTRA repeatedly along an activation path. The dotted arrow is no obligation. It shows the caller store s that RETURN combines with the callee's result in the admitted context c'. TOTAL demands that at least one c' exists.

RETURN pairs every caller store s that the claim contains at u with every callee result t that the claim contains in a context c' admitted for the call from s, and not only with the

result that the call from s produced. The context is the only link between caller and callee stores, so finer contexts make these pairs closer to the real ones.

The fifth obligation, TOTAL, concerns a call at a call site u in a caller context c from a caller store s. If $s \in$ `cover u c`, TOTAL requires the policy to admit at least one callee context for the call from s (`call_context_total_on`). The other four obligations do not suffice without it (`total_dropped_unsound`). At a call that gets no context, CALL and RETURN hold vacuously, so nothing constrains the claim at the continuation, while the resumed caller keeps its contexts. A store that the caller reaches after the return can therefore lie outside the claim.

Asking for contexts at every call would be too strong for entry-state policies, whose sets are empty for the caller stores that the analysis excludes (Section 4.3). Asking for them only at reachable calls would be weaker, since the claim contains every reachable caller store, but an analysis cannot decide which calls are reachable. It can only inspect its own claim, so TOTAL is stated over the stores the claim contains. Call strings, including the context-insensitive length zero, admit a context for every call and meet TOTAL for every claim (`call_context_total_on_of_fun`). For entry-state contexts it follows from the coverage of the analysis's entry operation. Isabelle states the five obligations as the assumptions of the locale `activation_coverage` (Figure 4.6). A *locale* fixes parameters and assumptions. Interpreting it for an instance proves the assumptions and yields the locale's theorems for that instance [69]. An analysis therefore becomes sound by interpreting a locale.

```
(* src/Program_Model/CFG/Collecting/Activation_Trace_Abstract.thy *)                    Isabelle
locale activation_coverage =
  fixes κ :: kenv
    and g :: "(var_id, exp) gcfg" and S :: "store set"
    and cover :: "cfg_node ⇒ 'c ⇒ store set"
    and adm :: "'c call_context_policy"
    and c₀ :: 'c
  assumes INIT[intro]: "⋀s. s ∈ S ⟹ s ∈ cover (cfg_entry g) c₀"
    and INTRA[intro]: "⋀u a v c s s'. (u, a, v) ∈ intra g
        ⟹ s ∈ cover u c ⟹ s' ∈ edge_step κ a s ⟹ s' ∈ cover v c"
    and CALL[intro]: "⋀u dst pars args p cont c c' s.
        (u, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g
        ⟹ s ∈ cover u c
        ⟹ c' ∈ adm u c (call_info_of (CallEdge dst pars args) p) s
             (call_enter κ (CallEdge dst pars args) s)
        ⟹ call_enter κ (CallEdge dst pars args) s ∈ cover (FunctionEntry p) c'"
    and RETURN[intro]: "⋀u dst pars args p cont c c' p' s t es.
        (u, CallEdge dst pars args, FunctionEntry p, cont) ∈ calls g
        ⟹ s ∈ cover u c
        ⟹ admits_call_context κ g adm u c p' s es c'
        ⟹ t ∈ cover (FunctionResult p) c'
        ⟹ combine_collect κ dst s t ∈ cover cont c"
    and TOTAL: "call_context_total_on κ cover adm g"
```

Figure 4.6: The declaration of `activation_coverage`, lifted from the theory. In RETURN, `admits_call_context g adm u c p' s es c'` states that some call edge from `u` into p' enters the store `es` from `s` and that the policy admits c' for this call. p' and `es` range over every call edge out of `u`. On compiled programs a call site has one call edge (`compile_prog_calls_source_unique`), so $p' = p$.

Theorem 4.9 (Context-indexed soundness, `activation_collect_sound`)**.** *If the claim meets the five obligations of* `activation_coverage`*, then for every node v and context c, the activation collecting semantics at v and c is contained in the claim* `cover v c`*.*

```
(* src/Abstract_Interpreter/Framework/Activation/Activation_Backbone.thy *)   Isabelle
theorem activation_collect_sound:
  assumes "activation_coverage κ g S cover adm c₀"
  shows "𝒜κ,adm,c₀,g,S v c ⊆ cover v c"
```

The proof is an induction over the four rules of valid traces, with a strengthened claim about each trace t. The trace carries at least one context, and `sink_store` t lies in the claim at `sink_node` t for every context t carries. The induction proves this for the whole chain of callers of a trace, because the ret rule needs it for the caller as well as for the callee. Root, intra and call follow from INIT, INTRA and CALL, and in the call case TOTAL supplies the context of the new activation, since the caller's store lies in the claim. The ret case is the hard step. For each context c of the resumed caller, the caller's store lies in the claim at the call site in c, so TOTAL admits some c' for the call. The callee carries c', because its contexts are exactly those admitted from its caller's contexts along its own entry edge, and its final store therefore lies in the claim at the result node in c'. RETURN combines the two stores in c.
The strengthened claim also gives every valid trace at least one context, so the activation collecting semantics of all contexts together is exactly the node collecting semantics.

Theorem 4.10 (Exhaustive contexts, `node_collect_eq_Union_activation_collect`)**.** *Under the five coverage obligations, including* TOTAL*, the node collecting semantics at every node v is the union of the activation collecting semantics at v over all contexts.*

```
(* src/Program_Model/CFG/Collecting/Activation_Trace_Abstract.thy *)          Isabelle
theorem node_collect_eq_Union_activation_collect:
  "𝒞 v = (⋃c. 𝒜 v c)"
```

Corollary 4.11 (Claims cover the collection, `node_collect_covered`)**.** *Under the five coverage obligations, every store of the node collecting semantics at v lies in the claim* `cover v c` *of some context c.*

```
(* src/Program_Model/CFG/Collecting/Activation_Trace_Abstract.thy *)          Isabelle
corollary node_collect_covered: "𝒞 v ⊆ (⋃c. cover v c)"
```

Apinis et al. make the corresponding observation for the abstract side [44, §3]. After local solving, the abstract values that reach a program point are bounded by the join of the solution's values there over the contexts the solver encountered.

Every finite source execution of a well-formed program is matched by a graph run (Chapter 3), which a valid activation trace represents (Section 4.2.3). Its contexts place its final store in their activation collecting semantics (Section 4.3), which every claim meeting the coverage contract contains. The following chapters construct such claims by abstract interpretation.

PART III

Analyses

5. Abstract Domains

Chapter 4 reduced soundness to five local obligations on a claim, which assigns each node and context a set of stores. Together they guarantee that the claim contains every store an execution in that context reaches at the node (`activation_collect_sound`). These sets are usually infinite. An analyzer, however, computes with finite abstract values, and its solver (Section 2.7) compares, joins, widens, and narrows them without knowing what they mean. Recall from Section 2.3 that an abstract value a denotes a set $\gamma(a)$. A value of a numeric domain such as Sign or Interval denotes a set of integers, and an abstract state d denotes a set of stores, $\gamma(d) \subseteq (\texttt{var_id} \to \mathbb{Z})$. We write γ for both, as the theories do.

An arbitrary lattice of abstract values is not enough. First, the order must agree with the meaning. The solver proves only inequalities $a \leq b$, while the obligations of Chapter 4 are inclusions between sets. The law

$$a \leq b \Longrightarrow \gamma(a) \subseteq \gamma(b)$$

turns each such inequality into an inclusion (`gamma_mono`, Figure 5.1). Second, a value can denote no integer without being the bottom element, as the interval $[5, 3]$ does. An analyzer that recognizes only the bottom element as empty stays sound, but it loses precision, because after the next join the result no longer records that the program point is dead.

This chapter therefore asks what an abstract domain must provide so that the analyzer can compute with it, every result contains the stores it stands for, and the representation itself loses no precision, such as the fact that a program point is unreachable.

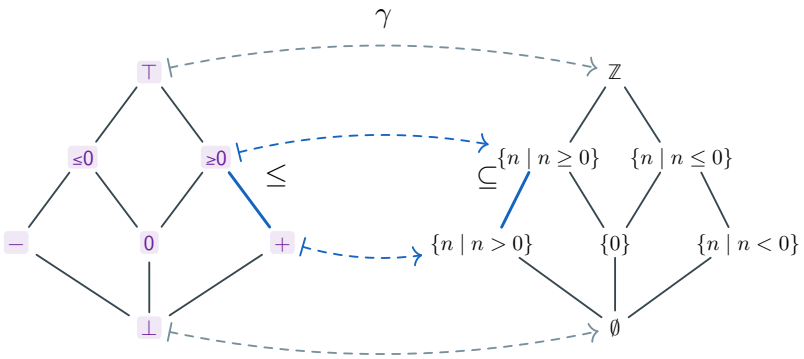


Figure 5.1: The Sign domain (left) and the sets of integers its values denote (right, mirrored), both ordered bottom to top. The dashed arrows show four instances of γ. Since γ is monotone, the blue step $+ \leq \geq 0$ becomes the inclusion $\gamma(+) \subseteq \gamma(\geq 0)$. Adapted from [7, Fig. 13.5].

5.1. What an abstract value means thy

A domain is a type A of abstract values, its *carrier*. The solver compares values, joins them where control flow merges, starts from a least value $\bot$, and extrapolates with the

widening ∇ and the narrowing Δ of Section 2.5. The analysis also needs a greatest value $\top$ for unknown values, an emptiness test, and a concretization $\gamma : A \to \mathcal{P}(\mathbb{Z})$ with $\gamma(\bot) = \emptyset$ and $\gamma(\top) = \mathbb{Z}$. A *numeric domain* (`numeric_domain`) adds the two requirements of the chapter opening as laws. The concretization is monotone (`gamma_mono`), and the emptiness test holds of a value exactly when its concretization is empty (`is_empty_correct`). Isabelle states such an interface as a *type class*, which collects operations and laws that an instance proves once for a carrier [70]. Unlike a locale (Section 4.4), a class has at most one instance for each type.

Intervals (Section 2.3) are this chapter's running example thy . As in Nipkow and Klein [7, §13.8.1], bounds are extended integers `eint`, and $\gamma([l, u]) = \{n \in \mathbb{Z} \mid l \le n \wedge n \le u\}$. On nonempty pairs the order and join are those of Section 2.3. Nipkow and Klein identify all empty pairs by a quotient type. Voblint's `ivl` is a plain datatype of bound pairs, so it must handle them itself. Every pair with $l > u$ is empty, and so are $[+\infty, +\infty]$ and $[-\infty, -\infty]$. The bottom value $\bot = [+\infty, -\infty]$ is only one of these, and an empty pair such as $[5, 3]$ is neither $\bot$ nor below it.

Arithmetic therefore replaces every empty result by $\bot$ (`normalize_ivl`). Otherwise $[1, 0] + [1, 1] = [2, 1]$ would keep creating new empty values, and the solver, which tests whether a value changed (Section 2.7), could fail to stabilize. Normalizing never changes the denoted set (`normalize_ivl_gamma`). The meet, however, must not normalize. It intersects the bounds, so $[0, 3] \sqcap [5, 9] = [5, 3]$, the greatest lower bound in the order on pairs. Returning $\bot$ would break this property, since $[5, 3]$ lies below both operands but not below $\bot$ (`meet_ivl_normalized_breaks_greatest`). The emptiness test therefore inspects the bounds (`is_bottom_ivl`) instead of comparing with $\bot$.

The widening is the standard one of Section 2.5 (`widen_ivl_core`), except that widening with $\bot$ returns the other operand. Every nonempty interval exceeds both bounds of $[+\infty, -\infty]$, so the standard rule would widen $\bot \nabla [0, 0]$ to $\top$. The narrowing (`narrow_ivl_td`) refines only infinite bounds, so $[0, +\infty] \Delta [0, 10] = [0, 10]$, and since each bound leaves infinity at most once, a narrowing sequence is finite.

5.2. Values at a machine kind thy

The domains of Section 5.1 denote sets of mathematical integers and compute with the arithmetic of $\mathbb{Z}$. A VIMP variable, however, holds a value of its machine kind k, and every arithmetic node wraps its result into the kind's range with `ik_norm` (Section 3.2). The classical operations are therefore unsound. In the program of Figure 5.2, interval arithmetic over $\mathbb{Z}$ bounds x by $[0, +\infty]$ after the guard, hence y by $[1, +\infty]$, and would prove `y > 0`. At `i32` the check fails for $x = 2^{31} - 1$, where `x + 1` wraps to -2^{31}.

Some facts, however, survive wrapping, and which of them an analysis keeps depends on the domain. Besides Sign and Interval, two classical domains express such facts. Parity records whether a value is even or odd, and Congruence records a residue class $c + m\mathbb{Z}$, the integers congruent to c modulo m [71]. For a 32-bit unsigned `n`, the program computes $a = 4n \bmod 2^{32}$ and $b = 3n \bmod 2^{32}$. Wrapping subtracts a multiple of 2^{32}. Since 4 divides 2^{32}, a stays a multiple of 4, and `a != 1` holds in every run. Since 3 does not divide 2^{32}, b loses its divisibility by 3, and $n = 2863311531$ gives $b = 1$. This section asks how a classical domain is used at machine kinds so that its results describe the wrapped values and keep the facts that wrapping preserves.

```
1  fun main() {                                                    VIMP ↗
2    i32 x;
3    u32 n;
4    if (x >= 0) {
5      i32 y = x + 1;
6      __voblint_check(y > 0);
7    }
8    u32 a = 4 * n;
9    u32 b = 3 * n;
10   __voblint_check(a != 1);
11   __voblint_check(b != 1);
12 }
```

Figure 5.2: Arithmetic at machine kinds. The first and last checks fail in some run, and `a != 1` holds in every run. Interval reports `y > 0` `UNKNOWN` with `interval: y=[-2147483648,2147483647]` (claim `dom-kinds-interval`). Congruence reports `a != 1` `PROVED` with `congruence: a=4ℤ` and `b != 1` `UNKNOWN` with `congruence: b=⊤` (claim `dom-kinds-congruence`).

Voblint keeps the classical carriers and attaches the kind outside them. A value of type `kd` is KBot, KTop, or a carrier value a tagged with a kind k, and it denotes the integers of $\gamma(a)$ that k represents:

$$\gamma(\mathsf{KD}(k,a)) = \gamma(a) \cap \mathtt{ik_range}\, k, \quad \gamma(\mathsf{KBot}) = \emptyset, \quad \gamma(\mathsf{KTop}) = \mathbb{Z}.$$

Values of different kinds join to KTop. Goblint likewise parameterizes every operation of its integer domains by the kind it computes at.

An operation at kind k computes the classical result and then *normalizes* it to k, the abstract counterpart of wrapping. If $n \in \gamma(a)$, the wrapped integer `ik_norm` $k\, n$ must lie in the concretization of the normalized value. Wrapping is the identity inside the range, so only a result that may leave the range needs work.

For an unsigned kind, C prescribes wrapping. For a signed kind, C leaves overflow undefined, and VIMP wraps (Section 3.4). The signed-overflow policy (`signed_overflow_policy`), after Goblint's option `sem.int.signed_overflow`, chooses how an analysis treats a signed result that may leave its range. Under the default Assume_Top the result becomes the kind's top, and under Assume_Wraparound it wraps as an unsigned result does. Both choices are sound, because the semantics wraps under either policy and the kind's top contains the wrapped value. Every soundness theorem therefore holds for either policy. Goblint's third setting, which assumes that no signed overflow occurs, has no counterpart, since it would drop values the semantics produces.

Each domain normalizes according to what its values can express. Interval keeps a result inside the range. Otherwise, if the kind wraps under the policy, it wraps a finite interval that is narrower than the modulus 2^w and whose wrapped endpoints stay ordered. Wrapping shifts such an interval as a whole by one multiple of 2^w (`ik_norm_interval_shift`), so converting $[260, 265]$ to `u8` gives $[4, 9]$. Every other overflow gives the kind's whole range. In Figure 5.2, `x + 1` yields $[1, 2^{31}]$. Its endpoints wrap to 1 and -2^{31}, which are out of order, so the shift rule does not apply and y gets the whole range under both policies.

Congruence relies on the same shift. Wrapping changes a value by a multiple of 2^w, so a class $c + m\mathbb{Z}$ is preserved modulo $\gcd(m, 2^w)$, and a constant ($m = 0$) stays a constant. In Figure 5.2, $4n$ keeps the class $4\mathbb{Z}$ since $\gcd(4, 2^{32}) = 4$, while the class $3\mathbb{Z}$ of $3n$ becomes $\mathbb{Z}$ since $\gcd(3, 2^{32}) = 1$. Under Assume_Top at a signed kind, Congruence cannot tell whether a sum or product wraps and returns its top. For example, `a + 1` with $a = 2^{31} - 1$ is -2^{31}

under Assume_Wraparound and $\top$ under Assume_Top (02-congruence_wraparound and its assume_top_twin).

Sign carries no magnitude, so it rules out a wrap only where the operand signs do, as described below. Every other nonzero sign normalizes to the kind's top, $\top$ at a signed kind and ≥ 0 at an unsigned one, and zero keeps its sign. Parity, in contrast, computes classically at every kind and under both policies, because wrapping subtracts a multiple of an even modulus, which preserves parity (even_ik_norm_iff).

The domain interface states these requirements as one law per primitive, quantified over the policy p. For arithmetic it reads (arith_ik_sound)

$$m \in \gamma(a) \wedge n \in \gamma(b) \Longrightarrow \texttt{ik_norm}\ k\ (\texttt{aop_eval}\ op\ m\ n) \in \gamma(\texttt{arith_ik}\ p\ k\ op\ a\ b),$$

where the theory passes one more argument, a flag stating that both operands lie in k's range. A carrier value denotes mathematical integers, so the law alone admits operands beyond the range, such as 2^{40} at i32. The evaluator sets the flag when both operand expressions have kind k, since every value of kind k lies in its range. Under the flag, some operations cannot leave the range. At a signed kind, a sum of operands with opposite signs lies between them, a product with a zero operand is zero, and a remainder is no larger in magnitude than its operands, so Sign keeps the classical sign of these operations. Congruence keeps the classical class of a remainder and of a quotient whose divisor excludes -1 at every kind. The law does not require a result to lie in the range, since the tag adds that restriction.

Two carrier operations also depend on the kind. The emptiness test must recognize a value whose integers all lie outside the kind, such as the sign $-$ at u8, and the interval widening needs the kind's limits. The class ik_domain therefore asks each carrier for a widening, a narrowing and an emptiness test at a kind, with the laws of their kind-free counterparts, and the tagged value applies them at the kind of its operands. The interval widening at a kind moves a growing bound to the kind's limit instead of to infinity, or to the new bound if that already lies beyond the limit, and the narrowing replaces only a bound that sits at that limit. In a loop that starts at $v = 1$ and adds 2 while $v < 51$, the loop head widens the upper bound of v to the limit of i32, and narrowing then lowers it to 52, as the state at the check `v <= 52` in the loop body shows (claim dom-stride2-interval). The kind-free narrowing refines only infinite bounds and would keep the limit. On intervals inside the range, each bound grows at most once under widening and leaves the limit at most once under narrowing. This argument bounds the widening and narrowing sequences of one value. It is not a lemma of the theories, and it does not make the solver terminate. Sign, Parity and Congruence widen by the join and narrow by the meet at every kind.

Wrapping loses precision that the classical domains would keep. In Figure 5.2 no domain avoids the loss, because b takes every u32 value, since 3 is invertible modulo 2^{32}, and y takes both 1 and -2^{31}, so only the top describes these values. Elsewhere the loss comes from the domain or the policy. Sign loses the sign of a sum whenever both operands may be positive, even where the program keeps its values small (01-sign_cannot_bound_magnitude). Under Assume_Top, Congruence alone loses the class of every signed sum and product, including those that cannot wrap, because its values do not bound magnitudes. Domains can, however, be combined so that they refine each other, and then the bounds of one component tell another that no value wraps.

5.3. From values to stores

A numeric domain describes the possible values of one variable. The obligations of Chapter 4, however, concern sets of stores `store` $=$ `var_id` $\to \mathbb{Z}$. This thesis uses two forms of abstract state. A *non-relational* state gives each variable its own abstract value and admits a store if each variable's value lies in its own abstract value. A *relational* state records constraints between variables, such as $x \leq y$, and admits a store if the store satisfies them. Neither is stronger than the other. For the stores with $0 \leq x \leq y \leq 5$ (Figure 5.3), the best non-relational state over intervals, $\{x \mapsto [0,5], y \mapsto [0,5]\}$, misses $x \leq y$ and admits $x = 5, y = 0$. The relational fact $x \leq y$ bounds neither variable and admits $x = y = -7$. Together they describe the set exactly.

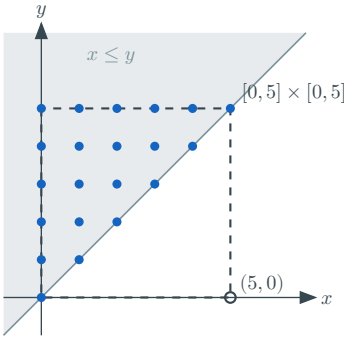


Figure 5.3: Stores over two variables as points (x, y). The stores with $0 \leq x \leq y \leq 5$ are the dots. The best non-relational state denotes the square (dashed), which also admits $(5, 0)$. The relational state $\{x \leq y\}$ denotes the half-plane above the diagonal (shaded), which is unbounded. Their intersection is exactly the dots. Illustrative.

An *abstract state* is a type D with the order and join the solver needs and a monotone concretization that maps each state d to the set $\gamma(d)$ of stores it describes. Isabelle overloads one constant, `gamma_S`, written $\gamma(d)$, for every kind of state.

5.3.1. A non-relational state thy

A non-relational state (`gabs_state`) maps every variable x to an abstract value $d(x)$. It admits a store if every variable's value lies in the concretization of its abstract value, $\gamma(d) = \{s \mid \forall x.s(x) \in \gamma(d(x))\}$ (`gamma_state`). Order and join work variable by variable, as for any function type in HOL (Figure 5.4), so this concretization is monotone too. The non-relational analyses use kind-tagged values (Section 5.2), and their concretization also requires every variable to hold a value of its own kind (`in_kinds`).

Some program points are reached by no execution, and the analysis should recognize them from their state. Each layer has its own least element. A domain has $\bot$, which for intervals is one empty value among several (Section 5.1), the tagged values have KBot, and the pointwise order has the *bottom state*, which maps every variable to the least value. For pointwise states, recognizing unreachability is difficult, because every state with one empty variable describes no store, and only one of them is the bottom state (Figure 5.4). Treating only the bottom state as unreachable misses most cases, since a branch condition usually empties a single variable, as in $\{x \mapsto \bot, y \mapsto \top\}$. Treating every empty state as unreachable is correct at the moment of the test, but a later join of two differently empty

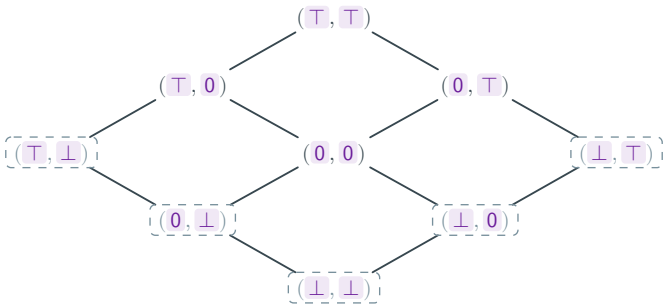


Figure 5.4: Pointwise states $(d(x), d(y))$ over two variables, with values from the fragment $\bot \leq 0 \leq \top$ of Sign, ordered variable by variable. The dashed states have a $\bot$ component and denote no store, although only the lowest one is the bottom state.

states can give a nonempty one (Figure 5.5). This is sound, since it only adds stores, but it loses the fact that the point is unreachable.

```
1 fun main() {                                                    VIMP
2   i32 x = __voblint_nondet_int();
3   i32 y = __voblint_nondet_int();
4   if ((x == 0 && x == 1) || (y == 0 && y == 1)) {
5     __voblint_check(false);
6   }
7 }
```

Figure 5.5: A branch no execution enters, since no integer equals both 0 and 1. Sign refines the left disjunct to $\{x \mapsto \bot, y \mapsto \top\}$ and the right one to $\{x \mapsto \top, y \mapsto \bot\}$. Both are empty, but their pointwise join is $\{x \mapsto \top, y \mapsto \top\}$. The branch turns each empty arm into the bottom state before the join, and the analyzer reports the check in the branch as unreachable (claim `dom-disjunct-sign`).

Voblint therefore makes unreachability a value of its own, as Nipkow and Klein do with an option type [7, §13.7]. Goblint likewise lifts the local state by an outer bottom element for dead code (`Analyses.Dom`), and a transfer that finds its path unreachable raises the `Deadcode` exception. A lifted state (`lifted`) is either Bot, meaning unreachable, with $\gamma(\text{Bot}) = \emptyset$, or Lifted$(d)$ for an ordinary state d. After every transfer, the analysis replaces an empty result by Bot (`normalize_lift`). The test relies on the exact emptiness test of the domain, since a pointwise state is empty exactly when some variable's value is (`is_empty_state_iff_gamma_state_empty`). The replacement only exchanges one representation of the empty set for another (`gamma_exec_normalize_lift`), so it cannot make the analysis unsound.

Once a state is Bot, it stays Bot, because transfers skip it and a join with Bot returns the other operand. The bottom state and Bot differ in role. The bottom state is one ordinary pointwise state among the empty ones, while Bot is the single value that marks a point as unreachable. In Figure 5.5, both arms are joined inside one branch transfer, so the test after the transfer comes too late, and Section 5.6 shows how the branch avoids this.

5.3.2. A relational state thy

A relational state records relations that the pointwise form cannot express, so its concretization maps a whole state to a set of stores. The type `order_rel` records a set P of variable pairs, where (x, y) stands for the constraint $x \leq y$, and a flag t stating that the

store lies in the set K of stores in which every variable holds a value of its kind (`in_kinds`). The concretization is `gamma_ord`:

$$\gamma(\mathsf{OFacts}(t, P)) = \{s \mid (t \longrightarrow s \in K) \wedge \forall (x, y) \in P.s(x) \leq s(y)\}, \quad \gamma(\mathsf{OBot}) = \emptyset.$$

The flag lets assignments record more pairs. If every variable holds a value of its kind, a copy `x = y` between two variables of one kind stores the value unchanged, so the analysis may record both $x \leq y$ and $y \leq x$. The initial state sets the flag, since every initial store lies in K. The top value leaves it unset, because it must describe every store. Below, the pair (x, y) is written $x \leq y$, and a value is written as its set of pairs when the flag does not matter.

More pairs describe fewer stores, so the order is reverse inclusion (`less_eq_order_rel`), with OBot at the bottom and $\top = \mathsf{OFacts}(\mathrm{False}, \emptyset)$, which constrains nothing. The join keeps the pairs both operands share, $\mathsf{OFacts}(t, P) \sqcup \mathsf{OFacts}(u, Q) = \mathsf{OFacts}(t \wedge u, P \cap Q)$ (`sup_order_rel`). The widening is the join. The solver's widening contract (`widening`) asks only for a value above both operands, which alone does not force termination, but here an ascending chain can only drop pairs from a finite set. The narrowing returns the newly computed value, which the narrowing law accepts since it lies between the two operands.

The carrier does not close its pairs under transitivity (Figure 5.6). The values $\{x \leq y, y \leq z\}$ and $\{x \leq y, y \leq z, x \leq z\}$ therefore denote the same stores but are different values, and the join of $\{x \leq y, y \leq z\}$ and $\{x \leq y, x \leq z\}$ is $\{x \leq y\}$, although both operands imply $x \leq z$. Pairs enter only at guards and assignments and relate two of the program's variables, so P is finite in every value the analysis constructs. The executable analyzer relies on this to store P as a list.

The store that maps every variable to 0 satisfies every pair, and 0 lies in the range of every kind. Every $\mathsf{OFacts}(t, P)$ therefore denotes at least one store, and only OBot denotes the empty set. Unlike the pointwise form, the carrier has a single representation of unreachability.

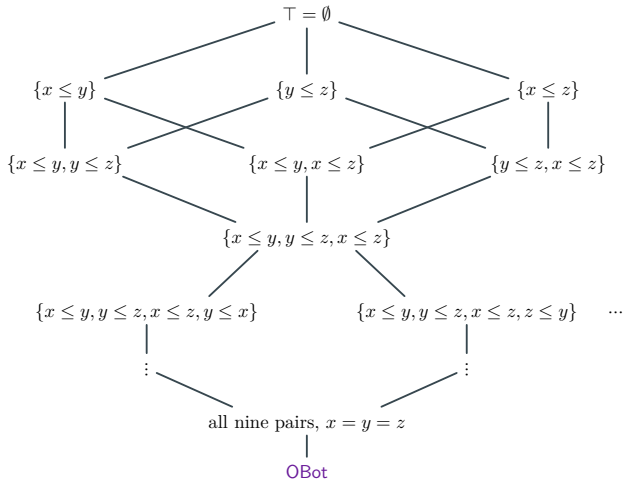


Figure 5.6: The relational states over the variables x, y, and z with the flag unset, ordered bottom to top by reverse inclusion. Each OFacts value is drawn as its set of pairs. The upper part shows all values built from $x \leq y$, $y \leq z$ and $x \leq z$. Values with other pairs, such as $y \leq x$, are drawn only below the lowest of these, and the dots stand for the rest. Nine pairs exist, including $x \leq x$, so the values are finitely many, and the lowest OFacts value holds all nine and denotes the stores with $x = y = z$.

A relational state stores no value per variable, so it cannot evaluate an expression to an abstract value. Its transfers update the constraints directly. An assignment `x = e` first forgets every pair that mentions x and then adds the pairs the new value implies. A copy `x = y` within one kind adds $x \leq y$ and $y \leq x$ when the flag is set, and x inherits the pairs of y. Further pairs $x \leq y$ or $y \leq x$ come from asking other analyses how the new value compares with y (Section 7.3). A guard comparing two variables adds the pairs its outcome implies (Section 5.6). The type instantiates the solver's classes, but it is not a numeric domain, so the operations on single values that follow serve only non-relational states.

5.4. Evaluating expressions thy

The carrier operations serve the solver. The analysis also needs operations that mirror the program's arithmetic, comparisons, and branch conditions. We call them the domain's *primitives*.

An assignment `x = e` needs the abstract value of e. Forward evaluation computes it from an abstract state d, as in the generic abstract interpreter of Nipkow and Klein [7, §13.5.3], and its result must contain every concrete result:

$$s \in \gamma(d) \Longrightarrow \texttt{eval}\ e\ s \in \gamma\big([\![e]\!]^\sharp\ d\big).$$

Voblint defines this evaluation once for every domain, using each operation at its machine kind (Section 5.2), and proves the law once (`aeval_sound`). The evaluation may also ask other analyses for the value of a subexpression, and the law then assumes that their answers are true. Section 7.2 makes this premise precise. If $d(x) = [0, 9]$ at a 32-bit kind, for example, Interval evaluates `x + 1` to $[1, 10]$.

A condition is an expression too, and VIMP reads a nonzero value as true. If the abstract value of a condition admits only one truth value, a branch can drop the arm that needs the other. Voblint therefore derives a truth test from the domain's comparison with zero. It answers Some(b) if every integer in $\gamma(a)$ has truth value b, and None otherwise (`truth_ik_sound`). For Interval, $[1, 5]$ is true, $[0, 0]$ is false, and $[0, 5]$ is undecided.

5.5. Answering queries

An assertion `__voblint_check(b)` asks whether b holds in every store that reaches its program point, and the analysis must answer from the abstract state there. Goblint's `assert` analysis asks the active analyses a query about the truth value of b, and Voblint's combined analyzer does the same (Section 7.3). This section shows how a non-relational domain decides b, which is the answer it contributes.

A domain answers two comparisons on abstract values, less(a, b) for $a < b$ and eq(a, b) for $a = b$. A definite answer must hold for every pair of integers the two values denote, and None, meaning unknown, is always allowed (`cop_of_sound`):

$$\text{less}(a, b) = \text{Some}(r) \wedge i \in \gamma(a) \wedge j \in \gamma(b) \Longrightarrow (i < j) = r,$$

and likewise for eq. Interval decides $a < b$ when one interval lies entirely below the other, where a shared endpoint suffices for the answer false, and $a = b$ when the intervals are disjoint or the same single integer. Sign decides $a < b$ whenever the signs separate the values. Parity only refutes equality between an even and an odd value. Congruence decides $<$ only for single integers and refutes equality when two residue classes are disjoint.

A check evaluates its condition b on an abstract state to *true*, *false* or *unknown*. Each comparison in b evaluates its operands to abstract values $\hat{a}$ and $\hat{b}$ (Section 5.4) and asks one of the two comparisons, with the operands swapped or the answer negated where needed:

$$a < b \rightsquigarrow \mathrm{less}(\hat{a}, \hat{b}), \qquad a > b \rightsquigarrow \mathrm{less}(\hat{b}, \hat{a}), \qquad a == b \rightsquigarrow \mathrm{eq}(\hat{a}, \hat{b}),$$
$$a \leq b \rightsquigarrow \neg\, \mathrm{less}(\hat{b}, \hat{a}), \quad a \geq b \rightsquigarrow \neg\, \mathrm{less}(\hat{a}, \hat{b}), \qquad a \neq b \rightsquigarrow \neg\, \mathrm{eq}(\hat{a}, \hat{b}).$$

Negation keeps *unknown*, and the connectives `!`, `&&` and `||` combine the answers of their operands in three-valued logic, so a definite *false* decides `&&` and a definite *true* decides `||`. Any other expression e is read as $e \neq 0$. A new domain therefore gets checks on arbitrary conditions by providing the two comparisons. Their soundness is proved once for every domain (`nonrel_classify_sound`). The answer *true* certifies that b holds in every store the state describes, *false* that it holds in none, and *unknown* makes no claim. In Sign, with x non-positive and y positive, `x <= y && y != 0` is answered from $\mathrm{less}(\hat{y}, \hat{x})$ and $\mathrm{eq}(\hat{y}, \hat{0})$ alone (Figure 5.7). A non-relational analysis that runs alone decides its checks this way (`nonrel_classify`). In the combined analyzer, its query handler answers the question about b with the same evaluation, converted to the shared answer type of Section 7.1.

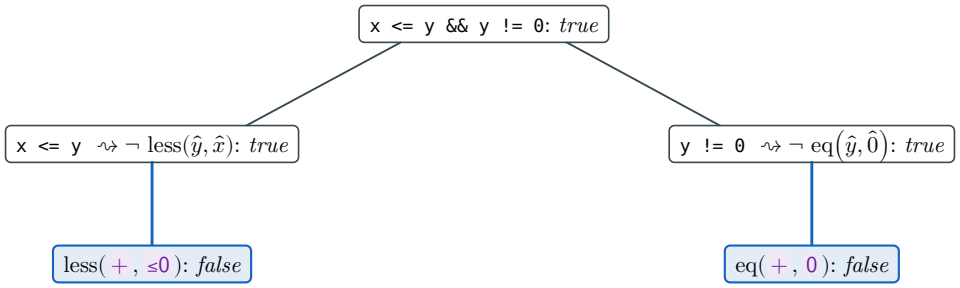


Figure 5.7: How forward evaluation answers a compound condition, in Sign with `sign: x=≤0, y=+` (claim `dom-query-tree-sign`). Each comparison becomes one of the domain's two comparisons (blue), as ⇝ marks; the answer follows the colon. Negation flips a definite answer, and `&&` is true because both operands are, so the check is PROVED.

Both operands of a connective are evaluated in the same state, so for x in $[1, 2]$, `x == 1 || x == 2` stays unknown, although every store satisfies it (claim `dom-disjunction-one-state`). Goblint is more precise here. CIL compiles `&&` and `||` into branches (`cabs2cil.ml`), and where `x == 1` fails, Goblint's interval refinement narrows $[1, 2]$ to $[2, 2]$ (`baseInvariant.ml`, [32, §4.5.1, Rem. 8]). Voblint's interval refinement of a failed equality would also exclude the endpoint, but Voblint evaluates a check in one state and refines only at a guard.

5.6. Learning from a guard thy

When the analysis enters a branch, it knows whether the condition held. On the true arm of `if (0 < x)`, for example, x is positive. Ignoring this is sound, since the state before the branch contains every store that takes either arm, but it costs precision, as the following program shows.

```
1  i32 x = __voblint_nondet_int();                                  VIMP ↗
2  if (x < -1000 || x > 1000) {
3    x = 0;
4  }
5  i32 y;
6  if (0 < x) {
7    y = x;
8  } else {
9    y = - x;
10 }
11 __voblint_check(y >= 0);
```

The first branch keeps x between -1000 and 1000. In this range `- x` cannot wrap, so y ends up as $|x|$ and the check always holds. Without the bound it could fail, since at $x = -2^{31}$, `- x` wraps to -2^{31}. Without refinement, however, the analysis learns nothing from either condition, x and y keep the top value of their kind, and the check stays unknown. Backward refinement runs the condition in reverse, as in the backward abstract interpretation of tests by Cousot [72, §§8.5, 10.1] and the backward analysis of Nipkow and Klein [7, §13.7]. Given abstract operands a_1, a_2 of a comparison that must hold, the domain returns refined operands a_1', a_2' that keep every concrete pair satisfying it. For $<$:

$$n_1 \in \gamma(a_1) \wedge n_2 \in \gamma(a_2) \wedge n_1 < n_2 \Longrightarrow n_1 \in \gamma(a_1') \wedge n_2 \in \gamma(a_2'),$$

and likewise for the other comparison operators (`refine_ik_sound`). An operand that is a sum or a difference passes the refinement on to its own operands by inverting the operation, over the mathematical integers when it cannot overflow and otherwise at its kind under the overflow policy.

The refined operand is then combined with the value known before. Nipkow and Klein use the lattice meet and require it to be exact on the denoted sets. Voblint asks only for an intersection that keeps every integer both operands share (`meet_ik_sound`), for Interval the meet of Section 5.1. In the program above, x starts at $[-2^{31}, 2^{31} - 1]$. The false arm of the first condition refines it to $[-1000, 1000]$, and joining the true arm's 0 keeps that interval. The second condition refines x to $[1, 1000]$ on its true arm and to $[-1000, 0]$ on its false arm, each arm bounds y from the refined x, and the join yields `interval: y=[0,1000]`.

The guard refinement applies the comparison refinements along the Boolean structure of the condition, in the manner of the backward filter of Nipkow and Klein [7, §13.7.1]. It never removes a store at which the guard has the required truth value, provided the answers of other analyses are true, the same premise as for evaluation (Section 5.4) (`nonrel_refine_sound`). This is exactly what the branch needs. A conjunction is refined from left to right, each conjunct once and without iterating to a local fixpoint, so the bounds can depend on the order of the conjuncts. With Interval, `0 < i && i < j && j < 10` bounds j to $[2, 9]$ but i only from below, and the reversed order bounds i to $[1, 8]$ but j only from above (claim `dom-conjunct-order`). The refinements are sound but not the most precise possible. A quotient, for instance, stops the refinement, so Interval cannot learn $y \leq 9$ from `y / 2 < 5`, although it holds in every run.

The branch transfer first drops an arm that the truth test rules out and otherwise applies the refinement. It inherits the refinement's soundness (`nonrel_branch_sound`). A disjunction needs one more step, because its arms are joined inside the transfer, before the test after the transfer can mark them unreachable (Figure 5.5). The refinement therefore replaces each comparison that empties one of its variables by the bottom state. An arm that is a conjunction of comparisons and contains such a comparison then adds nothing to the join, and the test after the transfer turns a joined bottom state into Bot (Section

5.3.1). A guard refines a relational state as well. On the true arm of `if (x < y)`, the state of Section 5.3.2 adds the pair $x \leq y$, losing the strictness, which is sound but less precise, and this refinement satisfies the same statement (`ord_branch_sound`).

5.7. The complete domain interface thy

The operations of this chapter fall into two groups. The *carrier operations* serve the solver. They are the order, join, bottom and top, widening, narrowing and emptiness test of Section 5.1, together with their versions at a kind (Section 5.2). The *primitives* mirror the program's operations at a machine kind. They are the constants and arithmetic of forward evaluation (Section 5.4), the comparisons (Section 5.5), and the comparison refinements and the intersection (Section 5.6). Every non-relational domain supplies both groups as an instance of the class `kind_int_domain`, whose laws are the soundness conditions of this chapter (Figure 5.8).

The class contains no transfer function, because the primitives determine every edge. An assignment overwrites the variable with the abstract value of the right-hand side, converted to the variable's kind, and a branch applies the refinement of Section 5.6. Procedure entry evaluates the arguments in the caller's state, binds them to the formal parameters, and sets every other local of the callee to KTop. These transfers are defined and proved sound once for every instance (`nonrel_spec_sound`), so a new non-relational domain supplies values and primitives only.

5.8. A carrier the interface excludes

Voblint cannot host every domain Goblint offers. The verified solver requires its values to form a `bounded_semilattice_sup_bot` (`TD_side_upd_rule`), whose join is a *least* upper bound (`semilattice_sup`), and the generic framework inherits the requirement. It excludes every carrier in which two values have upper bounds but no least one.

Goblint's exclusion-set domain `DefExc` becomes such a carrier in Voblint's setting. In Goblint, an exclusion value also stores a *bit range* r, which bounds the integers it denotes to those of a given bit width, such as the range of its kind. A Voblint carrier denotes mathematical integers, and the kind's range is attached outside it (Section 5.2), so the carrier must drop r. A non-bottom value is then `Definite n`, denoting $\{n\}$, or `Excluded S` for a finite set S, denoting $\mathbb{Z} \setminus S$ (Figure 5.9). `Definite 1` and `Definite 2` have no least upper bound. Every `Excluded {p}` with $p \notin \{1, 2\}$ lies above both, so by monotonicity of γ a least upper bound u would satisfy

$$\{1, 2\} \subseteq \gamma(u) \subseteq \bigcap_{p \notin \{1,2\}} (\mathbb{Z} \setminus \{p\}) = \{1, 2\},$$

but no value denotes exactly $\{1, 2\}$, since `Definite n` denotes a single integer and `Excluded S` all but finitely many.

Goblint's bit range avoids this argument. Goblint writes a non-bottom value with a range as `Excluded (S, r)`, denoting the integers r admits minus S. If R is that set of integers, then `Excluded (S, r)` with $S = R \setminus \{1, 2\}$ denotes exactly $\{1, 2\}$, although the join of two distinct `Definite` values returns `Excluded ({0}, r)` instead, or excludes nothing if one of them is 0. Voblint could admit this carrier only by carrying the range inside the carrier

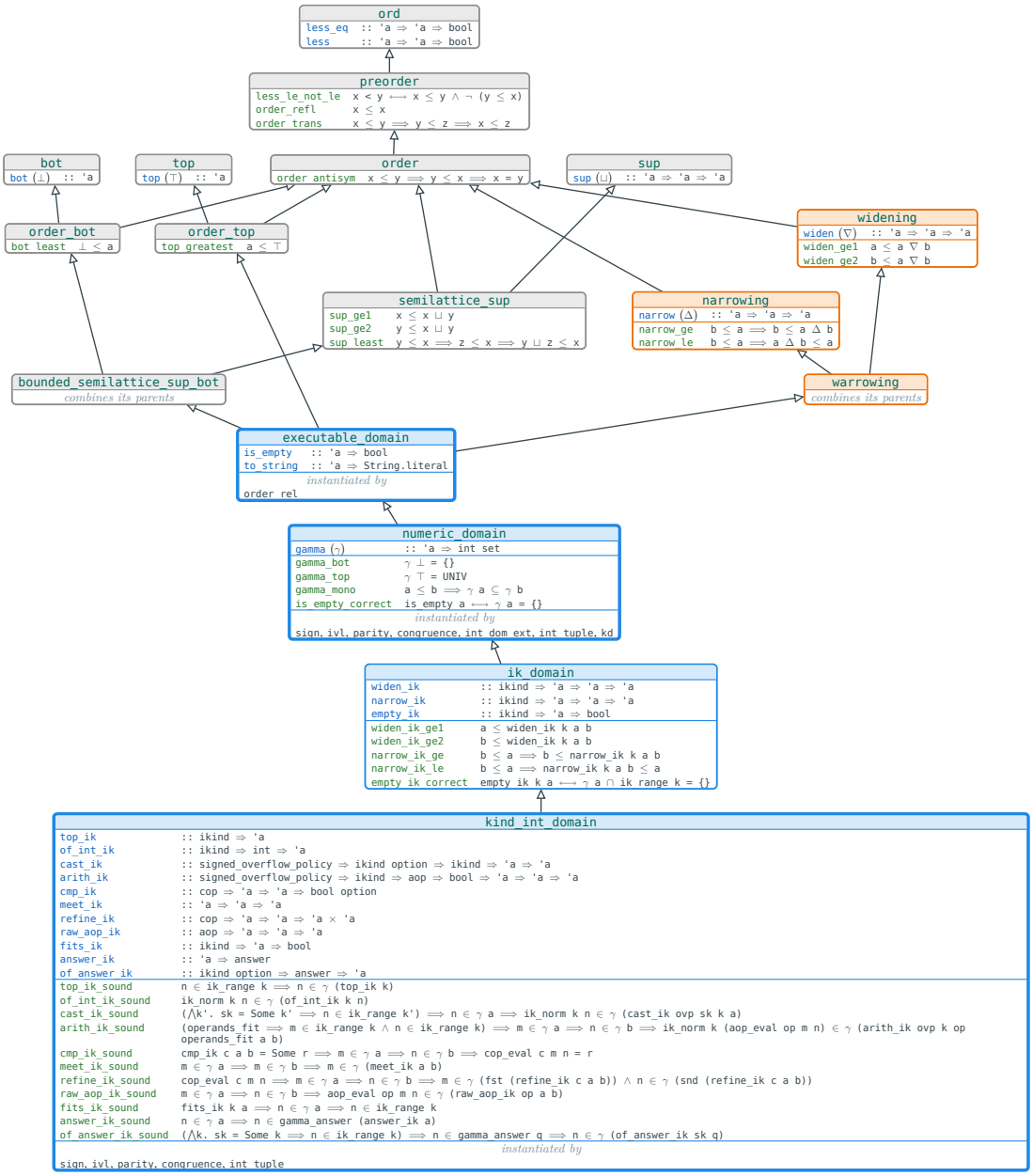


Figure 5.8: What a non-relational domain supplies over its carrier type `'a`, down to the class `kind_int_domain` an analysis instantiates. Each class lists the operations and laws it declares, which the classes below it inherit. Arrows point to what a class extends. A class that carriers instantiate has a heavier border and lists its instances. Rows the text does not discuss: `warrowing` combines widening and narrowing in one update (Section 2.5), `to_string` prints a value, `cast_ik` converts a value to another kind, `raw_aop_ik` computes over $\mathbb{Z}$ for the backward step of Section 5.6, `fits_ik` tests that a value lies in a kind's range, and `answer_ik` and `of_answer_ik` convert to and from the answers of Section 7.1, whose record type `int_dom` instantiates the domain as well. `int_tuple` is the reduced product of Section 11.3. Color gives where a node is declared: ☐ HOL, ☐ the vendored solver, ☐ Voblint.

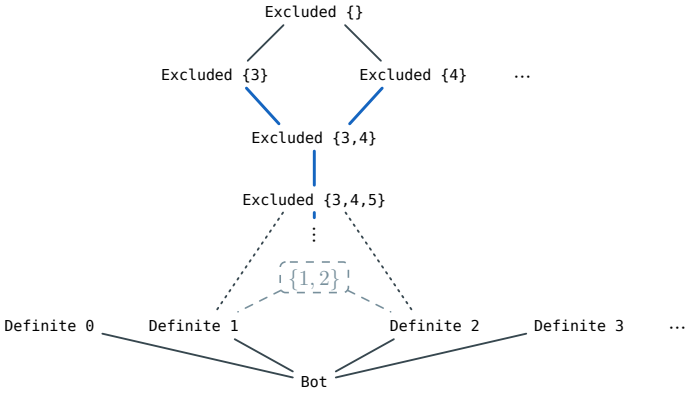


Figure 5.9: Goblint's exclusion sets without a bit range, ordered bottom to top: `Definite n` denotes $\{n\}$ and `Excluded S` denotes $\mathbb{Z} \setminus S$. Every `Excluded S` with $1, 2 \notin S$ lies above `Definite 1` and `Definite 2`. The blue chain of such upper bounds descends forever, and each of its elements lies above both values (dotted). A least upper bound would denote $\{1, 2\}$ (dashed), which no value does. Illustrative, not machine-checked.

value, or by weakening `semilattice_sup` to an upper-bound law throughout the solver and the framework.

The laws of the domain interface make every operation keep the stores its inputs admit, but each concerns a single value or state. Chapter 6 states what an analysis must prove about the edges and calls along which a program moves its state.

6. What an Analysis Supplies

Chapter 5 ended with laws about one abstract state. The obligations of Chapter 4, however, speak about edges, calls, and contexts. Most of what connects the two is the same for every analysis, namely the equations built from the control-flow graph, the calling contexts, and the solver. Sign and Interval differ only in the abstract effect of each program construct. Goblint therefore separates an analysis from the framework that runs it [2, §6], and Voblint makes the same split in its program and in its proof. An analysis supplies a record of abstract operations and proves the analysis contract about them, and the framework then discharges the obligations of Chapter 4 once for every analysis.

6.1. Transfers at edges and calls

An ordinary edge of the control-flow graph carries an action a, such as an assignment or a guard, whose concrete meaning Chapter 3 fixes. For a store s, `edge_step a s` is the set of stores the edge can produce. An analysis describes the stores at a program point by an abstract state d and supplies, for each action a, an abstract transfer $[\![a]\!]^\sharp$. The transfer is sound if it loses no successor:

$$s \in \gamma_D(d) \wedge s' \in \texttt{edge_step a s} \Longrightarrow s' \in \gamma_D\big([\![a]\!]^\sharp\, d\big).$$

This is the abstract counterpart of the obligation INTRA of Chapter 4 for one edge. Unlike Saan's abstract operators [32, §2.4.2], it does not require strictness, $[\![a]\!]^\sharp \bot = \bot$. The interface has one transfer per kind of action, named after the corresponding method of Goblint's `Spec`.

A call cannot be handled by one such transfer. After it returns, the caller's own locals are the ones from before the call, while the globals and the result come from the callee (RETURN). The abstract return must therefore see both sides. As in Goblint, an analysis supplies two operations at a call. *Enter* (`dgs_enter`) gives the abstract state the callee starts from, and *combine*, in two stages, gives the caller's state after the call. Take one call:

```
1 global i32 g;                                                    VIMP ↗
2 fun i32 inc(i32 a) { g = g + a; return a - 1; }
3 fun main() { g = 1; i32 x = 5; i32 y = inc(x); }
```

At the call `y = inc(x)`, enter describes two stores. The *entry state* e describes the store the callee starts with (`g = 1`, `a = 5`). The *resume state* q describes the caller's store (`x = 5`, `g = 1`), which combine later merges with the callee's exit state r (Figure 6.1).

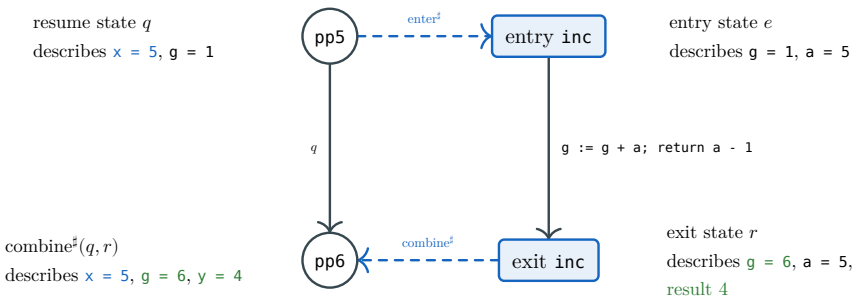


Figure 6.1: Enter and combine at the call `y = inc(x)`, with the concrete store each abstract state must describe (schematic, with the body of `inc` drawn as one edge). The combined state must describe the caller's locals (blue) together with the callee's globals and result (green), as `combine_collect` does.

Combine runs in the two stages of Goblint's `Spec`, mirroring the concrete return of Section 3.3. The first stage, combine_env$^\sharp$, merges the caller's locals from q with the callee's globals from r, as `combine_env` does for stores. The second stage writes the callee's result to the call's destination, here `y`, as `combine_assign` does for stores. Soundness is stated for the two stages together (Section 6.3), so an analysis may divide the work between them as it likes.

For soundness, enter must return a resume state that covers the caller's store and an entry state that covers the entered store (`entry_pairs_cover`). Like Goblint's `enter`, it may return several such pairs, which Goblint's path-sensitivity lifter uses to return one pair per path [73, §5.7]. Every analysis shipped with Voblint returns a single pair, so path-sensitive analyses remain future work.

6.2. Facts without a program point thy

Not every fact belongs to one program point. An analysis may keep, for example, one range for a global variable `g` that holds throughout the run. Seidl et al. describe a Goblint analysis with two domains [2, §6]. The local domain D_L holds facts per program point and context, and D_G holds the values of *analysis globals*, which are unknowns without a program point. An assignment to `g` publishes its value to the analysis global of `g` as a side effect, and an edge that reads `g` depends on that unknown. Goblint's `Spec` accordingly declares a local lattice `D` and a global lattice `G`.

The word *global* thus has two meanings. A *program global* is a variable of the VIMP program, and an analysis global is a fact the analysis keeps flow-insensitively. The two are independent. By default, the numeric analyses keep program globals in the local state and use no analysis global.

In Goblint, a transfer reads and publishes analysis globals through a *manager* argument, which also gives it the current local state. For example, `man.sideg g d` publishes `d` to `g`, and the framework passes such publications to the solver [2, §6]. Voblint adopts this design, and its transfers (`man_transfer`) receive a manager instead of a local state. A Voblint manager holds the local state d at the edge's source, a function that reads the value of an analysis global, a function that publishes a value to one, and a function that asks other analyses (Chapter 7). The transfer returns the new local state d'. The framework records which analysis globals it read and what it published, and turns the reads into dependencies of the

edge's equation and the publications into side effects. The equation of an edge therefore depends on an analysis global only where its transfer reads it, and a transfer that is a pure function of the local state reads only the state at the edge's source and publishes nothing.

6.3. The analysis contract thy

The framework takes an analysis as one parameter, the record `dg_spec` with one field per operation (Figure 6.2), namely the edge transfers of Section 6.1, enter, the two combine stages, and a query handler through which other analyses can ask it for facts. Of the edge transfers, skip♯ handles an edge without effect, assign♯ an assignment, branch♯ a guard, and special♯ a call of a library function such as `__voblint_nondet_int`. body♯ handles the edge from a procedure's entry node into its body, return♯ the edge to its result node, which a `return e` or the end of the body takes, and event♯ a check, which leaves the store unchanged.

```
(* src/Abstract_Interpreter/Framework/Spec/DG_Spec.thy *)                    Isabelle
record ('x,'k,'v,'dl,'dg) dg_spec =
  dgs_skip          :: "('x, 'k, 'v, 'dl, 'dg) man_transfer" ("skip♯")
  dgs_assign        :: "var_id ⇒ exp ⇒ ('x, 'k, 'v, 'dl, 'dg) man_transfer" ("assign♯")
  dgs_special       :: "exp gspecial_call ⇒ var_id ⇒ ('x, 'k, 'v, 'dl, 'dg) man_transfer"
    ("special♯")
  dgs_branch        :: "exp ⇒ bool ⇒ ('x, 'k, 'v, 'dl, 'dg) man_transfer" ("branch♯")
  dgs_body          :: "pname ⇒ ('x, 'k, 'v, 'dl, 'dg) man_transfer" ("body♯")
  dgs_return        :: "exp option ⇒ pname ⇒ ('x, 'k, 'v, 'dl, 'dg) man_transfer" ("return♯")
  dgs_enter         :: "call_info ⇒ ('x, 'k, 'v, 'dl, 'dg) man_enter_transfer"
    ("enter♯")
  dgs_event         :: "analysis_event ⇒ ('x, 'k, 'v, 'dl, 'dg) man_transfer" ("event♯")
  dgs_combine_env   :: "call_info ⇒ ('x, 'k, 'v, 'dl, 'dg) man_combine_transfer"
    ("combine'_env♯")
  dgs_combine_assign :: "call_info ⇒ ('x, 'k, 'v, 'dl, 'dg) man_combine_transfer"
    ("combine'_assign♯")
  dgs_query         :: "('x, 'k, 'v, 'dl, 'dg) man_query"
```

Figure 6.2: The declaration of `dg_spec`, lifted from the theory. Every field except the query handler carries, marked ♯, the name of the corresponding method of Goblint's `Spec`. The edge each transfer handles is given in the text. A `call_info` names the callee, its formals, the arguments, and the destination of a call.

The goal is a fixed set of laws from which the obligations of Chapter 4 follow for every record that satisfies them. Most of them form the *analysis contract* `analysis_contract`. Since a transfer may read analysis globals, a local state alone no longer fixes which stores it describes. The contract therefore uses a concretization $\gamma_{DG}(d, \rho)$ of a local state d under an *environment* ρ, which maps every analysis global to its value. It states three laws. First, γ_{DG} is monotone in both arguments (`analysis_contract.gammaDG_mono`), which turns the solver's inequalities into inclusions as γ does for a domain. Second, the step law (`analysis_contract.step_sound`), the counterpart of INTRA, reads

$$s \in \gamma_{DG}(d, \rho) \wedge s' \in \texttt{edge_step a s} \Longrightarrow s' \in \gamma_{DG}(d', \rho \sqcup p),$$

where the manager hands the transfer the local state d at the edge's source and reads analysis globals from ρ, and the transfer returns d' and publishes p ($\bot$ wherever it publishes nothing). A transfer that publishes nothing thus meets the condition of Section 6.1 with ρ as a fixed parameter. Third, the combine law (`analysis_contract.combine_sound`) is the counterpart of RETURN in the same shape. It concerns the two combine stages run one after the other. Given stores of the resume state q and of the exit state r, the combined state

must describe the store that takes the caller's locals from the first and the program globals and result from the second. A transfer is a program that passes its result on to the rest of the equation, so it could in principle do so twice or not at all, and what it publishes would then be ill defined. A well-formedness premise (`dg_spec_wf`) requires every operation to pass its result on exactly once. Beyond these laws, the contract asks the domains only for a join semilattice with a least element, and never for a map from variables to abstract values (Section 5.3.2).

The contract supplies the context-independent core of INTRA and RETURN. CALL requires that the callee's entry covers the entered store *in every context the policy admits for the call*. Entry coverage provides the covering pair, but the context policy of Chapter 4 decides in which contexts it is analyzed, so the contract alone does not discharge CALL. TOTAL, that every covered call reaches some context, follows from the policy, and for entry-state contexts also from entry coverage. INIT requires the analysis's initial state to cover the initial stores, which the analysis discharges when it is registered (`dg_analysis.place_init`).

6.4. Program globals: flow-sensitive or flow-insensitive thy

The contract leaves open where program globals are kept. By default they are part of the local state, as in Goblint's default base analysis for single-threaded programs (`base.ml`, [32, §5.6.1]). The other placement keeps each program global at an analysis global of its own, written by side effects and read where it is used. The analysis is then *mixed flow-sensitive*, flow-sensitive in the locals and flow-insensitive in the globals.

VIMP

```
global i32 Gx;
fun set() { Gx = 1; }
fun i32 get() { return Gx; }
fun main() { i32 x = 1; set(); i32 y = get(); }
```

The flow-insensitive value of `Gx` must cover the initial `Gx = 0` (VIMP starts every global at 0, Section 3.3) as well as the 1 written by `set`. Sign therefore claims only `≥0` for `y` after `y = get()` (claim `mixed-flow-sign-shared`), although every run ends with `y = 1`. Flow-sensitively, Sign carries `+` from `set`'s exit into `get`'s entry and derives `+` (claim `mixed-flow-sign`). In sequential VIMP the flow-insensitive placement can thus cost precision, and a guard on such a global refines nothing, as in Goblint's non-relational thread-modular analyses [32, §5.6.2]. The placement suits multithreaded programs, where the simplest thread-modular analysis keeps shared globals flow-insensitively [2, §1, Ex. 4], [32, §5.3.1], which a threaded extension of VIMP could build on.

Like Goblint and Apinis et al. [2, §3], [44, §5], the analyzer keeps one unknown per program global, so a write that changes `g` destabilizes only the right-hand sides that read `g` and their dependents. The placement is a wrapper that the framework puts around an analysis that keeps no analysis globals of its own, in the shipped analyzer the combination of all selected analyses (Chapter 7). The wrapper adds the analysis globals itself. At each edge it reads the program globals the edge mentions from their analysis globals and publishes those the edge can write. The result meets the analysis contract whenever the inner analysis is sound and the domain satisfies monotonicity laws and a frame law for the globals a transfer does not assign (`keyed_split_contract`). The combined analysis satisfies them (`mcp_cs_keyed`), so `run_voblint_source_sound` covers this placement.

The interface is modular. An analysis proves facts about its own operations only, while context handling, equation construction, and solving are implemented and proved once by the framework.

7. How Analyses Cooperate

Chapter 6 fixed what one analysis supplies, a record of operations whose soundness is proved operation by operation. Its transfers, however, see only the analysis's own state, while a check may need a fact that one analysis knows and another must use. In the program below both guards hold at the assignment, so $x = y$ and `z` is 1.

```
fun main() {
  i32 x = __voblint_nondet_int();
  i32 y = __voblint_nondet_int();
  if (x <= y) {
    if (y <= x) {
      i32 z = (x == y);
      __voblint_check(z == 1);
    }
  }
}
```

VIMP ↗

Interval keeps one range per variable. After the guards, `x` and `y` are still unconstrained within their kind, so `x == y` evaluates to $[0, 1]$ and the check stays `UNKNOWN` with `interval: z=[0,1]`. The order analysis is relational. Its state is a set of pairs (x, y), each meaning that the value of x is at most the value of y. After the guards it holds (x, y) and (y, x), so it knows that $x = y$, but it tracks no value of `z` and reports `UNKNOWN` too. Their combination decides the check if Interval's assignment can use what the order analysis knows at that point (Figure 7.1).

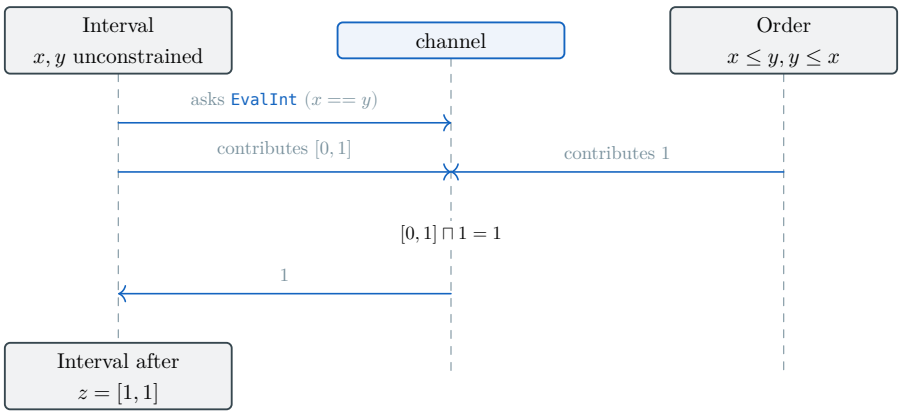


Figure 7.1: One step of Interval and the order analysis at `z = (x == y)` in the program above (schematic). Interval's assignment asks `EvalInt` $(x == y)$, which values $x == y$ may take. The channel asks the handler of every analysis and gives Interval the meet of their contributions. Each analysis updates only its own part of the state. The final states match the analyzer run `PROVED` with `interval: z=[1,1]; order: {x≤y ∧ y≤x}`.

Goblint solves this with its *master control program* (MCP). The activated analyses run side by side, each on its own part of one combined state, and a transfer may ask all of them a question through the manager's `ask` function. No machine-checked argument shows that these exchanges are sound. If Interval's assignment were proved sound from the order analysis's invariant, the two proofs would be coupled, and every new analysis could reopen old proofs. Voblint instead gives each analysis one proof obligation that does not mention its partners, and analyses meeting it combine into one analysis that meets the analysis contract of Section 6.3.

7.1. Questions and answers thy

The analyses must agree on what a question means and on what an answer claims. A *query* asks for a fact about the stores a state describes. This analysis query differs from the query of a local solver, the unknown it starts from (Section 2.6). Voblint has one kind, named after Goblint's. The query `EvalInt` e asks which integers the expression e may evaluate to. Its answer is a record (`int_dom`) of a sign, an interval, a parity and a congruence, lifted by a bottom and by a top element that claims nothing (`query_lift`). An answer a *holds* at a store s if the value of e in s lies in $\gamma(a)$ (`eval_holds`). A comparison evaluates to 0 or 1, so the exact answer 1 to `EvalInt` $(x == y)$ says that $x = y$. A non-relational domain converts between its own values and this record with the operations `answer_ik` and `of_answer_ik` of its interface (Figure 5.8). Answers are restricted to this record. A domain whose facts it cannot express, such as the exclusion $x \neq 5$ for an x in a wide interval, contributes only what the record can say. A new kind of fact would need a new query or a wider answer type, together with the two laws below.

Two laws (`query_algebra`) let answers from several analyses work together. The top answer $\top$ holds at every store, so an analysis that cannot answer a question answers $\top$ and claims nothing. If two answers hold at a store, so does their meet, so a question asked of all analyses can be answered by the meet of their answers. In the example, Interval answers $[0, 1]$ after both guards and the order analysis answers 1, and their meet $[0, 1] \sqcap 1 = 1$ says that $x = y$. Goblint's `MCP.query'` combines answers the same way, as formalized by Apinis [73, §5.5].

A transfer may ask several questions. The framework therefore hands it all answers at once, as a *channel* ch (`channel`), a function from queries to answers. A channel *holds* at a store s if every answer it gives holds at s (`channel_holds`). By the two laws, the channel answering $\top$ everywhere holds at every store, and the pointwise meet of two channels that hold at s holds at s (`query_algebra.channel_holds_inf`).

7.2. Proving an analysis against any channel thy

Interval's assignment in Figure 7.1 evaluates its right-hand side with `aeval_ask`, the abstract evaluation every non-relational analysis uses. For an expression e other than a literal, it asks `EvalInt` e of all analyses, itself included, and meets the answer with its own value of e. Section 7.3 shows why a question to itself does not loop. At $z := (x == y)$ Interval receives 1, meets it with its own $[0, 1]$, and sets z to $[1, 1]$, which is right only if the answer is true.

Voblint makes the channel an argument of the transfer and the truth of its answers a premise of the proof. The transfer `ls_step` $\ell\ ch\ a\ x$ of an analysis ℓ receives a channel ch besides the action a and the value x before the edge. Soundness is the condition of Section 6.1 with one extra premise, that ch holds at the store. For every channel,

$$s \in \gamma(x) \wedge \texttt{channel_holds}\ ch\ s \wedge s' \in \texttt{edge_step}\ a\ s \Longrightarrow s' \in \gamma(\texttt{ls_step}\ \ell\ ch\ a\ x),$$

which `ls_step_sound_iff` splits into one law per kind of edge. In the example, the premise excludes the stores where $x \neq y$, at which the answer 1 is false. At the others the value of e lies in both the answer and the analysis's own value, hence in their meet. This argument covers every non-relational analysis and every partner whose answers hold (`aeval_ask_sound`). The premise that the channel holds is the one left open by the evaluation and refinement laws of Section 5.4 and Section 5.6.

An analysis without analysis globals is written as a *local specification* (`local_spec`). It has the operations of Figure 6.2, and each operation takes a channel. The first combine stage takes two, one that holds at the caller's store and one that holds at the callee's exit store. Its handler `ls_query` answers queries about a state.

Soundness of a local specification (`sound_local_spec`) asks for a monotone concretization and one law per operation (`sound_local_spec_iff`). The edge and combine laws are the counterparts of the step and combine laws of the analysis contract (Section 6.3), and the enter law matches entry coverage, each assuming that its channels hold. The new handler law requires the handler's answers about a state x to hold at every store of $\gamma(x)$ at which the channel it receives holds (`sound_query`). It is the only law about an analysis that its partners rely on.

Astrée and Verasco use the same idea. In Astrée an abstract transfer only has to cover concrete arguments and results that satisfy the constraints of its channels [20, §§5.3, 6]. Verasco, whose channels are inspired by Astrée's and whose term we adopt, proves each transfer function under the hypothesis that the channels it receives are correct [34, §7]. Voblint uses the same form for Goblint's queries.

7.3. Many analyses over one state thy

The analyses cannot be solved one after another, since each may need the other's facts at the same program point and context. Interval needs the order analysis's facts at `z = (x == y)`, and to record $x \leq y$ where y is assigned a constant, the order analysis needs a numeric bound on x. The framework therefore runs all selected analyses as a single analysis whose abstract state is a tuple with one field per analysis. At `z = (x == y)` this state pairs Interval's state, in which x and y are unconstrained, with the order analysis's state $\{x \leq y, y \leq x\}$. The tuple is ordered, joined, and widened field by field (`analysis_product`), so each field evolves as it would alone, except that its transfers may ask the other fields. If one selected field shows that a node is unreachable, the whole tuple does (`mcp_norm`).

The combined state `mcp_st` has one field per analysis the analyzer ships, and a run uses the fields of the analyses the user selects (Figure 7.2). The tuple describes a store if every selected field does, $\gamma(x) = \bigcap_{\gamma_i \in \Gamma} \gamma_i(x)$ (`mcp_gamma`), where Γ lists the concretizations of the selected fields and each γ_i reads its own field of x.

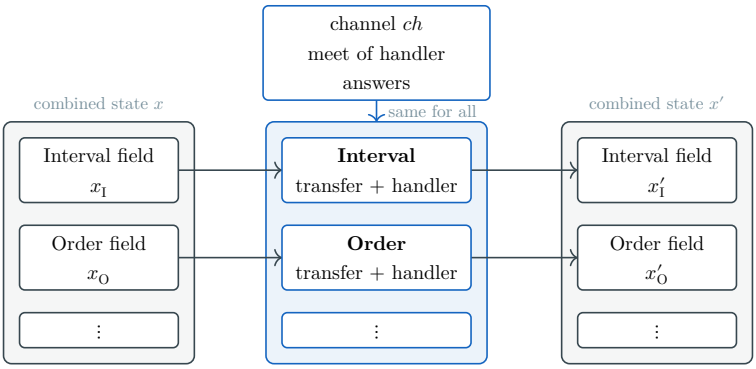


Figure 7.2: One step of the combination (schematic). Each component reads its own field of the combined state x and writes its own field of x', and the components run in turn. All components receive the same channel ch (blue), which the framework builds from x as the meet of the answers of all handlers.

We call a local specification that runs on one field a *component*. Each component reads and writes only its own field, through a *lens* (`lens_of`), as Goblint's MCP hands each analysis its own part of the combined state. A component therefore leaves the concretizations of all other fields unchanged (`field_frame`). `mcp_combine` runs the components in turn, gives every one the same channel, and meets the answers of their handlers. If the list of components is nonempty, the components are independent in this sense, and each is a sound local specification, then the combination is a sound local specification for the intersection of their concretizations (`mcp_combine_sound`), and `mcp_comp_sound` applies this to every nonempty selection of distinct analyses. Distinctness matters because two copies of one analysis would write the same field, so neither would leave the other's concretization unchanged. The analyzer meets both conditions, because `run_voblint` answers an empty selection or a repeated analysis with `Invalid_Activation` before it compiles the program (Section 10.1).

The framework therefore has to build, for every state x, a channel that holds at every store of $\gamma(x)$. It asks the combined handler, whose components may ask in turn, and answers $\top$ to any query that is asked again while it is being answered, as Goblint's `MCP.query'` does. It also stops at a fixed nesting depth, where the logic reads the answer as $\top$ and the generated code aborts. Every answer of this channel holds (`ls_channel_sound`). The argument looks circular, since the handler law assumes a channel that holds and the channel is built from the handlers. Induction on the remaining depth makes it well founded. An answer cut off at a cycle or at the depth bound is $\top$ and holds trivially. Any other answer comes from a handler whose own questions are answered one level deeper, where they hold by induction, so the handler law makes the answer hold. Passing this channel to the operations turns a sound local specification into a record that meets the analysis contract (`dg_spec_of_contract`). The combination of any nonempty selection of distinct analyses therefore meets the analysis contract, like a single analysis.

Checks are decided from the same kind of answers. At a check with condition b, every active analysis answers `EvalInt` about the truth value of b from its own part of the solved state, and the analyzer meets these answers (`mcp_answer`). A non-relational analysis answers with the three-valued evaluation of Section 5.5. An exact 1 proves the check, and

an exact 0 refutes it (`mcp_classify`). Several analyses together can thus decide a check that each alone leaves unknown, as the run `PROVED` of the opening example shows.

7.4. What a new analysis has to prove

A new analysis joins the combined state by proving the laws of its local specification, which mention no other analysis. It also needs a field in the combined state `mcp_st` and the lens onto it, which are ordinary theory text checked by Isabelle like the rest. A non-relational domain instantiates the class `kind_int_domain` (Section 5.7), and one generic proof derives the laws for every instance (`nonrel_spec_sound`). Any other analysis starts from conservative defaults, a handler that answers $\top$ and operations that keep the state at skips, guards, procedure bodies and events, which are sound for every concretization. The default first combine stage keeps the caller's state. The analysis then supplies the assignment, the special calls, the return, enter, and the second combine stage. It proves each operation it supplies, and the combine law for its second stage after the default first one, since that law concerns both stages together (`sound_conservative_local_spec`). The order analysis also replaces the default at guards, answers comparisons between variables it has ordered, asks at assignments, and returns with no facts (`ord_spec_sound`).

Components cannot read or publish analysis globals (Section 6.2). The flow-insensitive placement of Section 6.4 wraps the whole combination and adds analysis globals for the program globals itself. Goblint's MCP, in contrast, lets each analysis keep analysis globals of its own, tagged with its index. No theorem guarantees that the combination is more precise than its components, and programs such as the opening one only show concrete cases where it is.

Since each analysis is proved sound against every channel that holds, adding an analysis needs one new soundness proof and leaves the soundness proofs of the existing analyses unchanged. The laws so far mention no contexts, equations, or solver. Chapter 8 builds an equation system over nodes and contexts from the operations of the combined analysis and discharges the obligations of Chapter 4 for it.

PART IV

From Analyses to Verdicts

8. Equations, Contexts, and Routing

Chapter 4 reduced soundness to a *claim*, which assigns to every node v and context c a set of stores `cover v c`, and stated five local obligations under which the claim contains every store that an execution brings to v in context c. To compute such a claim, Voblint turns the procedure-aware graph of Chapter 3 into a system of equations. The goal is that every post-solution σ on a set of solved unknowns that executions cannot leave satisfies

$$\mathcal{A}_{\kappa,\texttt{adm},c_0,\texttt{g},\texttt{S}}\ \texttt{v}\ \texttt{c} \subseteq \gamma_M(\sigma(v,c))$$

at every solved node v and context c. The left side is the activation collecting semantics of Section 4.3. The right side reads the solved state through the concretization of Section 6.3, $\gamma_M(\sigma(v,c)) = \gamma_{DG}(d,\rho)$, where d is the local state that σ assigns to (v,c) and the environment ρ holds the values that σ assigns to the analysis globals. Calls make this hard, because the callee's context may depend on abstract values that become known only while the equations are solved. The running example is the program of Figure 3.1.

8.1. Unknowns and right-hand sides thy

The running example calls `bump` first with 5 and then with 4. If both activations shared one abstract state at the entry of `bump`, that state would have to contain both values, the return would merge the two results, and the analysis could no longer show that the first result is exactly 6. Contexts avoid this merge by giving the two activations separate copies of `bump`.

Voblint therefore indexes the unknowns of the equations by contexts. For every node v and context c there is a *local* unknown (v,c), the unknown $[v,c]$ of Chapter 4, whose value $\sigma(v,c)$ is the abstract state for the activations of context c at v. These are the indices of the claim, and Goblint's local unknowns have the same shape [73, §5.2]. The equations also have *global* unknowns without a program point (Section 2.7), which hold the analysis globals of Section 6.2 and the entry states that calls publish to their callees.

The equation of (v,c) collects every abstract state that can reach v in context c. These states come from four sources: the initial state at the program entry, the transfer along each local edge into v, the result of each call that returns to v, and, at a procedure entry, the entry states (the abstract states in which the callee starts, Section 6.1) that calls publish to it. A local edge keeps the activation's context (Section 4.3), so its transfer reads the predecessor in the same context c. The generated right-hand side (`routed_node_rhs`) is the join of the four sources, evaluated under a valuation τ (the statement's name for it, unrelated to the strategy trees of Section 2.7), an arbitrary assignment of values to the unknowns:

```
(* src/Abstract_Interpreter/Framework/Constraints/DG_Indexed_Generator.thy *)                Isabelle
lemma routed_node_rhs_parts:
  assumes wf: "⋀w. ∀p ∈ set (routed_contribution_programs pred_sel site_sel route it cmb
                       extra g c w). sp_wf p"
  shows
  "eq (routed_node_rhs pred_sel site_sel buffer_key_at route it cmb extra g bot0 d₀ g₀)
      (v, c) τ =
   DG (rhs_init g bot0 d₀ v
       ⊔ (⨆(u, a)←pred_sel g v c. rhs_edge it τ c u a)
       ⊔ (⨆(u, ca)←site_sel g v. rhs_call cmb route τ c u ca v)
       ⊔ rhs_seed extra route τ c v) bot"
```

Here $\bigsqcup_{x \leftarrow xs} f(x)$ joins $f(x)$ over the list xs, the empty join is $\bot$, and the lists range over the local edges into v and the calls that return to v. The four parts inside `DG` are the four sources in the order above. The remaining parameters select the edges and calls and fix how the right-hand side is encoded for the solver, which Section 9.1 explains, and `DG` pairs the local state with an unused global half. A post-solution lies above this join, and γ_M is monotone, so $\gamma_{M(\sigma(v,c))}$ contains what each source describes.

At a call, a *routing* function $\text{context}^\sharp(u, c, e)$ chooses the callee's context from the call node u, the caller context c, and the entry state e. Under entry-state contexts the choice depends on e, which is known only while the equations are solved, so the callee's entry cannot list in its equation the calls that contribute to one of its contexts. Each call instead publishes its entry state as a side effect (Section 2.6). Apinis et al. [44, §6] and Goblint publish it directly to the callee's entry. In Goblint this entry holds a value of the local domain but has no right-hand side of its own (`constraints.ml`). In Voblint's equations the entry is a local unknown with a right-hand side, and the verified solver accepts side effects only to global unknowns, so Voblint publishes to a *seed* Activation_Seed $p\ c'$, the global unknown (`global_unknown`) that carries the entry state of callee p in context c' (Section 9.1.2). Every call routed to (p, c') publishes to this seed, and the equation of the entry (FunctionEntry p, c') reads it.

In Figure 8.1, the equation of the continuation $(pp3, c_0)$ reads the caller's value at $(pp2, c_0)$ and applies enter (Section 6.1), which yields an *entry pair* of a resume state q_1 and an entry state e_1 that binds n to $[5, 5]$. The entry state selects the callee context $c_1 = [[5, 5]]$. The equation publishes e_1 to the seed of c_1, reads the result r_1 of `bump` there, and joins

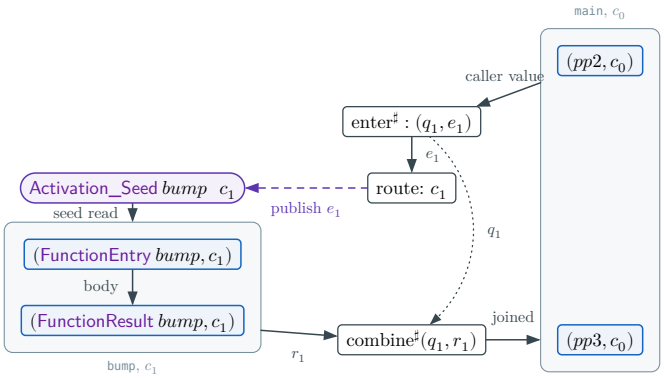


Figure 8.1: The first call of the running example under entry-state contexts. The right-hand side of $(pp3, c_0)$ routes the entry state e_1 to c_1, publishes it to the seed of `bump` in c_1 (purple, dashed), reads the result there and joins the combined state into its own value. Blue boxes are local unknowns, the purple box is a global unknown, white boxes are steps of the right-hand side, and gray frames group one procedure in one context.

combine$^\sharp(q_1, r_1)$ into its own value. The second call does the same with $c_2 = [[4, 4]]$, so the two calls use separate copies of `bump` and the first result stays exactly 6. The generic framework allows several call edges at one call site and several entry pairs per call, and it routes each pair separately. A compiled VIMP program, however, has one call edge per call site (`compile_prog_calls_source_unique`), and the shipped analyses return one entry pair per call.

A caller's equation reads no unknown of the callee other than its result, so a recursive call needs no special treatment. Its cycle of dependencies is handled like that of a loop, with a widening point where a read closes it (Section 2.7). The explainer's solver replay animates this protocol.

If the bottom test of the analysis classifies the entry state as empty, no store enters the callee, so the call publishes nothing and does not read the callee's result, which would make the solver evaluate the callee's body for no store. Goblint's normal-call transfer skips bottom entry states in the same way. The test is sound because a state it classifies as empty must have an empty concretization (`is_bot_sound`).

8.2. Routing and context policies thy

The analyzer chooses a callee context from abstract information. The concrete semantics of Chapter 4 assigns each activation its contexts through the context policy *adm* of Section 4.3, which may depend on concrete stores that the analyzer never sees. For entry-state contexts, *adm* must follow the analyzer's choice, so it is defined from the solved valuation σ, as Section 4.2.1 anticipated. The argument is nevertheless not circular. The valid traces and their node collecting semantics are fixed before any analysis runs, and only the labelling of traces by contexts depends on σ. The source-level result concerns only the union over all contexts, and this union equals the node collecting semantics under every policy whose claim meets the five obligations (the theorem `node_collect_eq_Union_activation_collect` of Section 4.4). The proof therefore fixes σ first, defines *adm* from it, and shows that σ covers the induced activation collecting semantics.

The equations are sound only if the two context choices agree. Suppose a concrete call from a caller store s, which the caller's solved value covers, enters the callee in store s', and *adm* admits c' for it. Then enter, applied to the caller's solved value, must return an entry pair (q, e) that the equations route to c', with q covering s and e covering s' (Figure 8.2):

$$\textit{adm} \text{ admits } c' \text{ for } (s, s') \implies \text{some entry pair } (q, e) \text{ is routed to } c' \text{ with } s \in \gamma(q) \text{ and } s' \in \gamma(e).$$

Otherwise the unknowns of c' would have to cover a store that no equation put there.

We call this condition *routing adequacy* (`routed_context.routing_adequate`). It also requires the callee's entry unknown in c' to be in the solved set, because σ bounds the stores of an unknown only there (Section 8.3). CALL and RETURN need routing adequacy, because the call publishes e and its continuation reads the callee's result in the same c'. A second condition, *totality* (`routed_context.routing_total`), requires *adm* to admit at least one context for every concrete call from a covered caller store. Otherwise a concrete call could be assigned no context and would disappear from the activation collecting semantics, which the obligation TOTAL of Section 4.4 rules out. Both conditions are premises of the soundness theorem and are proved once for each context policy.

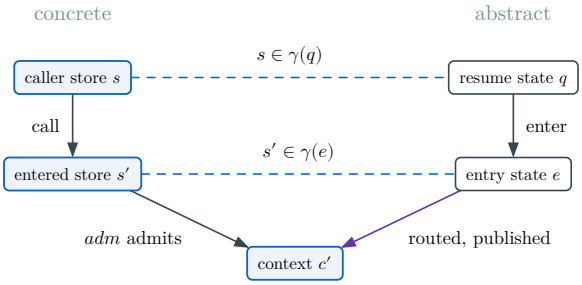


Figure 8.2: Routing adequacy. If the concrete semantics assigns a call to c' (left), the equations must route a covering entry pair to the same c' (right).

policy	adm **admits** c' **iff**	**routed to**	**why they agree**
call strings	$c' = \text{take}_k(u \cdot c)$	$\text{take}_k(u \cdot c)$	the same term
entry states	$c' = e\|_{\text{formals}}$, (q, e) covers (s, s')	$e\|_{\text{formals}}$	adm reads the routing

Table 8.1: The contexts adm admits for a call at site u from caller context c, with caller store s and entered store s', and where the equations route the entry state e. $u \cdot c$ prepends the site, take_k keeps the first k entries, $e|_{\text{formals}}$ lists the formals' values in e, and an entry pair (q, e) of the caller's solved value covers (s, s') when $s \in \gamma(q)$ and $s' \in \gamma(e)$. Call strings are a *store-independent* policy, which computes the context from the call site and the caller context alone, and $k = 0$ is the context-insensitive analysis.

For call strings, adm admits exactly the context that the routing function computes for the call, which does not depend on e (`context_policy_of_fun`). Totality is then immediate (`call_context_total_on_of_fun`), and routing adequacy reduces to the entry coverage of Section 6.1 and a routed entry unknown in the solved set.

Figure 8.3 shows how the policies differ on a call chain. A call string of length one separates the two calls of `wrap` but merges them again in `scale`, where both arrive from the same call site. Length two and entry-state routing keep them apart.

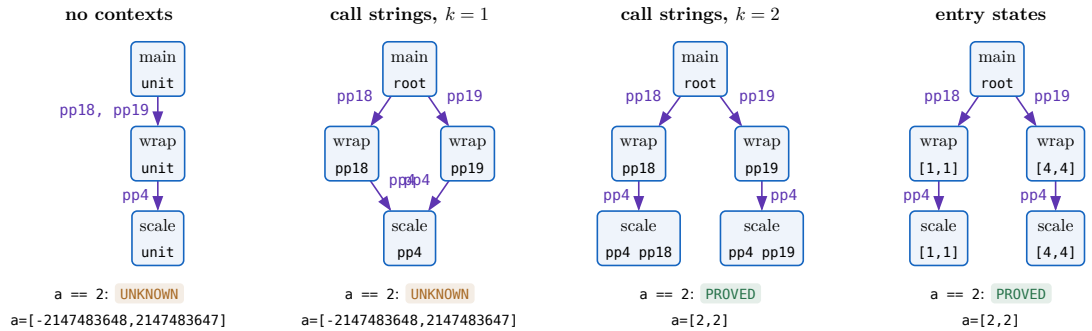


Figure 8.3: Procedure copies under four context policies, on a nested-call program: `scale(v)` returns `v * 2`, `wrap(w)` returns `scale(w)` from `pp4`, and `main` calls `a = wrap(1)` at `pp18` and `b = wrap(4)` at `pp19`, then checks `a == 2`. Boxes are copies labeled with their contexts (`root` is that of `main`, and `unit` is how the analyzer prints the single context of $k = 0$, the empty call string), and arrows are calls labeled with their sites (Interval, claims `ctx-demo-*`). Where both calls reach one copy of `scale`, they publish to one seed, which the update rule widens to $v \geq 1$ (Section 9.1.2), so `v * 2` may overflow and `a` can be any `i32` value.

An entry-state context is the list $e|_{\text{formals}}$ of the abstract values that the entry state e gives the callee's formals. For `bump(n)` and an e that binds n to $[4,5]$, it is $[[4,5]]$. If the caller's solved state has $x \in [4,5]$ at `bump(x)`, the equations publish $n \mapsto [4,5]$ to the context $[[4,5]]$, and an execution with $x = 4$ must then be admitted in $[[4,5]]$, where its covering entry state was published, and not in $[[4,4]]$. The entry-state policy (`routed_entry_context_rel`) therefore admits c' for a call from s with entered store s' only when $c' = e|_{\text{formals}}$ for an entry pair (q, e) with $s \in \gamma(q)$ and $s' \in \gamma(e)$. The equations publish that same e to c', which gives routing adequacy, and entry coverage (Section 6.1) gives every covered call such a pair, which gives totality. Because the policy reads the entry pairs of the solved values, it is defined from σ, in the order described at the start of this section.

8.3. Soundness of the generated system thy

By the soundness theorem of Section 4.4 (`activation_collect_sound`), every claim that meets the five obligations contains the activation collecting semantics. It therefore suffices to discharge the obligations for the claim that the solved equations define. Let V be the solved set, the local unknowns at which σ satisfies the equations as inequalities. The claim is

$$\texttt{cover v c} = \gamma_M(\sigma(v,c)) \text{ if } (v,c) \in V, \quad \texttt{cover v c} = \emptyset \text{ otherwise.}$$

Outside V the solver bounds nothing, so σ may describe too few stores there, and only the empty claim needs no bound. In exchange, executions must not leave V. We call V *closed under execution steps* if a local edge leads from a node of V whose state describes some store to a node of V, and a call site in V has its continuation in V. Routing adequacy adds the callee's entry in the routed context. Table 8.2 lists which part of the right-hand side discharges each obligation.

obligation	what the generated equations provide
INIT	initial part, which lies above d_0
INTRA	edge part and the step law
CALL	call part publishing to the seed, seed part, routing adequacy
RETURN	call part reading the result, combine law, routing adequacy
TOTAL	totality

Table 8.2: How the parts of the generated right-hand side of Section 8.1 meet the five obligations of Chapter 4, for an arbitrary program.

The argument holds for every domain and context policy. Its locale `routed_context` assumes a post-solution σ on a solved set closed under execution steps, the analysis contract of Chapter 6 (`analysis_contract`), a sound bottom test, a sound callee resolution, and adequate and total routing. Callee resolution lists, from the caller's state, the procedures a call may enter, and it is sound if it lists the actual callee for every covered caller store. The analyzer reads the callee from the call edge, since every VIMP call names its callee. The theorem adds that the program entry is solved in the initial context and that the initial state d_0 describes the initial stores S_0 under an environment ρ_0 of analysis-global values below the solved ones, $S_0 \subseteq \gamma_{DG}(d_0, \rho_0)$ (ρ_0 is $\texttt{e}_0$ in the statement).

Theorem 8.1 (Routed collecting soundness, `activation_collect_dg_sound`)**.**

```
(* src/Abstract_Interpreter/Framework/Context/Routed_Context.thy *)
theorem activation_collect_dg_sound:
  fixes S0 :: "store set" and c₀ :: 'c
  assumes entry_cov: "(cfg_entry g, c₀) ∈ vars"
    and init_sound: "S0 ⊆ γ_DG d₀ e₀"
    and init_env_le: "e₀ ≤ genv global_of sigma"
  shows "A_κ,adm,c₀,g,S0 v c ⊆ cover v c"
```

Each row of Table 8.2 follows from the bound that the post-solution gives one part of the right-hand side, together with a law of the analysis contract and the monotonicity of γ_{DG}. The step that needs care is RETURN, where a trace resumes its caller with the final store of a callee activation that carries some context c', so that store lies in the claim at the callee's result in c'. Routing adequacy, applied to the same call and caller store s, gives an entry pair (q, e) that the equations route to this same c' with $s \in \gamma(q)$, so the call part of the continuation reads exactly that result and the combine law places the resumed store in the claim. `fun_route_activation_collect_sound` and `entry_state_activation_collect_sound` instantiate the theorem for store-independent and entry-state routing, and Chapter 9 computes such a post-solution.

9. Solving the Equations

Chapter 8 showed that every post-solution of the generated equations covers the activation collecting semantics, provided the solved set contains the program entry and is closed under execution steps (`activation_collect_dg_sound`). The analyzer computes such a post-solution with the verified top-down solver of Tilscher et al. [4], which is proved partially correct. Two steps connect the two results. The solver needs each right-hand side as a strategy tree (Section 2.7), a small program that reads one unknown at a time and makes every read and every publication explicit, so the equations must be brought into this form. The solver's correctness theorem must then be connected to the premises of the soundness theorem. Both steps assume that the solver returns, which no theorem establishes (Section 9.6).

9.1. Adapting the equations to the solver

A direct encoding of the equations, along the lines of Apinis et al. and Goblint, needs four things that the interface of the vendored solver does not offer (Table 9.1). Chapter 8 stated the equations with the first three adapters already in place. Buffering is left out there because it changes only when publications are issued, not the equations (Section 9.1.4).

the direct encoding needs	the solver offers	adapter
read a result chosen from the caller's value	one read at a time	strategy tree (Section 9.1.1)
write the callee's entry unknown	side effects to globals only	seeds (Section 9.1.2)
different local and global values	one value type	product (Section 9.1.3)
publish twice to one target	update rule at every side effect	buffering (Section 9.1.4)

Table 9.1: Where the direct encoding of the equations and the interface of the vendored solver differ, and the adapter that bridges each.

9.1.1. Dynamic reads thy

A local edge always reads its predecessor, whereas a call chooses what to read from values it has already read. The continuation $(pp3, c_0)$ of the running example reads the caller $(pp2, c_0)$, computes from its value the entry state $n = [5, 5]$ and the context c_1, and only then knows that it must read (FunctionResult $bump, c_1$). A strategy tree allows this, because

each continuation receives the value just read and may choose the next read from it. The contribution of a call at u in context c (`rhs_call`) is the value of the tree

$$t_{\text{call}} = \mathsf{QueryL}\ \big((u, c), \lambda d.\ \mathsf{Side}\ \big(\mathsf{Activation_Seed}\ p\ c', e,\ \mathsf{QueryL}\ \big((\mathsf{FunctionResult}\ p, c'), \lambda r.\ \mathsf{Answer}\ \big(\text{combine}^\sharp(q, r)\big)\big)\big)\big),$$

where d is the caller's value, $(q, e) = \text{enter}^\sharp(d)$, $c' = \text{context}^\sharp(u, c, e)$, and r is the callee's result. The Side step publishes the entry state to the callee's seed. The generated tree (`routed_call_program`) also handles entry states that the bottom test classifies as empty and several callees or entry pairs.

9.1.2. Local entries

A call has to pass its entry state e to the callee's entry $(\mathsf{FunctionEntry}\ p, c')$, which in Voblint's equations is a local unknown with a right-hand side. Apinis et al. publish to the entry directly [44, §6], and Goblint does the same with its local side effect `sidel` (`constraints.ml`), whose target holds a value of the local domain but has no right-hand side (`constraints.ml`). The vendored solver's Side accepts only global unknowns, and local targets would require changing the solver and redoing its correctness proof.

Voblint therefore routes the entry state through the seed $\mathsf{Activation_Seed}\ p\ c'$ of Section 8.1, a global unknown. The call publishes e to it with Side, and the equation of the entry reads it back with QueryG. Publishing alone does not make the solver evaluate the callee, since it solves only unknowns that some tree reads. The call, however, also reads the callee's result, and solving the result reaches the entry and its seed.

Because seeds are global unknowns, the solver merges the contributions of different calls with its *update rule* for globals, the operation that combines a new contribution with the value the unknown holds (Section 9.4 compares the five rules). Without contexts, both calls of the running example publish to the one seed of `bump`. The default rule, warrow, widens the seed when a contribution is not below its value, so the seed first holds $n = [5, 5]$ and then widens when $[4, 4]$ arrives. The analyzer then reports `UNKNOWN` for `a == 6` with `interval: a=[-2147483647,6]`. Widening moved the lower bound to the least `i32` value, and the call added one. The joining rule keeps `interval: a=[5,6]` (claim `pg-contexts-none-join`), which is the loss due to merging alone.

9.1.3. One value type thy

A local unknown holds an abstract state of the analysis's local domain, and an analysis global holds a value of its global domain (Section 6.2). The vendored solver has a single value type for all unknowns, so every unknown holds a pair (`dg_state`) of a local and a global half, uses one half and leaves the other at $\bot$ (Table 9.2). Order and join on pairs are componentwise, so the solver's requirements on the value type follow from those on the two halves. A seed is a global unknown that holds a local value, the entry state it passes to the callee.

solver unknown	role	value
(v, c), local	the state at v in context c	$(d, \bot)$
`Activation_Seed`, global	an entry state published by a call	$(e, \bot)$
`Analysis_Global` x, global	the analysis global x	$(\bot, g)$

Table 9.2: Which half of `dg_state` each kind of unknown uses. The global unknowns are named by `global_unknown`.

Transfers never see the pair or the seeds, because the manager of Section 6.2 gives each transfer the local state and access to the analysis globals. Goblint instead combines the two kinds of value in a lifted sum, in which each unknown holds a value of exactly one kind. Voblint uses the product because its componentwise lattice structure keeps the Isabelle proofs simple, at the cost of an unused $\bot$ half in every unknown.

9.1.4. Repeated publications thy

One right-hand side can publish to the same global unknown twice. In `if (b) { x = f(1); } else { x = f(2); }` both calls resume at the node after the conditional, so its right-hand side contains both calls, and under the context-insensitive policy both publish to the one seed of `f`. Declaratively, two writes to one target g mean one bound:

$$\mathsf{Side}\ (g, a); \mathsf{Side}\ (g, b) \quad \text{means} \quad \sigma(g) \geq a \sqcup b.$$

The vendored solver applies the update rule at every Side, first to a and then to $a \sqcup b$. The update rules record each origin's latest contribution, the *record per origin* of Section 2.7, and all rules except join compute the new value from these records (Section 9.4). In every re-evaluation the recorded contribution therefore first shrinks to a and then grows back to $a \sqcup b$. Under warrowing the shrinking step narrows, the growing step widens again, and the solver may not stabilize. The update rules of Stemmler et al. assume one joined contribution per target and evaluation [45, §3], and Goblint's per-origin narrowing rule for globals (`narrow-globs`) joins the side effects of an evaluation before updating (`td3UpdateRule.ml`).

Voblint therefore buffers the publications of a right-hand side and joins those to the same target until a flush point (`buffer_sides`). At a node where at most one call returns, it flushes before every local read, so that an entry state reaches the callee before the callee's result is demanded, in the order of Goblint's normal-call transfer (`constraints.ml`). At a node where several calls return, such as the node after the conditional above, flushing before each read would issue one Side for each call to the same seed. The right-hand side therefore flushes only when it answers, and each seed receives one Side per evaluation. The callee's result is then first read before its entry state arrives. This costs time but not soundness, because the late publication changes the seed, the solver destabilizes the callee's entry, and the caller is evaluated again with the updated result.

Buffering changes only when publications are issued. It leaves the value, the joined publications, and the reads of a right-hand side unchanged (`traverse_rhs_buffer_sides`, `sides_of_rhs_buffer_sides`, `dep_aux_buffer_sides`), so the buffered right-hand side that the analyzer runs answers the same join of four parts as in Section 8.1. Under widening update rules the timing can change the computed result, though not the equations it solves.

9.2. A demand-driven solve

The generator defines a right-hand side for every pair of a node and a context (`compiled_routed_eqs_for`). Under entry-state contexts over intervals the contexts range over an infinite type, and even the intervals of one 32-bit variable number about 2^{63}, so a solve cannot enumerate them. It starts instead from one query unknown, the unknown whose value is asked for (Section 2.6), which is unrelated to the analysis queries of Section 7.1. This query unknown is the result of `main` in the initial context c_0 (`root_query`, as in [44, §3]), and the solve evaluates an unknown only when a right-hand side it is solving reads it [4], [61]. The solve thus runs backward from the result and discovers contexts as it goes. At a call it computes the callee's context from the caller's value and demands the callee's result there. Table 9.3 and Figure 9.1 follow this solve on the running example, generated from the analyzer's solver trace (Interval, warrowing, claim `pg-contexts-trace`). The entry states $e_1 = \{n \mapsto [5,5]\}$ and $e_2 = \{n \mapsto [4,4]\}$ route to the contexts $c_1 = [[5,5]]$ and $c_2 = [[4,4]]$.

phase	evaluating	what happens
1	`exit_main` to `pp2`	the root query at `exit_main` queries back through `pp6`, `pp5`, `pp4`, `pp3` and `pp2` to `entry_main`
2	`entry_main`	QueryG reads Activation_Seed *main* $c_0 = \bot$, joins d_0
3	`pp3`	computes e_1 and routes to the new context c_1, publishes it to Activation_Seed *bump* c_1, queries (`exit_bump`, c_1), which queries back to the entry, which reads Activation_Seed *bump* $c_1 = e_1$: the result is `interval: bump::n=[5,5], main::a=⊤, main::b=⊤, return=[6,6]`
4	`pp4`	computes e_2 and routes to the new context c_2, publishes it to Activation_Seed *bump* c_2, queries (`exit_bump`, c_2), which queries back to the entry, which reads Activation_Seed *bump* $c_2 = e_2$: the result is `interval: bump::n=[4,4], main::a=⊤, main::b=⊤, return=[5,5]`. The answers then propagate up through `pp5` and `pp6` to `exit_main`

Table 9.3: The solve of the running example, condensed into phases. The second column names the unknown, or chain of unknowns, whose right-hand side the phase evaluates.

The checks at `pp4` and `pp5` read the solved values `interval: a=[6,6]` and `interval: b=[5,5]`, and the analyzer proves both. Without contexts, $c_1 = c_2$, both calls publish to one seed, and `bump` is solved once for both (Section 9.1.2). A terminating solve thus reaches finitely many unknowns, in the contexts it discovers.

9.3. What a terminating solve guarantees

A solve that returns gives a valuation σ and the set V of local unknowns it stabilized. The soundness theorem of Section 8.3 needs post-solution bounds on a set that contains the program entry and is closed under execution steps, which executions therefore never leave. The solver instead bounds σ on a set that contains the query unknown and is closed under the reads of right-hand sides.

9.3.1. The certificate thy

The verified solver states its guarantee as the certificate `part_post_solution` of Section 2.6, over the equation system T, the valuation σ and the solved set `vars`:

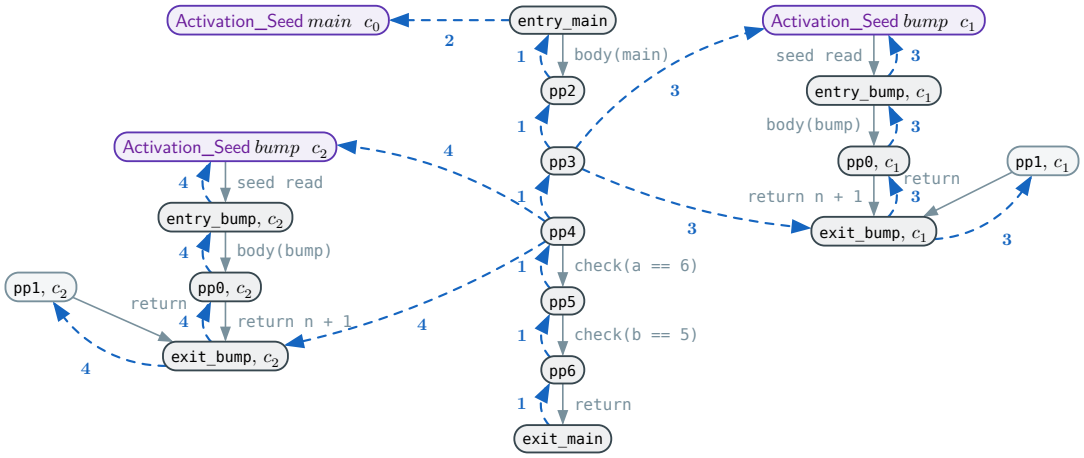


Figure 9.1: The solve of Table 9.3 on the unknowns. Gray arrows are graph edges, including a seed feeding its entry, and blue dashed arrows are the solver's queries and publications, labeled with their phases. The copies of `bump` for c_1 (right) and c_2 (left) are demanded as the solve discovers those contexts. Gray `pp1`, the unreachable end of the body of `bump`, stays $\bot$.

Isabelle

```
(* vendor/td-verification/Basics_side.thy *)
abbreviation part_post_solution ::
    "('x,'g,'d::bounded_semilattice_sup_bot) eqsT ⇒ 'x ⇒ ('x + 'g ⇒ 'd) ⇒ 'x set
    ⇒ bool" where
  "part_post_solution T x sigma vars ≡
    x ∈ vars ∧ (∀u ∈ vars.
    dep_L T sigma u ⊆ vars ∧
    eq T u sigma ≤ sigma (Inl u) ∧
    sides_of_rhs (T u) sigma ≤ sigma)"
```

T maps each unknown u to its strategy tree $T\,u$. Evaluating a tree under the final valuation σ determines the answer it returns ($\mathtt{eq}\,T\,u\,\sigma$), the local unknowns it reads ($\mathtt{dep_L}\,T\,\sigma\,u$), and the contributions it publishes, joined per target and $\bot$ where it publishes nothing (`sides_of_rhs`). All three are taken under the final valuation because the unknowns a tree reads depend on the values it has already read (Section 9.1.1). The certificate states four facts about the query unknown x and the solved set V:

$$x \in V \qquad \text{(C1)}$$

$$\forall u \in V.\, \mathtt{dep_L}\,T\,\sigma\,u \subseteq V \qquad \text{(C2)}$$

$$\forall u \in V.\, \mathtt{eq}\,T\,u\,\sigma \leq \sigma(u) \qquad \text{(C3)}$$

$$\forall u \in V.\, \mathtt{sides_of_rhs}\,(T\,u)\,\sigma \leq \sigma \qquad \text{(C4)}$$

(C1) puts the query unknown into the solved set, and (C2) closes the solved set under reading. (C3) is the post-solution bound for local unknowns, and (C4) bounds every published contribution at once, since it compares two valuations pointwise. Together, (C3) and (C4) are the pair condition of Section 2.6, and `part_post_solution_iff_rhs_pair` proves the certificate equivalent to (C1), (C2) and that pair bound at every $u \in V$. The returned σ may lie above the least solution, because the solver widens and narrows at its widening points (Section 2.7), and (C1) to (C4) are all the solver guarantees about σ.

In the running example the query unknown is (FunctionResult $main, c_0$), and V holds the 15 local unknowns the solve of Section 9.2 reached. The tree of $(pp3, c_0)$ reads

$(pp2, c_0)$ and $(\mathsf{FunctionResult}\ bump, c_1)$, so both lie in V by (C2). (C3) at $(pp3, c_0)$ requires $\mathrm{combine}^\sharp(q_1, \sigma(\mathsf{FunctionResult}\ bump, c_1)) \leq \sigma(pp3, c_0)$, where q_1 is the resume state of the first call's entry pair, and (C4) there requires $e_1 \leq \sigma(\mathsf{Activation_Seed}\ bump\ \ c_1)$.

9.3.2. Solved backward, used forward

The solver discovers unknowns backward from the result of `main`, whereas the soundness proof follows executions forward from the entry and uses the bound (C3) at every node an execution visits. Since the argument is the same in every context, we speak of nodes below.

(C2) connects the two directions. A node's equation reads its predecessors, and the node after a call reads the call site, so every node that reaches a solved result along such steps is solved as well. It therefore suffices that every node of a procedure reaches the procedure's result along graph edges, whether or not the guards on the way can hold. Code after a `return` is the case to check, since no execution reaches it but the compiler still compiles it. In Figure 9.2, line 3 reaches the result through `return 1`, and line 6 reaches it through the return edge without a value that the compiler adds at the end of every body (Section 3.6.1). Once the result of `f` is solved, (C2) therefore makes every node of `f` solved, dead code included.

```
1 fun i32 f(i32 b) {        VIMP ↗
2   return b;
3   if (b < 0) {
4     return 1;
5   }
6   b = 2;
7 }
```

Figure 9.2: Code after a `return` in `f`. No execution reaches lines 3 to 6, yet every line reaches the result of `f`: line 3 through `return 1`, line 6 through the final return edge.

The proof therefore uses the nodes of procedures whose result is solved in the same context. An execution starts at the entry of `main`, which is solved because the result of `main` is the query unknown (C1) and every node reaches its procedure's result. Intra edges and the step from a call site to its continuation stay inside a procedure, and a call reads the callee's result in the context it computes, which is then solved by (C2). A call whose entry state the bottom test classifies as empty reads no result (Section 8.1), so its callee may stay unsolved, but then no covered caller store makes the call, since such a state describes no store. Executions thus never leave this set.

The final return edge is the implicit return of a body that reaches its end without a `return`. In a body that always returns, it starts in dead code, where the analyzer computes $\bot$, and adds nothing. Without it, code after a `return` could end in a node without successors, and the proof would need a syntactic description of the code no `return` cuts off.

The restricted set is `dg_analysis.live_unknowns`, and `dg_analysis.live_unknowns_cover` proves from a well-formed program and a terminating solve alone that it contains the entry and is closed under execution steps. The final theorem therefore needs no premise about which unknowns the solve visited.

9.3.3. The solver as a parameter thy

The soundness argument uses the certificate (C1) to (C4) of a solve that returns and three further facts, and none of them depends on how the solver works. The solved set is finite, the run lies in the domain on which the certificate and finiteness hold (Section 2.7), and the executable run returns that same result. Finiteness lets the analyzer combine the finitely many contexts solved at a node into one verdict. The solver is therefore a parameter of the analysis (`certified_solver` states the certificate and the three facts), and another solver, for instance one with local side effects, attaches to Voblint by proving them. `td_certified_solver` proves them for the vendored solver under every update rule.

9.4. Merging contributions to global unknowns thy

A global unknown such as a seed receives contributions from several places. The solver merges each new contribution into the value it holds with an *update rule*. The certificate, and with it soundness, holds under each of the five rules below (`td_certified_solver`), so the choice affects only precision and termination. Joining keeps every contribution exactly, but a value may then grow along a very long chain, such as the intervals $[0, 0], [0, 1], [0, 2], \ldots$ of a 32-bit counter. Widening bounds this growth but can lose precision. Stemmler et al. compare existing update rules and propose new ones [45, §3–4], and Tilscher et al. formalize a generic interface for them and prove five rules sound against it [4]. Voblint exposes all five. Every rule records the latest contribution of each origin, the right-hand side that sent it (Section 2.7), and in the analyzer each origin's contributions to one target arrive already joined (Section 9.1.4). The rules differ in how they compute the value and whether they widen:

- *join* joins each contribution into the value and never reads its records;
- *join per origin* records each origin's latest contribution and reports the join of the records;
- *warrow* records the same, and warrows the value toward the join of the records;
- *warrow per origin* warrows the origin's old record with the new contribution, then joins the records;
- *bounded narrowing* does the same, but also counts how often an origin has switched from widening to narrowing. Once the count reaches a bound, a switch to narrowing still applies one narrowing step, but further smaller contributions of that origin are ignored until it grows again.

Table 9.4 runs all five on one sequence of contributions, and the project site runs them on a program with several shared callee entries. In the analyzer, the rule decides how a callee's entry state accumulates across call sites and, with flow-insensitive program globals (Section 6.4), how a global accumulates its writes. No rule is more precise on every program, and on one program the joining rules give no answer within the time limit where the warrowing rules answer (Section 13.3).

9.5. Making abstract states executable thy

The solver decides after every evaluation whether an unknown changed, so it needs a decidable order on abstract states. The pointwise analyses of Chapter 5, however, are specified

rule, vendored update	**1:** A sends `[0,3]`	**2:** B sends `[4,4]`	**3:** A sends `[1,1]`	**4:** A sends `[0,8]`
join `update_global_always_join`	`[0,3]`	`[0,4]`	`[0,4]`	`[0,8]`
join per origin `update_global_per_origin`	`[0,3]`	`[0,4]`	`[1,4]`	`[0,8]`
warrow `update_global_warrowing_apinis`	`[0,3]`	**`[0,+∞]`∇**	`[0,4]`	**`[0,+∞]`∇**
warrow per origin `update_global_warrowing_per_origin`	`[0,3]`	`[0,4]`	`[0,4]`	**`[0,+∞]`∇**
bounded narrowing, bound 5 `update_global_bounded_narrowing`	`[0,3]`	`[0,4]`	`[0,4]`	**`[0,+∞]`∇**

Table 9.4: One global unknown under each update rule, from $\bot$, after four interval contributions from origins A and B. At step 3, join per origin replaces A's `[0,3]` by `[1,1]`. Warrow widens at step 2, because `[0,4]` is not below `[0,3]`, and narrows at step 3. Warrow per origin widens only when A's own contribution grows, and bounded narrowing agrees with it here. A widened bound is bold and marked ∇. The vendored rules compute the cells on the unbounded intervals `ivl`. At a machine kind, widening jumps to the kind's limit instead of ∞ (Section 5.2). The contributions are illustrative and come from no program.

over total functions from variable identities to abstract values tagged with their kind (`tabs`, the relational order analysis has its own state type). This suits proofs, but deciding $f \leq g$ for two such functions means checking $f(x) \leq g(x)$ for infinitely many identities x. Nipkow and Klein meet the same problem in the abstract interpreter of Concrete Semantics and represent a state by a finite list of variables with their values, in which every unlisted name reads as $\top$ [7, §13.6]. This is a *default dictionary*, a finite dictionary that answers every missing key with a default. Voblint uses the same idea with two defaults and represents a total function exactly, so it loses no precision. We call a state in this representation an *executable state*.

Two defaults are needed because VIMP initializes globals to zero, as C does for objects with static storage (§6.7.9p10), and leaves the locals of `main` arbitrary within their kinds (Section 3.3). An executable state therefore pairs a default dictionary for locals with one for globals, each a default and a finite list of overrides. In the initial state (`nonrel_init_st`), the local default is $\top$ and the global default is the abstract zero of kind `i32`. Because an abstract value carries its kind, every declared global is overridden by the zero of its own kind.

Different representations can describe the same state, because overrides of distinct names may appear in any order and an override equal to its default changes nothing. A *quotient type* identifies representations up to an equivalence [74], and its operations are lifted with `lift_definition`. The executable state is a quotient type that identifies the representations that answer every lookup alike:

```
(* src/Abstract_Interpreter/Exec/State/Default_St_Base.thy *)                Isabelle
quotient_type ('k, 'a) gdefault_st =
  "('k, 'a::bot) gdefault_st_rep" / "eq_default_st_rep"
  morphisms rep_default_st Abs_default_st
  by (rule equivp_eq_default_st_rep)
```

Two executable states are therefore equal exactly when every lookup agrees (`default_st_eq_iff`), and comparing the two defaults and the finitely many listed names decides the order (`le_default_st_rep_code_iff`).

A lookup reads the dictionary of the identity's *location* $\ell(x)$, local or global (`is_global`), and returns the override if there is one and the default otherwise. We write $d\langle l\rangle$ for the lookup at location l. Looking up every variable at its location gives the total function $fun(d)$ that an executable state represents (`default_st_to_fun`), and an executable state denotes what its function denotes for the non-relational analyses, the stores whose values lie within their kinds and in the concretization of each identity's abstract value:

$$\gamma(d) = \gamma(fun(d)) = \{s \mid \texttt{in_kinds}\ s \wedge \forall x.s(x) \in \gamma(d\langle\ell(x)\rangle)\}.$$

The solver never computes $\gamma(d)$ and uses only the executable lattice operations. The executable states instantiate the lattice classes, including widening and narrowing, which HOL's pointwise function instances lack, on `gdefault_st` whenever the values do, so the generic solver runs on it unchanged.

Each lattice operation applies the value domain's operation to the two defaults and to each listed name, so under lookup it agrees with the pointwise operation on functions. For the join, `default_st_get_sup` states $(d \sqcup e)\langle l\rangle = d\langle l\rangle \sqcup e\langle l\rangle$, and analogous lemmas cover the other operations. The transfer functions and procedure entry commute with the represented function in the same way,

$$fun(\mathrm{op}_{\mathrm{exec}}(d)) = \mathrm{op}_{\mathrm{abs}}(fun(d)),$$

and since $\gamma(d) = \gamma(fun(d))$, the soundness facts of Chapter 6 transport from the abstract to the executable operation (`nonrel_exec_spec_sound`). The commutation is proved once for every non-relational domain, because the abstract and executable steps are derived from the same operations.

Emptiness, on which `DEAD` rests (Chapter 5), is the one operation the listed overrides do not decide, since an unlisted identity can make the state empty through its default. The emptiness test therefore inspects both defaults and the overrides that the represented function reads, and it succeeds exactly when that function maps some identity to an empty value (`st_is_bot_iff`). A state it finds empty denotes no store (`st_is_bot_sound`), so replacing it by Bot loses no store.

9.6. Why termination is not proved

The previous sections assume that the solve returns, and no theorem shows that every solve does. HOL admits only terminating recursion, so Isabelle defines the solver with a domain predicate (`solve_dom`, Section 2.7), the arguments on which its recursion terminates, and proves the certificate of Section 9.3 on that domain. The analyzer runs the executable form `solve_c`, which returns only when the recursion finishes, and `solve_dom_of_solve_c` turns a returned run into domain membership. The analyzer's soundness theorems therefore have no termination premise.

Under call strings of bounded length k, which include the context-insensitive policy as $k = 0$ with the empty call string as its single context, the contexts of a compiled program form a finite space (`compiled_call_strings_finite`), and so do its unknowns. Under entry-state contexts no theorem bounds the number of contexts. With Interval, a recursion that changes its argument at every level demands a fresh context at every level,

and widening bounds the values of existing unknowns but not their number. A recursion that passes its 32-bit argument on changed can thus enter with about 2^{32} distinct values (`01-entry_state_context_chain_timeout`). Goblint's configuration for the SV-COMP competition avoids this explosion by omitting intervals from the contexts of recursive functions [32, §3.4.3]. Finitely many unknowns do not ensure termination either. Without contexts, the interval recursion `f(x) { f(x + 1) }` entered with $x = 0$ makes the one seed of `f` grow through $[0, 0], [0, 1], [0, 2], \ldots$, and the joining update rules never widen this chain, which has about 2^{31} steps before `x + 1` wraps (`01-recursion_grows_join_timeout`). Neither program answers within its time limit, which shows neither divergence nor termination. Both are recursive. For partial tabulation over abstract calling contexts, Schulze Frielinghaus et al. give a local solver that terminates on every non-recursive program, even over lattices with infinite ascending or descending chains [75].

The fixed widths have a further consequence, which we argue here and the development does not prove. A compiled program has finitely many nodes and fixed-width variables, so its graph runs meet only finitely many stores restricted to the identities the program uses, and a run keeps one stack of frames, each recording a return node, a destination and the caller's store (Section 3.6.2). The graph runs are therefore those of a pushdown system with finitely many control states and stack symbols, in which reachability is decidable. Whether some graph run violates a check is thus decidable, and by the forward simulation (Section 3.7) a negative answer carries over to every source execution. The exact procedure explores a state space exponential in the number of variable bits, since three 32-bit variables already give 2^{96} stores, and Voblint does not use it. Rice's theorem (Chapter 1) thus does not apply to compiled VIMP programs, and abstraction is needed there because of the size of the state space.

The vendored termination theorems cover top-down solvers without side effects over a finite type of unknowns. Under this condition the solver with separate widening and narrowing phases terminates, given widening and narrowing operators whose iterations are ultimately stable [76, Thm. 2], and the warrowing solver additionally needs a precise widening and monotonic right-hand sides and dependencies [76, Cor. 1]. These theorems need the Isabelle type of unknowns to be finite. Voblint's unknowns pair nodes of type `cfg_node` with contexts, and this type is infinite although each program uses finitely many nodes, so the theorems apply to none of its configurations. Fixed widths leave only finitely many intervals at each kind, but they make neither the type of unknowns finite nor the widening precise. Seidl and Vogler prove on paper that their side-effecting solver terminates when side effects target only unknowns without a right-hand side, finitely many unknowns are encountered [61, §9, Thm. 5], and the widening and narrowing iterations are ultimately stable [61, §3]. The two joining update rules never widen, so this result does not carry over to them, and we have not checked whether it covers the three warrowing rules of Section 9.4. Mechanizing such a result for the vendored solver is future work, so the end-to-end theorem is a partial-correctness result about every report the analyzer returns.

10. What a Report Guarantees

Solving the equation system of Chapter 8 with the solver of Chapter 9 yields an abstract state for every pair of a CFG node and a context the solver reached, and a value for every global unknown. By Section 9.3.2, these states cover every execution. A user, however, reads neither states nor contexts but a report with one verdict for every `__voblint_check` and a diagnostic wherever a division or a signed operation may fail or a conversion may receive a value its target kind cannot represent. Neither `PROVED` nor `REFUTED` claims that its check is reached. `PROVED` says the condition holds whenever a run reaches the check, and `REFUTED` says that it fails there.

10.1. From solution to report thy

The constructions of the previous chapters are packaged behind one HOL function, `run_voblint`. It takes an analysis configuration (`voblint_conf`), the kind declarations ds of the program's variables and a surface VIMP program p. The configuration selects the analyses that run together (Chapter 7), the update rule (Section 9.4), the context policy (Section 8.2), the placement of program globals (Section 6.4), and the signed-overflow policy the domains read. The analyzer compiles the elaborated program to its graph `elab_cfg` p ds (Chapter 3), builds the equation system (Chapter 8), solves it from the exit of `main` (Chapter 9), and builds a report from the solution.

Before it compiles anything, `run_voblint` checks that the list of analyses is nonempty and distinct and that the program meets the structural conditions the compilation theorems of Chapter 3 assume, so that a report exists only where the soundness proofs apply. A failing input is answered with `Invalid_Activation` (here *activation* means the selection of analyses, unrelated to procedure activations) or `Malformed_Program`. `No_Answer` is the value HOL assigns to a solve whose recursion does not finish, and the generated code then runs forever instead of returning it (Section 9.6). Every answer `Analysed` res thus comes from inputs that pass both tests (`run_voblint_report_contract`), and the executable program test implies the well-formedness premise of the proofs (`wf_program_compile_input_exec_sound`). Soundness at the source level therefore needs no premise about the configuration or the program.

The report (`analysis_report`) keeps the solver's abstract states, each under its CFG node and context, and builds the verdicts and diagnostics on them. Since the solver may analyze a procedure once per context (Section 8.2), a CFG node can carry several states. The playground draws both pictures. Its Analysis view shows one node per pair of a program point and a context, and its Control-flow view shows each node of the compiled CFG once, with the states of all its contexts grouped there. The report takes the second view. At a node v it describes every store that one of its states there describes, over all contexts the solver solved v in (`report_states_at`):

$$\bigcup_{\text{Lifted d} \in \text{report_states_at res v}} \texttt{act_gamma as d},$$

where `act_gamma` is the concretization of the selected analyses as. It intersects their individual concretizations as in Section 7.3 and reads a report state as γ_M reads a solved state in Chapter 8.

The union drops the contexts on purpose. A context is a label the analysis attaches to activations to keep calls apart (Section 8.2), so a source execution carries no context it could be compared with, and a user asks what holds at a program point, whichever call led there. Inside the proof the contexts still matter, because each per-context state over-approximates the activation collecting semantics of its context (Section 8.3, with the solved set of Section 9.3.2).

10.2. Verdicts

Each check at v receives one of four verdicts, `PROVED`, `REFUTED`, `UNKNOWN`, or `DEAD`, which the command-line tool prints at the source position of the check and the browser playground shows inline in the editor.

How a verdict is computed. Each state d in `report_states_at res v` gives its own verdict for a check (`classify_point`). The state Bot gives `DEAD`, and any other state is classified by `mcp_classify`, which asks every active analysis for the truth value of the check's condition through the query system of Section 7.1 and combines the answers by meet. An exact 1 gives `PROVED`, an exact 0 gives `REFUTED`, and any other answer gives `UNKNOWN`. A definite answer holds in every store the state describes (`mcp_classify_sound`).

The verdict of the CFG node joins the verdicts of its states (`aggregate_verdicts`). It is `PROVED` if every state other than Bot gives `PROVED`, `REFUTED` if every such state gives `REFUTED`, and `UNKNOWN` otherwise. No definite verdict is sound when the states disagree. If one context proves `x > 0` and another refutes it, the stores reaching v in the first satisfy the condition and those in the second violate it, so no single definite verdict holds for every store at the node. Bot states are neutral, so a node at which the report holds no state, or only Bot states, is `DEAD`. This is sound for nodes the solver never reached, because executions never leave the solved unknowns (Section 9.3.2), so such a node collects no store.

What a verdict claims about stores. To state that the verdicts are correct as an inclusion of sets, we turn each verdict at v into the set of stores consistent with it. If v carries the check `x > 0` with verdict `PROVED`, this set is $\{s \mid s(x) > 0\}$. `REFUTED` gives the stores in which the condition fails, and `UNKNOWN` gives every store. Conditions are evaluated with VIMP's `eval`:

```
(* src/Executable_Surface/CLI/Analysis_Report.thy *)                    Isabelle
fun verdict_holds :: "check_result ⇒ exp ⇒ store ⇒ bool" where
  "verdict_holds Check_Proved e s = truthy (eval e s)"
| "verdict_holds Check_Refuted e s = (¬ truthy (eval e s))"
| "verdict_holds Check_Unknown e s = True"
```

A `DEAD` check claims that no store arrives at v. This claim is carried by the report's empty states at v, so the verdict itself restricts no store, and as a set of stores it admits every store (`check_stores`). A store is consistent with what the user reads at v if it lies in the set of every check at v. In a compiled program a node carries at most one check, because the compiler starts every check at a fresh node (Section 3.6.1). Later statements write

`PROVED res v c` and `REFUTED res v c` when the report lists a check of condition c at v with that verdict. As in the theories, c here names a condition, not a context. This set can be much larger than the set of stores the report's states at v describe, because a verdict keeps only the truth value of one condition. Soundness needs only that every store reaching v lies in it.

Since a node verdict is definite only if all its states agree, a definite node verdict holds at every store the report covers at v, in particular at every store that reaches v (`prog_report_of_sound`).

10.3. The chain at one check

Let res be a report `run_voblint` returns for a program p with kind declarations ds, g = `elab_cfg` p ds its compiled graph, κ = `kenv_of` ds the kind of every variable, and S = `init_stores` κ its initial-store set (Section 3.3, Chapter 4). A store is *reached* if some finite run of `main` in the elaborated program, started from a store in S, arrives at it. The argument that a verdict is sound places a reached store at a CFG node v and follows a chain of inclusions between sets of stores there. It ends in the two sets of the report: the stores its states at v describe, and the stores its verdicts at v admit. Each step is a theorem:

a store s a source run reaches, at a node v related to the run

$$
\begin{aligned}
&\overset{\in}{\underset{\texttt{source_reaches_node_collect}}{}} && \mathcal{C}_{\kappa,\texttt{g,S}}\ \texttt{v} \\
&\overset{=}{\underset{\texttt{node_collect_eq_Union_activation_collect}}{}} && \bigcup_{c} \mathcal{A}_{\kappa,\texttt{adm},c_0,\texttt{g,S}}\ \texttt{v}\ \texttt{c} \\
&\overset{\subseteq}{\underset{\texttt{run_voblint_sound}}{}} && \bigcup_{\texttt{Lifted d} \in \texttt{report_states_at res v}} \texttt{act_gamma as d} \\
&\overset{\subseteq}{\underset{\texttt{mcp_classify_sound}}{}} && \bigcap_{\texttt{ch} \in \texttt{report_checks_at res v}} \texttt{check_stores ch}
\end{aligned}
$$

The first step places a reached store in the node collecting semantics at a node v that the simulation `csim` relates to the run's configuration (Section 3.7). The second is the exhaustive-contexts theorem of Section 4.4, which holds for every policy whose claim meets the five obligations of Chapter 4, and contexts enter the argument only at this step. The third inclusion is the analyzer's soundness theorem `run_voblint_sound`. It bounds the stores collected at v, over all contexts, by the union of the report's states at v, without saying which state covers a given store. Its proof bounds each per-context set by the solved state in that context (Section 8.3) and then uses the equality of the second row. The fourth is the verdict argument of Section 10.2, which `mcp_classify_sound` proves for one state and `prog_report_of_sound` for a node, and `run_voblint_sound` also states it for the reached stores at a check's node.

The program of Figure 10.1 calls `f` twice, first with 1 and then, if the input x satisfies $1 \le x \le 3$, with x. So `m` is 2, 4, or 6 at the checks. The check `m >= 2` always holds, and `m == 2` holds at the first call and fails at the second call when x is 2 or 3.

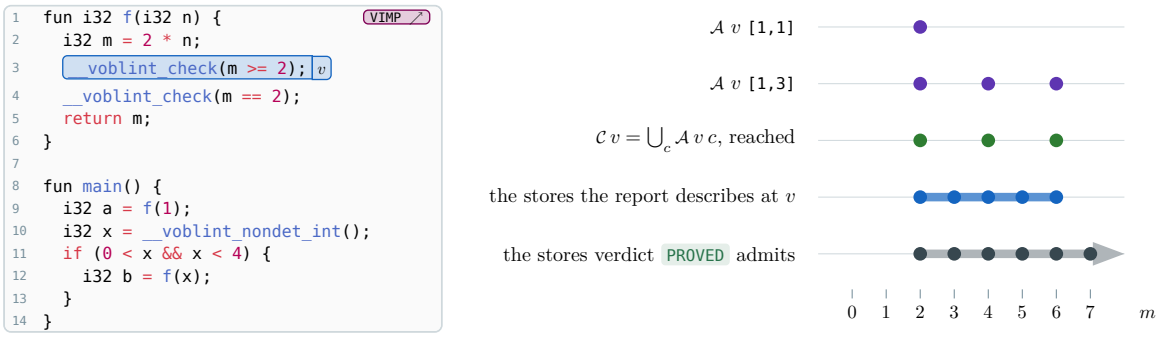


Figure 10.1: Left: the program. Right: the chain at the node v of the check `m >= 2` (line 3), projected on m, with Interval and entry-state contexts. Each row is a set of stores. Both per-context rows lie inside the union row, and each later row lies inside the rows below it. The report admits 3 and 5, which no run reaches, and the stores verdict `PROVED` admits are unbounded, since a store is not restricted to a kind's range. The check `m == 2` (line 4) sees the same stores. Report and verdict are analyzer output (claim `chain-split-entry`). The first three rows are derived by hand.

`f` is analyzed in two contexts, one per entry state. The check `m >= 2` holds for every admitted store in both, so its verdict is `PROVED`. The check `m == 2` sees the same sets, but the contexts disagree. The first call enters with $n = 1$, and the check is `PROVED` there. The second enters with $n \in [1, 3]$. Its state `[2,6]` also admits 3 and 5, and the check is `UNKNOWN`, the only sound answer, since the run with $x = 1$ satisfies it and the runs with $x = 2$ and $x = 3$ violate it.

The report does not say which context covers a reached store, so the node verdict joins the context verdicts, and `PROVED` and `UNKNOWN` give `UNKNOWN`. Without contexts, both calls feed the one seed of `f`, which the warrowing update rule widens, and the check is `UNKNOWN` at `interval: m=[-2147483648,2147483647]` (claim `chain-split-none`). All of these verdicts are sound. They differ only in how many calls the context policy keeps apart.

10.4. The source-level theorem thy

One theorem states the chain at one store for the exported analyzer. It quantifies over an arbitrary analysis configuration and program, the arguments of `run_voblint` (Section 10.1). It is about `run_voblint` itself, the constant that code generation exports, so it concerns the analyzer the tools run, up to the trusted code generator and target toolchains (Section 12.5).

Theorem 10.1 (Source-level soundness of the analyzer, `run_voblint_source_sound`)**.**

```
(* src/Executable_Surface/CLI/Run_Voblint.thy *)                    Isabelle
theorem run_voblint_source_sound:
  assumes run: "run_voblint conf ds p = Analysed res"
    and s0: "s0 ∈ init_stores (kenv_of ds)"
    and exec:
      "(kenv_of ds),(elab_ptable p ds) ⊢ (main_body (elab_ptable p ds), s0, []) →p*
      (residual, s, frs)"
  shows "∃v stk. elab_ptable p ds, elab_cfg p ds, prog_main_name
                    ⊢ (residual, s, frs) ≈ (v, s, stk)
           ∧ s ∈ reaching ds p v
           ∧ (∃d. Lifted d ∈ report_states_at res v
                 ∧ s ∈ mcp.act_gamma ds (activation_of (conf_analyses conf)) d)
           ∧ ¬ DEAD res v
           ∧ (∀c. PROVED res v c ⟶ truthy (eval c s))
           ∧ (∀c. REFUTED res v c ⟶ ¬ truthy (eval c s))
           ∧ (SAFE res v ⟶ (∀ob ∈ obligations_at ds p v. s ⊨ ob))"
```

The theorem speaks about the elaborated program, with the core procedure table `elab_ptable` p ds of Section 3.2 and its graph `elab_cfg` p ds. It assumes three premises:

(P1) `run_voblint` conf ds p = Analysed res: the analyzer returned a report. Input validity and solver termination are therefore not separate premises (Section 10.1), and the theorem is a partial-correctness result: termination is not proved for every program (Section 9.6).

(P2) `s0 ∈ init_stores (kenv_of ds)`: the run starts from an initial store of Section 3.3, with every global 0 and every other variable some value of its kind.

(P3) a finite `pstep` execution of the elaborated program runs from the body of `main` to a configuration $(\text{residual}, s, \text{frs})$ with store s. Any stopping point is allowed, so nonterminating programs are covered through their prefixes.

The conclusions are stated at graph nodes, but a source configuration names none, and its residual can occur at several places of the program. The simulation `csim` of Section 3.7 supplies the node by relating the source configuration to a graph configuration at a node v with the same store, and we say that the configuration is *observed* at v. A configuration reachable along two paths can be observed at two nodes. For a check this does not weaken the guarantee. A run about to execute a check is observed only at a node that carries the check's edge, and `run_voblint_check_sound` (Section 10.5) finds the check's report entry by its label there, so the existential choice of v matters only for configurations between checks.

The conclusion states that the configuration is observed at some node v, and that at this v

(S1) `s ∈ reaching ds p v`: the store is collected at that node, where `reaching ds p v` abbreviates the node collecting semantics $\mathcal{C}_{\kappa,\mathtt{g},\mathtt{s}}$ `v` of the elaborated graph from the initial stores (Section 10.3);

(S2) some state `Lifted d ∈ report_states_at res v` the report holds at v, under one of its contexts, describes s;

(S3) v is not `DEAD`, every `PROVED` check listed at v holds in s, every `REFUTED` check is false in s, and if no arithmetic diagnostic names v (`SAFE res v`), every obligation `ob ∈ obligations_at ds p v` of the divisions, signed operations and conversions evaluated at v holds in s, written `s ⊨ ob`. Section 10.5 explains these obligations.

Whenever the analyzer returns a report, every store that a run of `main` reaches is thus collected, covered by the report and consistent with its verdicts at a node where the run is observed, which is the chain of Section 10.3 read at one store. The statement mentions no activation and no context, since source executions carry neither. Both appear only inside the proof, through the per-context bounds and the equality of the second inclusion. `run_voblint_spine` keeps them visible. It states the chain once for every context policy, together with a representing activation trace and a context it carries whose set contains the reached store. Its two assumptions are that every valid activation trace carries some context and that the union over contexts equals the node collecting semantics, and `run_voblint_call_string_chain` and `run_voblint_entry_state_chain` discharge them for the shipped policies. The theorems `run_voblint_dead_unreached` and `run_voblint_arithmetic_safe` state the `DEAD` clause and the obligation clause of (S3) directly at a node, for every collected store and without a source run.

10.5. What a verdict says about executions thy

What the four verdicts mean. By the theorem of Section 10.4, each verdict becomes a claim about *covered executions*, the finite runs of the elaborated `main` from an initial store in `init_stores`, stopped at any point. Because the report's states may also describe stores that no execution reaches, a verdict can only claim what holds whenever an execution reaches its check.

- `PROVED`: whenever a covered execution reaches the check, the condition holds (`run_voblint_check_sound`).
- `REFUTED`: whenever a covered execution reaches the check, the condition fails (`run_voblint_check_sound`). It is not a verified counterexample: it establishes no reaching execution.
- `UNKNOWN`: the analysis decides neither.
- `DEAD`: no covered execution reaches the check: nothing is collected at its node (`run_voblint_dead_unreached`), and a check that an execution reaches is never `DEAD` (`run_voblint_check_sound`).

`PROVED` and `REFUTED` hold vacuously when no execution reaches the check, and only `DEAD` claims anything about reachability. The corresponding theorem for `PROVED` and `REFUTED` is:

```
(* src/Executable_Surface/CLI/Run_Voblint.thy *)                                    Isabelle
theorem run_voblint_check_sound:
  assumes run: "run_voblint conf ds p = Analysed res"
    and s0: "s0 ∈ init_stores (kenv_of ds)"
    and exec:
      "(kenv_of ds),(elab_ptable p ds) ⊢ (main_body (elab_ptable p ds), s0, []) →ₚ* (residual, s, frs)"
    and chk: "next_check residual = Some (l, e)"
  shows "∃c ∈ set (report_checks res). check_label c = l ∧ check_exp c = e
           ∧ s ∈ reaching ds p (check_point c)
           ∧ check_verdict c ≠ Dead
           ∧ (check_verdict c = Decided Check_Proved ⟶ truthy (eval e s))
           ∧ (check_verdict c = Decided Check_Refuted ⟶ ¬ truthy (eval e s))"
```

Truth and reachability are separate. In the program below, `n` and `x` are 32-bit unsigned, so $x = 4n \bmod 2^{32}$. Since 4 divides 2^{32}, x is a multiple of 4 and never 1, 2 or 3, so no run reaches the check. Interval does not track divisibility, so it keeps `interval: x=[1,3]` there, where `x > 0` holds, and reports `PROVED`. The verdict is sound: it claims `x > 0` only for runs

that reach the check. Int also tracks $x \equiv 0 \pmod 4$, a class that the reduction modulo 2^{32} preserves, finds the state empty, and reports `DEAD`.

```
fun main() {
  u32 n;
  u32 x = 4 * n;
  if (0 < x && x < 4) {
    __voblint_check(x > 0);
  }
}
```

VIMP

An unreached check is therefore not necessarily `DEAD`. The analyzer reports `DEAD` only where its emptiness test recognizes every state at the node as describing no store. The test is sound (`sound_emptiness_mcp`) but not exact. Each numeric domain decides the emptiness of its own value exactly, but the combined test asks each active analysis separately, and two analyses can each admit stores while no store satisfies both. An unreached check can thus miss `DEAD` because its state describes stores that no run reaches, as Interval's state above does, or because the test does not recognize that it describes none. The verdict is sound either way.

Arithmetic diagnostics. VIMP gives every arithmetic operation a defined result where C leaves some undefined or implementation-defined (Section 3.4), so the analyzer checks these hazards without a check in the program. The divisor of a division or remainder must be nonzero, and a signed operation must have a mathematical result inside its kind. For a signed remainder the quotient must also lie inside the kind, which `INT_MIN % -1` violates, since the quotient 2^{31} exceeds `i32`. A conversion, implicit or explicit and including the result of `min` or `max`, must receive a value its target kind represents. Each such operation carries an obligation, which the analyzer decides from the analyses' states. It reports an *error* (a refuted diagnostic) if the analyses show that the obligation fails, a *warning* if they cannot show that it holds, and nothing otherwise. An error is sound. Whenever a covered execution reaches its node, the obligation fails (`run_voblint_arithmetic_refuted`). A warning means only that the analysis could not establish the obligation.

The conversion obligation is that a conversion keeps its value, so it also covers conversions that C defines. C reduces a value converted to an unsigned type modulo 2^w (§6.3.1.3p2), as VIMP does, so the conversion of 300 to `u8` in the listing of Section 3.2 stores 44 in both languages. The analyzer still reports it as an error, because the conversion changes the value in every run. A declaration without an initializer raises no diagnostic. The frontend writes it as a call of `__voblint_nondet_int` (Section 3.2), which yields a value of the destination's kind. Even a node without diagnostics is not shown free of undefined behavior in C, because the diagnostics concern arithmetic only and VIMP does not model C's indeterminate values, some reads of which are undefined (§6.3.2.1p2).

How a domain bounds a signed operation that may overflow, by any value of the kind or by following the wraparound, is a run option after Goblint's `sem.int.signed_overflow` (Section 5.2). The concrete semantics wraps under both settings, and the guarantee for errors above and the following one hold under either. At a node v without a diagnostic, every obligation of an expression evaluated at v, on an intraprocedural edge or in a call argument, holds in every store collected at v. If a `return` edge of a callee leaves v, the returned value fits the narrower kind to which a caller converts it, and the result of a `min` or `max` computed at v fits the kind of its destination (`run_voblint_arithmetic_safe`).

PART V

Instances and Practice

11. Five Domains and an Order Analysis

Chapter 10 proved that a report is sound regardless of which analyses produced it. A user can choose among the numeric domains Sign, Interval, Parity, and Congruence, their combination Int, and the relational order analysis. The four base domains are textbook material, and the project site lets readers try them interactively. Int combines Interval and Congruence and gains precision by letting them exchange facts. The formalization proves that this exchange keeps every integer a value stands for and shows why narrowing must skip it.

11.1. The implemented domains

The four base domains, Sign thy , Interval thy , Parity thy , and Congruence thy , are the classical non-relational domains [50], shown with one value each in Figure 11.1. Sign and Parity are finite. A congruence class cannot grow forever either, because growing from a single integer gives it a positive modulus and every later strict growth replaces that modulus by a proper divisor [50, §4.8], an argument not mechanized here. Sign, Parity and Congruence therefore widen by the join, and only Interval has a widening that jumps ahead (Section 5.1).

domain	value	the integers it denotes
		−6 −4 −2 0 2 4 6 8 10
Sign	≥0	… ■ ■ ■ ■ ■ ■ ■ ■ …
Interval	[-2,5]	… ■ ■ ■ ■ ■ ■ ■ ■ …
Parity	1+2ℤ (odd)	… ■ ■ ■ ■ ■ ■ ■ ■ …
Congruence	2+3ℤ	… ■ ■ ■ ■ ■ ■ ■ ■ …
Int	[1,9], 1+4ℤ	… ■ ■ ■ ■ ■ ■ ■ ■ …

Figure 11.1: One value of each implemented domain, as the analyzer prints it, and the integers it denotes between −6 and 10. Congruence and Parity values print as $c + m\mathbb{Z}$, the integers congruent to c modulo m ($m\mathbb{Z}$ when $c = 0$). An end cell is filled when the set continues beyond the window. The values are illustrative. The Int value denotes the intersection of its components' meanings, here $\{1, 5, 9\}$.

Each of the five domains is an instance of the class `kind_int_domain` (Section 5.7), and the order analysis proves the laws of its local specification (`ord_spec_sound`). The generic construction derives sound transfer functions for every instance of the class, first on function-valued states (`nonrel_spec_sound`) and then on their executable representation (`nonrel_exec_spec_sound`, Section 9.5). Hence `run_voblint_source_sound` covers every selection of these analyses. Making a new domain selectable, however, is not generic. It needs a new case of the `analysis` datatype, a field of the combined state, and the cases that give the field its lens, its concretization and its answers (Chapter 7). The proofs about the

combined state, an instance of the locale `mcp_registry`, proceed by cases on the analysis and must cover the new case.

11.2. One loop, five answers

The loop below is the fixture `05-mine_ex410_stride2_parity.vimp`, adapted from Goblint's regression test `56-witness/29-mine-tutorial-ex4.10.c` after Example 4.10 of Miné's tutorial [50]. In every run v takes the odd values $1, 3, \ldots, 51$ and exits with $v = 51$, so every check holds.

```
1  fun main() {                                                    VIMP
2    i32 v = 1;
3    while (v < 51) {
4      __voblint_check(v >= 1);
5      v = v + 2;
6      __voblint_check(v <= 52);
7    }
8    __voblint_check(v >= 51); line 8
9    __voblint_check(v <= 52);
10   __voblint_check(v == 51); line 10
11 }
```

domain	records	state at the exit	`v >= 51` line 8	`v == 51` line 10
sign	sign of a value; finite	`v=+`	UNKNOWN	UNKNOWN
interval	bounds; infinite ascending chains	`v=[51,52]`	PROVED	UNKNOWN
parity	evenness	`v=1+2ℤ`	UNKNOWN	UNKNOWN
congruence	residue modulo m	`v=⊤`	UNKNOWN	UNKNOWN
int	bounds and residue, reduced	`v=51`	PROVED	PROVED

Table 11.1: The stride-2 loop under each domain (claims `dom-stride2-*`), at the two highlighted checks of the listing, under the default signed-overflow policy Assume_Top. Only Int, which combines the bound and the oddness, proves `v == 51`.

Each base domain records a different property of v. Interval finds the bound through widening and narrowing, but its exit state still allows 52, and an interval cannot express "odd". Parity knows that v is odd but nothing about its size, and Sign knows only that v is positive. Congruence alone loses even the oddness, because under Assume_Top it cannot exclude that `v + 2` wraps and returns its top (Section 5.2). Int keeps the class, because its interval shows that no value wraps, and by putting the bound and the oddness together it proves `v == 51` (Section 11.3).

11.3. The reduced product thy

An Int value (`int_tuple`) holds one interval and one congruence and stands for the integers that both allow. Running the two side by side would be sound, but a fact that needs both of them would not be found. In the program below, the guards keep x between -2 and

10, and the assignment makes x one more than a multiple of 4. Only 1, 5, and 9 satisfy both, so `x >= 1 && x <= 9` holds. Yet the interval alone still allows -2 and the residue class alone allows -3, so neither component decides the check. A *reduction* lets the components sharpen each other, as in the reduced product [47], [50, Def. 6.3, Ex. 6.2]. Here the residue class moves the interval's bounds to the nearest values that are 1 modulo 4, giving $[1, 9]$. The reduced value stands for the same integers and decides the check (Table 11.2).

```
fun main() {                                                    VIMP
  i32 n = __voblint_nondet_int();
  i32 x = 4 * n + 1;
  if (x >= -2) {
    if (x <= 10) {
      __voblint_check(x >= 1 && x <= 9);
    }
  }
}
```

component of x	without reduction	with reduction
intervals	`[-2,10]`	`[1,9]`
congruences	$1+4\mathbb{Z}$	$1+4\mathbb{Z}$
verdict of the check	UNKNOWN	PROVED

Table 11.2: The value of x at the check of the program above, without reduction and with the default reduction (claims `int-reduction-never` and `int-reduction-fixpoint`), both under wraparound overflow (Assume_Wraparound). Since `4 * n` may overflow, Assume_Top would drop the class (Section 5.2). Both columns describe the integers 1, 5 and 9 in the guard's range, but only the reduced value decides the check.

Reduction also proves `v == 51` in Table 11.1. Inside the loop, the guard `v < 51` leaves $[1, 50]$, and together with "odd" the reduction shrinks this to $[1, 49]$. The increment then gives $[3, 51]$ instead of Interval's $[3, 52]$, the loop head settles at $[1, 51]$, and after the loop only 51 remains.

A reduction round has two steps. The first sharpens the residue class with a finite interval, so a class that meets the interval in one value becomes that value, and one that meets it in none becomes empty. The second moves the interval's bounds to the nearest members of the class. Through `refine_mode` the user chooses whether to skip reduction, reduce once, or reduce until nothing changes. Every mode keeps every integer a value stands for (`it_refine_mode_sound`). Goblint's `IntDomTuple` uses a similar reduced-tuple design with up to six optional components. Int corresponds to it restricted to the interval and congruence components, reduces in the same order, and offers the three modes of Goblint's `ana.int.refinement` setting. Goblint defaults to no reduction, and Voblint to reducing until nothing changes.

The default mode iterates with HOL's while combinator, which in the logic returns no result when the rounds never settle, and Int then keeps the unreduced value. HOL can define this case because a definition may ask whether some number of rounds reaches a stable value without computing the answer. Its code equation instead runs the rounds until they settle, so the generated OCaml may loop where the HOL function returns. The code equation is a theorem, so, up to the trusted code generator, every value the generated code

does return equals the HOL value, and the soundness theorem, which speaks only about reports that are returned, applies unchanged. A run that loops gives no report, as a solve that does not terminate gives none (Section 9.6).

At a machine kind (Section 5.2), Int's arithmetic computes the interval first. When the mathematical interval lies in the kind's range, no value wraps, and the congruence keeps its classical class even where Congruence alone would return its top (`04-int_no_ov_keeps_class`). In Figure 5.2, Int keeps $a \in 4\mathbb{Z}$ and rounds its interval to $[0, 2^{32} - 4]$ (claim `dom-kinds-int`).

Int reduces the result of every conversion, every arithmetic operation, and every step of the guard refinement. Join, widening, and narrowing work componentwise, as in Goblint for join and widening. For narrowing, the solver contract forces the choice. The verified solver relies on the bracket laws $b \leq a \Delta b \leq a$ for $b \leq a$ (Section 2.5). Reduction keeps the integers a value stands for but can make its components smaller in the componentwise order, so a reduced narrowing can fall below b when b itself is not reduced. The domain library's product of a sign, an interval, a parity and a congruence (`int_dom`) gives an evaluated example. There $b = (\top, [-1, 0], \text{even}, \top)$ denotes only 0 and is narrowed against $\top$. Reducing the result lets the exact interval bound sharpen the sign to non-positive, strictly below the sign $\top$ of b (`post_narrow_refinement_would_violate_narrow_ge`). Goblint's `IntDomTuple` does reduce after narrowing. No proof obliges its lattices to satisfy the bracket laws, so Goblint can make this choice and Voblint cannot.

11.4. The order analysis thy

None of these domains can express that $x \leq y$, because each describes the variables one at a time. The order analysis keeps a set of such pairs, the relational state of Section 5.3.2. A guard between two variables adds a pair. An assignment forgets the pairs involving the changed variable and then asks the shared query channel how the new value relates to each variable, and every active analysis, the order analysis included, may answer (Section 7.4). In the program below, the guard `x < y` removes only the extreme values of the kind from Interval's ranges, so Interval keeps `interval: x=[-2147483648,2147483646], y=[-2147483647,2147483647]` and reports `UNKNOWN` at the first check. The order analysis records `order: {x≤y}` and reports `PROVED`.

VIMP

```
fun main() {
  i32 x = __voblint_nondet_int();
  i32 y = __voblint_nondet_int();
  i32 z = __voblint_nondet_int();
  if (x < y) {
    __voblint_check(x <= y);
    if (y <= z) {
      __voblint_check(x <= z);
    }
    y = y + 1;
    __voblint_check(x <= y);
  }
  i32 w = x;
  __voblint_check(w == x);
}
```

Because the analysis does not combine pairs transitively, it cannot prove `x <= z` under `y <= z`. It holds `order: {x≤y ∧ y≤z}` there and answers `UNKNOWN` (Section 5.3.2). The assignment `y = y + 1` throws away every pair involving y, leaving `order:` $\top$. Forgetting is necessary, since at $y = 2^{31} - 1$ the increment wraps to -2^{31} and `x <= y` fails for $x = 0$. A copy `w = x` between two variables of one kind, in contrast, records both orders and gives w the pairs of x. The analysis then holds `order: {w≤x ∧ x≤w}` and proves `w == x`, where Interval answers `UNKNOWN`.

The base domains record different properties, Int lets two of them share facts, and the order analysis proves relations that no domain over single variables can express. Run with the numeric domains in one combined state (Chapter 7), it asks them at each assignment how the new value compares with the other variables, and a check is decided by the meet of all answers.

12. From Theorem to Tool

Chapter 10 proved that every report `run_voblint` returns is sound. A user, however, writes source text and reads a report in a terminal or a browser tab instead of evaluating a HOL function.

12.1. From theorem to code thy

Chapter 9 replaced the two objects of the argument that cannot run, the function-valued abstract states and the solver defined through its domain predicate, with executable versions (Section 9.5, Section 9.6). Every function that `run_voblint` uses therefore has *code equations*, the equations that determine what code generation emits, and Isabelle's code generator [48] exports it, together with the constructors and selectors a caller needs, into one OCaml module, `Voblint_Generated.ml`. The command-line tool and the playground reach this module only through the facade `cli/voblint.ml`, so no handwritten code runs between receiving the syntax tree and returning the report.

The command-line tool links the module with handwritten adapters into a native program, and for the playground `wasm_of_ocaml` compiles the same code to WebAssembly, which a web worker in the reader's browser calls (Figure 12.1). Both tools thus run the generated code of the function the theorems are about, up to the code generator and the compilers.

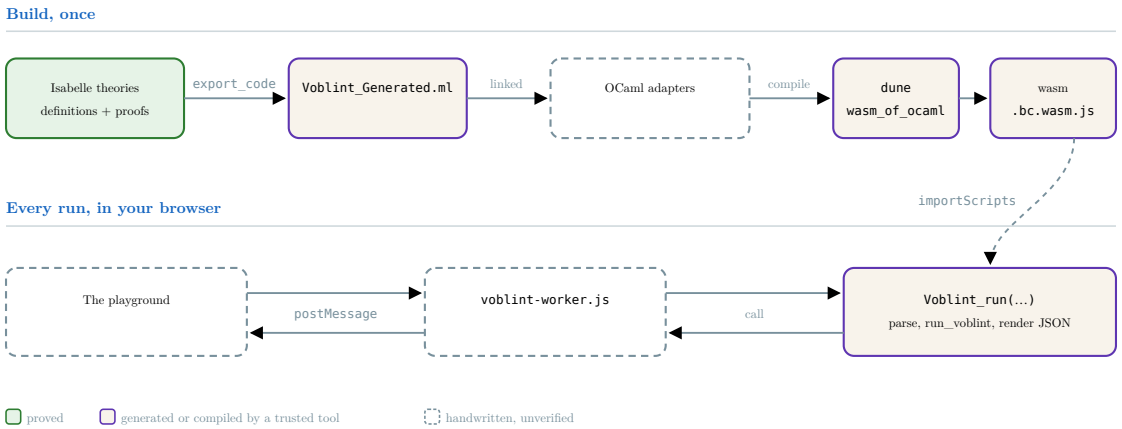


Figure 12.1: How the generated `run_voblint` reaches the browser. `export_code` writes it into `Voblint_Generated.ml`, which `dune` and `wasm_of_ocaml` compile with the handwritten adapters into a WebAssembly bundle that a web worker calls. Adapted from the project site.

12.2. The generated code

The exported module has 19,139 lines and keeps the data representations of the HOL definitions. Type classes become records of operations passed as arguments, and finite sets and maps become lists that membership and lookup scan. The solver keeps its valuation as a function, and each update wraps the previous valuation in another closure, so a lookup may traverse a chain as long as the number of earlier updates.

Up to the trusted code generator, this code computes the function the theorems are about, but no proof says anything about its running time. The generated code also evaluates every term as written. A definition that named the program's entry node through the compiled graph would recompile the whole program at every equation evaluation. The analysis therefore receives the entry node as a parameter, fixed to the entry of `main`, and a theorem shows that this node is the entry of every compiled graph (`cfg_entry_compile_prog`).

The list representations are hard to replace. Isabelle's library implements sets by red-black trees only for element types with a linear order, which abstract values, check results, and entry-state contexts lack. A finite map in place of the solver's function-valued table would remove the closure chains, but it would change the state of the vendored solver and require its proof to be redone. This thesis aims at a sound analyzer, and it reports no measurement of the generated code's time or memory.

12.3. The unverified shell

The generated `run_voblint` takes a configuration, kind declarations and a syntax tree and returns a typed answer, so unverified code surrounds it on both sides. Before it, an `ocamllex` lexer and a Menhir parser turn the program text into a surface program (`imp_prog`) and the kinds of its variables. They also resolve block scopes, turn a declaration without initializer into a call of `__voblint_nondet_int`, and write each check's source position into its label. After it, `render_report` turns the report's states into text, and handwritten code prints the command-line report or serializes the answer as JSON, from which the playground draws the graph and places verdicts and arithmetic diagnostics in the editor. The generated code computes VIMP's machine-integer values as arbitrary-precision Zarith integers, which the semantics keeps within each kind's range by explicit normalization (Section 3.4). For the program below, Figure 12.2 shows the call and the answer that cross this boundary.

```
fun main() {                                                    VIMP ↗
  i32 k = 10;
  i32 q = 100 / k;
  i32 d = __voblint_nondet_int();
  i32 y = 100 / d;
  __voblint_check(0 < q);
}
```

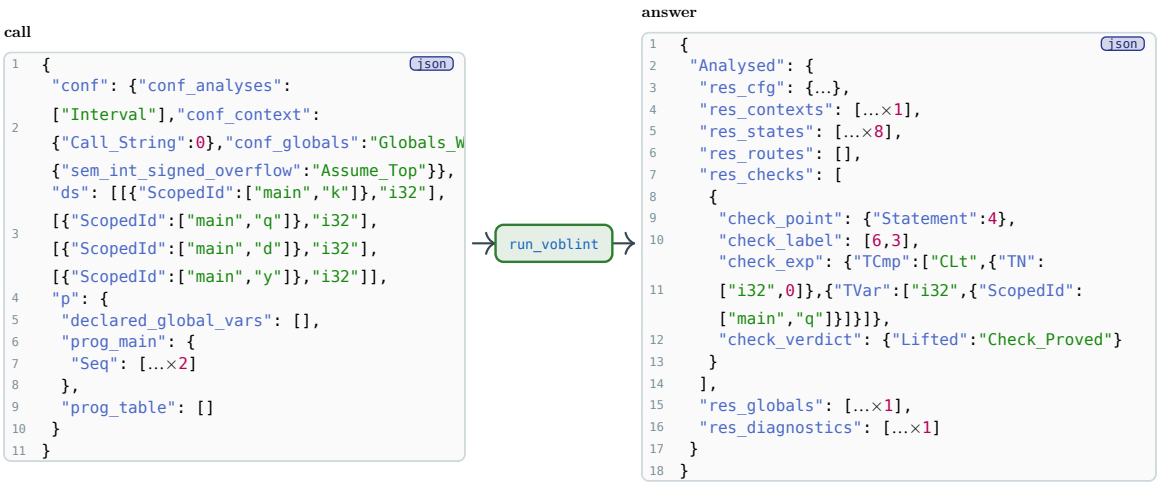


Figure 12.2: What the generated analyzer receives and returns for the program above (claim `shell-json`). The call carries the configuration and the parsed syntax tree `p` (the surface column of Figure 3.3). The answer carries the report with its states rendered as text and its one check in full. Everything before the call, and the rendering from the answer on, is unverified. `{...}` and `[...×n]` fold parts of the JSON. The JSON prints the verdict `Decided Check_Proved` of Section 10.5 under the name `Lifted` of the constructor that `Decided` abbreviates.

The playground is a static page whose toolbar selects the configuration argument of `run_voblint`, over which `run_voblint_source_sound` quantifies, and whether the solver's steps are traced (Figure 12.3a).

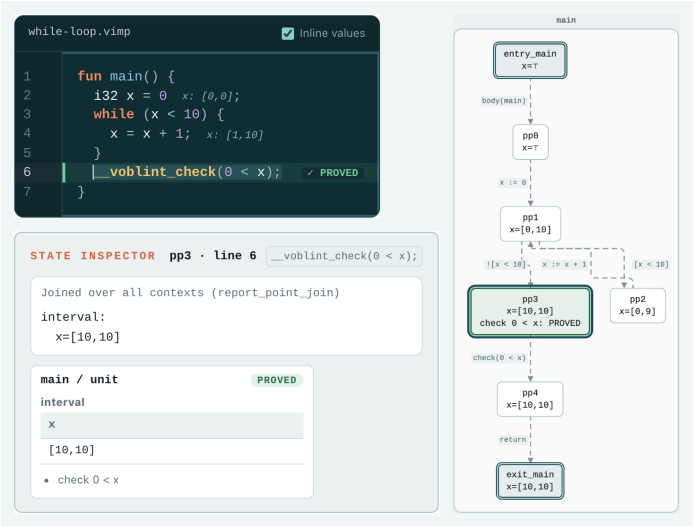
12.4. Tracing the exported solver thy

To explain a run, both tools can trace the solver's steps: which unknown is queried when, which update destabilizes which unknown (Table 9.3), and where widening applies. Tracing must leave the definitions and proofs of the vendored solver [4] untouched. Voblint therefore supplies traced versions of its code equations that report each step through one constant, `trace_event`, which is () in the logic, so each traced equation is proved equal to the one it replaces (for example `solve_rec_c_traced`). Code export uses the traced equations, and in OCaml `trace_event` maps to a hook that records the event when tracing is on. This target mapping joins the trusted base, since the hook must return, raise nothing, and leave the solver's values alone. The trace and its replay in the playground (Figure 12.3b) are unverified observations and never feed back into a result.

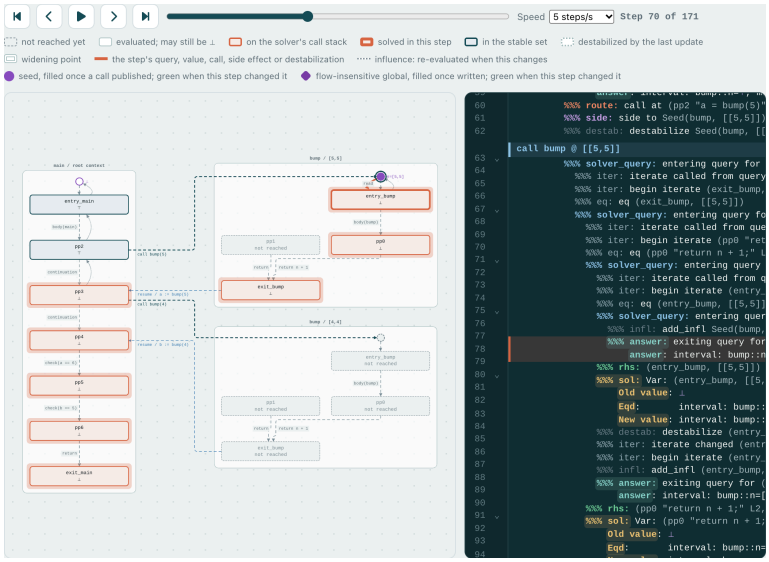
12.5. The trust boundary

Figure 1.4 places the main components of the delivered analyzer relative to the proof.

What is proved. `run_voblint_source_sound` and the verdict theorems of Section 10.5 hold for every report `run_voblint` returns, from the syntax tree it receives to the semantic report. The proof covers the elaboration, the compiler, the well-formedness test, the vendored solver with its executable refinement and the executable state carrier. The



(a) one run in three views: the check's verdict in the editor, the state inspector at the check, and the solved graph. Settings `analysis interval, globals warrow, context none` open this run.



(b) one step of the solver replay for the calls of `bump` (Table 9.3): the graph shows each unknown's value at this step, and the trace beside it marks the step's line. Settings `analysis interval, globals warrow, context entry-state` open this run.

Figure 12.3: The playground. Both panes run the generated analyzer of Figure 12.1 in the reader's browser. The screenshots are illustrative.

vendored solver is a copy of the verified solver of Tilscher et al. [4] with three of the four changes proposed upstream for this thesis and its own build fix, and its proofs are checked with the rest of the development (Section 14.3). Its repository is private, so the development cannot yet be built from public sources alone.

What is trusted. Two groups lie outside the proof. The first is the handwritten code around the generated analyzer: the lexer, the parser with its block-scope resolution

and its rewriting of declarations without initializer, the translation of a request into a configuration, and the presentation, including the placement of arithmetic diagnostics. `render_report` is generated and passes verdicts and diagnostics through unchanged (`render_report_columns`), but no theorem relates its state strings to the semantic values. The second is the foundation: Isabelle's kernel, its code generator with the target mappings (`code_printing`), the trace hook among them, the OCaml and WebAssembly toolchains, Zarith or, in the browser, a JavaScript replacement of it, and the browser. Only regression and property tests cover the handwritten code.

Whether the definitions are adequate. A proof establishes consequences of `pstep` and cannot show that `pstep` is the language a reader has in mind. Section 3.4 argues this for the fragment of C that VIMP models.

What the soundness theorem does not establish. A parse error, an `Invalid_Activation`, `Malformed_Program`, or `No_Answer` answer, a timeout, and a run that never returns yield no Analysed report, and neither does an aborted code equation, one that raises an exception in OCaml where the logic returns a fallback value, for instance when query recursion exceeds its depth bound. The handwritten adapter reports this abort as `No_Answer`, so the `No_Answer` a user sees comes from this adapter. In all these cases the theorem gives no verdict. A hang may come from the solve, from the fixpoint reduction of Int, whose code equation iterates where the HOL function gives up and returns its input (Section 11.3), or from the toolchain.

13. Evaluation

Chapter 10 proved that every report `run_voblint` returns is sound. A machine-checked theorem can still hold for uninteresting reasons. Its premises may never be met together, its obligations may be weaker than they look, or it may exclude only errors that no real analyzer makes. J. Kim [77, §6] validates a mechanized model in two ways. Concrete witnesses show that the premises of its theorems can be met, and direct falsification removes one condition and checks that the theorem relying on it fails. Following this method, we first ask whether the premises can hold together and whether the obligations exclude real errors. We then show on concrete programs what precision the analyzer reaches under the theorem and where it ends, and how the code outside the theorem is tested.

The evidence differs in strength. A *machine-checked* result is an Isabelle theorem and holds, under its premises, for every input its statement ranges over. An *evaluated* result is an Isabelle fact proved by `eval`, which runs the generated code of one concrete solve and therefore trusts the code generator. An *executable* result records the answer of one program at fixed analysis settings, obtained through the parser and renderer outside the theorem (Section 12.5). *Source inspection* and *repository measurement* support statements about the structure or size of the development.

13.1. Non-vacuity thy

Evidence: machine-checked, with the concrete solves evaluated. A theorem whose premises are unsatisfiable holds vacuously. The end-to-end theorem assumes an initial store, a source run, and an `Analysed` answer, which the analyzer gives only if its solve terminates. D. Marmsoler [78, §9] shows with one small executable instance that the locale assumptions of a formal model can be satisfied, and we discharge the premises of the end-to-end theorem in the same way.

The instance is the two-call program of Figure 3.1 under Interval, entry-state contexts, and warrowing, with the source run that returns from `bump(5)` and `bump(4)` and stops before the first check. Evaluation computes an answer in which both checks are `PROVED`, and `nv_source_certified` instantiates `run_voblint_source_sound` with every premise discharged. The run's store thus reaches the graph node v of the stopped run, a reported state describes it, and v is not `DEAD`. Likewise, `nv_check_proved_sound` instantiates `run_voblint_check_sound` at the check `a == 6`. For the reachability claim, a second program sets `x = 1` and guards a check by `x < 0`, which the analyzer marks `DEAD`, and `nv_dead_unreached` derives from this verdict that no store reaches that node, by the lemma behind `run_voblint_check_sound` that a reached check is never `DEAD`. The source runs are proved by rule application, and the compiled graphs and the answers by `eval`. No such instance discharges the premises of the arithmetic theorems `run_voblint_arithmetic_safe` and `run_voblint_arithmetic_refuted` on a concrete program.

These proofs by evaluation extend the *trusted computing base*, the code that must be correct for a proof to be believed, which is normally only Isabelle's inference kernel. The method `eval` compiles a closed proposition to code, runs it, and accepts the result through the code generator's evaluation oracle. In 2025, a defect in normalization by evaluation, which also extends trust beyond the kernel, admitted a proof of `False` [79]. The instances of this section therefore rest on this extended base.

13.2. Do the obligations exclude real errors?

Evidence: machine-checked, and executable against a defect documented upstream. For three coverage conditions the theories give a counterexample to a weakened form. Two of them use a program whose `main` calls `f(1)` and then `f(5)`, with the entry value of `f`'s parameter as the callee's context. RETURN combines the caller's store at a call in context c with a result of the callee in a context c' that the call admits, here $c' = 1$ for `f(1)`. A weakened RETURN reads the callee's result in the caller's context $c = 0$ instead. A claim that covers `f` in every context except 0 leaves its result in context 0 empty, so the weakened obligation holds vacuously after the first call. This claim meets the weakened obligation and all the others (`ret_weak_obligations`) but covers no store after `f(1)`, although the run reaches that point with $a = 1$ (`return_at_caller_context_unsound`). TOTAL requires every call to admit at least one callee context. Without it, a context relation may admit none, so CALL and RETURN hold vacuously. A claim that stops at the first call site then meets the others (`tot_weak_obligations`) but misses the store at which the run resumes after the call (`total_dropped_unsound`, Section 4.4). The third drops the pairing of entry coverage (`entry_pairs_cover`). For `y = p(x)` with `x = 1` and a procedure `p` that returns its formal, an answer may then cover the caller's store with one pair and the entered store with another, and combining each pair's callee result with its own continuation excludes the run's $y = 1$ (`unpaired_entry_cover_unsound`). These three selected conditions do not show that every premise of the development is needed.

A domain obligation, in turn, excludes a defect that occurred in Goblint. In issue 1156 (Chapter 1), Goblint claimed `c % 2 == 1` for $c \in \{-5, -7\}$, although C's truncating remainder gives -1 for both values. Pull request 1161 restricted when the congruence domain returns a constant remainder and added the program as regression test `37-congruence/14-negative.c` [14]. Its annotations require Goblint to leave the first check unknown and mark the second, `c % 2 == -1`, as true but beyond Goblint's current precision. We did not re-run the pre-fix Goblint, and the evidence for its behavior is the upstream report.

In Voblint, the pre-fix remainder cannot be proved sound. The Congruence remainder must meet `congruence_mod_sound`, which requires the truncating concrete remainder `c_mod` (Section 3.4) of any two values to lie in the concretization of the abstract remainder. The pre-fix answer, the constant 1 for $(1 + 2\mathbb{Z}) \bmod 2$, violates this obligation at -5 (`prefix_congruence_mod_unsound`). The theorem holds for every abstract remainder that returns this constant, and Goblint's pre-fix operator returned it according to pull request 1161, so an obligation of this form would have rejected that operator as well. Our fixture transliterates the test into VIMP (Figure 13.1). Congruence alone knows only $c \in 1 + 2\mathbb{Z}$ and decides neither check. Int also knows that c lies in $[-7, -5]$, refutes the first check, and proves the second.

```
fun main() {                                    VIMP
  i32 top =
  __voblint_nondet_int();
  i32 c = -5;
  if (top) {
    c = -7;
  }
  __voblint_check(c % 2 == 1);
  __voblint_check(c % 2 == -1);
}
```

check	Congruence	Int
		REFUTED
c % 2 == 1	UNKNOWN congruence: c=1+2ℤ	int: c=intervals: [-7,-5] congruences:1+2ℤ
		PROVED
c % 2 == -1	UNKNOWN congruence: c=1+2ℤ	int: c=intervals: [-7,-5] congruences:1+2ℤ

Figure 13.1: Goblint's regression test for pull request 1161 in VIMP, and the verdict and state at each check with Congruence alone and with the Int product, both without contexts. Concretely, `c % 2` evaluates to -1 on both executions.

Adding an analysis. *Evidence: source inspection.* The obligations are also local to each analysis. Adding the order analysis took soundness proofs for its local specification and its query (`ord_spec_sound`, `ord_query_sound`), a lattice instance, and the case, field and selection cases of the combined state that Section 11.1 lists. No theory of the framework or of the numeric domains refers to it outside document text. A new numeric domain proves its lattice laws and operations sound once, as an instance of `kind_int_domain`, and the generic construction derives its transfer functions and their soundness.

13.3. Precision on concrete programs

The theorem says nothing about precision, since an analyzer that answers `UNKNOWN` everywhere satisfies it (`unknown_everywhere_sound`). Each example below therefore fixes the concrete behavior first, then shows what one mechanism keeps or loses, and supports a claim about its program only.

Contexts. *Evidence: evaluated and executable.* Figure 8.3 showed which calls each context policy separates. Evaluation shows the gain on two programs. When `main` calls one procedure from two sites, call strings of length 1 prove the two checks after the calls, which the context-insensitive analysis leaves unknown (`call_string_verdicts`). When the two calls reach a procedure only through a second procedure, length 2 proves two checks that length 1 leaves unknown (`call_string_depth_verdicts`). Precision does not improve gradually with the length, however. In the following program `down(100)` recurses to `down(0)`, so `n >= 0` holds at every call:

```
fun down(i32 n) {                               VIMP
  __voblint_check(n >= 0);
  if (n > 0) {
    down(n - 1);
  }
}

fun main() {
  down(100);
}
```

A call string of length k keys an activation by its last k call sites, so it separates the first k recursion depths. With $k = 100$, each of the 101 activations of `down` has its own string, the graph has 102 procedure copies including `main`, and the check is PROVED. With $k =$

99, the two deepest activations share the string of 99 recursive call sites, so this context calls itself and its entry receives the decreasing values of `n`. Widening removes the lower bound there, the graph has 101 copies, and the check is UNKNOWN (claims `cost-down-k99` and `cost-down-k100`).

program	join	join per origin	warrow	warrow per origin	bounded narrowing
two call sites: `p(1); p(2);`, check `x <= 2` in `p`	PROVED `interval: x=[1,2]`	PROVED `interval: x=[1,2]`	UNKNOWN `interval: x=[1,2147483647]`	PROVED `interval: x=[1,2]`	PROVED `interval: x=[1,2]`
call in a loop: `g(x)` for `x` from 0 to 9, check `x < 10` in `g`	PROVED `interval: x=[0,9]`	PROVED `interval: x=[0,9]`	UNKNOWN `interval: x=[0,2147483647]`	PROVED `interval: x=[0,9]`	PROVED `interval: x=[0,9]`
growing recursion: `f(x)` calls `f(x + 1)` from `f(0)`, check `x >= 0`	no answer in 5 s	no answer in 5 s	UNKNOWN `interval: x=[-2147483648,2147483647]`	UNKNOWN `interval: x=[-2147483648,2147483647]`	UNKNOWN `interval: x=[-2147483648,2147483647]`
shrinking recursion: `f(a)` calls `f(9 / (a + 2))` while `a < 4`, from `f(-1)`, check `a <= 9`	PROVED `interval: a=[-1,9]`	PROVED `interval: a=[-1,9]`	PROVED `interval: a=[-1,9]`	PROVED `interval: a=[-9,9]`	PROVED `interval: a=[-9,9]`

Table 13.1: Verdict and state of each row's check under five update rules (Interval, no contexts, bounded narrowing at its default bound 5), as on the project site, where each cell opens its run in the playground. Except in the growing recursion, every check holds in every execution. "No answer in 5 s" means the command-line `--timeout 5` stopped the solve.

Update rules. *Evidence: executable.* Table 13.1 runs the five update rules of Section 9.4 on four programs. In the first row, `p(1)` and `p(2)` publish to the same entry seed. Warrow joins the two contributions before widening, so the second looks like growth and the upper bound goes to the largest 32-bit value. Per-origin warrowing widens each call site's constant contribution separately, so nothing grows. Evaluation proves the same contrast on a system with two writers to one global (`two_writer_slot_warrow_loses_upper_bound`, `two_writer_slot_warrow_per_origin_exact`). On the shrinking recursion, where `a` concretely takes only -1 and 9, every rule proves the check. Warrowing per origin and bounded narrowing, however, widen the recursive call's own contribution and lose precision, reaching the lower bound -9 where the other three rules keep -1. Once widening has lowered `a`, the divisor `a + 2` may be negative, and `9 / (a + 2)` then ranges down to -9. Only the check of the growing recursion fails in some execution, because at 32 bits `x + 1` wraps after 2^{31} nested calls, so `UNKNOWN` loses nothing there. The joining rules raise the bound one value per round and give no answer within the limit, which shows neither divergence nor termination of their solve (Section 9.6). Bounded narrowing reaches the verdicts of per-origin warrowing on all four programs.

Refinement inside Int. *Evidence: executable.* Int combines Interval and Congruence in one value and refines them against each other (Section 11.3). The following program needs both directions of this exchange. Here `q` is $4n + 1$ for some $n \in [0, 20]$, so `r` is 1.

```
fun main() {
  i32 n = __voblint_nondet_int();
  if (0 <= n && n <= 20) {
    i32 q = (12 * n + 3) / 3;
    i32 r = q % 4;
    __voblint_check(r == 1);
    __voblint_check(r > 0);
    __voblint_check(r % 2 == 1);
  }
}
```

VIMP

Interval alone bounds `r` to `[0,3]` and reports `UNKNOWN` at `r == 1`. Congruence alone cannot exclude that `12 * n` wraps under Assume_Top (Section 5.2), keeps `r` at $\top$, and reports `UNKNOWN`. In Int, the interval bounds `q` to $[1, 81]$, so no operation wraps and the congruence keeps $q \in 1 + 4\mathbb{Z}$. Since `q` is also nonnegative, its truncating remainder modulo 4 is exactly 1, and Int proves the check with `1` (claims `int-refine-*`).

Queries between analyses. *Evidence: executable.* Separate analyses refine each other only through queries (Chapter 7), and in the following program they do so in both directions:

```
fun main() {
  i32 c = __voblint_nondet_int();
  i32 x, y;
  if (0 < c) { x = 0; y = 10; } else { x = 20; y = 30; }
  i32 z = (x <= y);
  __voblint_check(z == 1);
}
```

VIMP

Int alone joins the branches, so `z` lies only in `[0,1]` and the check is `UNKNOWN`. The order analysis alone cannot compare a variable with a constant and reports `UNKNOWN`. Together, at both assignments to `y` the order analysis asks Int whether x is at most the assigned constant and records the pair (x, y) on both branches. At `z = (x <= y)` Int asks the query `EvalInt` $(x \leq y)$, the order analysis answers 1, and the check is `PROVED` (claims `coop-mutual-*`). For one program, evaluation also shows that Interval and Order together prove a check that each alone leaves unknown (`coop_verdicts`).

13.4. Where precision ends

The examples above were chosen to show what a mechanism gains. To find what Voblint cannot prove, we use programs chosen independently of it. Monniaux's tutorial on the flavors of abstract interpretation walks through finite lattices, relational domains, widening, and exact solving, and gives for each a small program that defeats it [80]. We pinned each of its programs that VIMP can express as a regression fixture at `i32`. Every check in them holds in every run, so each `UNKNOWN` below is lost precision. No combination of Voblint's analyses, signed-overflow policies (Section 5.2), and refinement modes proves the non-inductive, absolute-value, skipped-increment, and reset-at-equality checks below. For comparison we ran Goblint at revision `5320a6b` on C versions of the same programs with the options that target each loss (Table 13.2). In C, signed overflow is undefined. Goblint's default setting `assume_top` of `sem.int.signed_overflow`, which our runs keep, sends a signed result that may overflow to the top of its type, as Voblint's Assume_Top does, whereas

VIMP's semantics wraps. Our build of Goblint lacks the Apron library that provides its relational domains, so Table 13.2 shows only its non-relational domains.

program	Voblint	Goblint				
		interval	delay 10	Int, delay 3	thresholds	comparisons
rule of signs[a]	UNKNOWN	PROVED	PROVED	PROVED	PROVED	PROVED
not inductive	UNKNOWN	UNKNOWN	UNKNOWN	PROVED	UNKNOWN	UNKNOWN
absolute value	UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN
bounded loop	PROVED	PROVED	PROVED	PROVED	PROVED	PROVED
reset at bound	PROVED	PROVED	PROVED	PROVED	PROVED	PROVED
skipped increment	UNKNOWN	UNKNOWN	PROVED	UNKNOWN	PROVED	UNKNOWN
reset at equality	UNKNOWN	UNKNOWN	PROVED	UNKNOWN	PROVED	UNKNOWN

Table 13.2: The flavors programs [80] (fixtures `28-flavors`) in Voblint with Interval, except Int for the invariant that is not inductive and Int with the order analysis for the absolute value. [a]Voblint runs Sign here, which Goblint lacks, and Voblint's Interval proves the check as Goblint's does. Goblint ran at `i32` (`int`) on the C programs of `goblint-flavors`, and its assertion verdict "will succeed" is shown as PROVED, "is unknown" as UNKNOWN. Goblint columns: Interval; widening delayed 10 times (`solvers.td3.widen_gas`); Interval with Congruence and their refinement, delayed 3 times; Interval with threshold widening on every constant of the program; on the constants of comparisons only. Goblint's outputs are recorded in `results.tsv` beside its programs.

Finite lattices. *Evidence: executable.* The rule of signs says that the sum of two negative numbers is negative [80, slide 8]. At a machine kind the sum of two negative `i32` values may wrap to a nonnegative one, and since a sign bounds no magnitude, Sign cannot exclude the wrap even for constants:

```
fun main() {
  i32 x = -5;
  i32 y = -10;
  i32 z = x + y;
  __voblint_check(z < 0);
}
```

Sign reports UNKNOWN, and Interval keeps $z = -15$ and proves the check (claims `flavors-vanilla-*`). This loss needs no repair, because Sign could prove the check only by assuming that the sum does not overflow, which VIMP's wrapping semantics does not justify.

Invariants that are not inductive. *Evidence: executable.* An abstract fixpoint is at best the least *inductive* invariant the domain can express, and a property may hold without being inductive [80, slide 27]. In the following loop `i` takes the values 4 and 2 at the check, so `i > 0` holds there. However, `i - 2` of a positive value may have any sign, so the loop does not preserve positivity and Sign reaches every sign at the loop head:

```
fun main() {
  i32 i = 6;
  i = i - 2;
  while (i != 0) {
    __voblint_check(i > 0);
    i = i - 2;
  }
}
```

Interval and Congruence together can express an inductive invariant that proves the check, $i \in [0, 4]$ and i even, from which the guard `i != 0` leaves $[2, 4]$. Voblint's Int still reports `UNKNOWN`, because widening at the loop head sends the lower bound to the `i32` limit, where `i - 2` may overflow, and the value becomes the kind's top (claims `flavors-sign-not-inductive*`). Goblint proves the check with refined Interval and Congruence when widening is delayed three times (Table 13.2) or stopped at the threshold 0 of the guard. The threshold columns of Table 13.2 use Interval alone, which leaves the check unknown, so the threshold run with Int is recorded only in `results.tsv`.

Paths and convexity. *Evidence: executable.* The tutorial's coffee example computes an absolute value and checks that it is positive only for a nonzero argument [80, slide 38]:

```
fun main() {
  i32 x;
  i32 y = 0;
  if (-1000 <= x && x <= 2000) {
    if (x < 0) {
      y = -x;
    } else {
      y = x;
    }
    if (y >= 1) {
      __voblint_check(x != 0);
    }
  }
}
```

VIMP

The check needs $y \leq |x|$ after the branch, which holds only as the disjunction of $y = x$ on one path and $y = -x$ on the other. A non-relational domain loses both relations, and a convex relational domain joins the branches into their convex hull [81, §4.4], which keeps $y \geq |x|$ but loses $y \leq |x|$, so polyhedra fail as well [80, slides 42–43]. Voblint's order analysis ends with $x \leq y$, and the check is `UNKNOWN` (claims `flavors-coffee*`, and the fixture `02-coffee_absolute_value.vimp` is the same program without the bounds on `x`). Goblint reports the check unknown in every configuration we ran. Proving it requires keeping the disjunction, as trace partitioning does by keeping the states of the two branches apart [26, §7.1.5], [82, §1.1] and path partitioning does for the last outcome of each conditional [50, §6.3.4]. In Voblint, either would be a local state indexed by the path taken, with its own soundness obligation that every run belongs to some part, and no shipped analysis keeps such a state.

Widening and narrowing. *Evidence: executable.* The tutorial's two widening examples that narrowing repairs are both `PROVED` in Voblint (claims `flavors-fudge` and `flavors-narrowing-reset`). The first, the bounded loop, counts `i` up to 10 and checks `i == 10` after the loop. The second, the reset at bound, is a counter that resets once it reaches 10:

```
fun main() {
  i32 i = 0;
  while (1) {
    i = i + 1;
    if (i >= 10) {
      i = 0;
    }
    __voblint_check(i <= 9);
  }
}
```

VIMP

Narrowing recovers the upper bound 9 [80, slide 59], and the check is proved over `[-2147483646,9]`. The lower bound is lost because widening reaches the `i32` limit, where `i + 1` may overflow, and narrowing cannot bring it back. With Interval alone, Goblint loses the same lower bound for the same reason and leaves a check of `i >= 0` at the same point unknown. A widening delay of 10, threshold widening, or Int with a delay of 3 proves that check (`results.tsv`).

Narrowing foiled. *Evidence: executable.* Two variants of the same counter defeat narrowing. In the first, the skipped increment, a nondeterministic branch may skip the increment [80, slide 60]:

```
fun main() {
  i32 i = 0;
  i32 c;
  while (1) {
    c = __voblint_nondet_int();
    if (c) {
      i = i + 1;
      if (i >= 10) {
        i = 0;
      }
    }
    __voblint_check(i <= 9);
  }
}
```

VIMP

Because the skipping path carries the loop-head value back unchanged, every interval $[0, c]$ with $c \geq 9$ is a fixpoint, the widened value included, and a decreasing iteration cannot descend below a fixpoint [50, §4.7.3, Ex. 4.7], [83]. In the second, the reset at equality, the counter resets on `i == 10` instead of `i >= 10` [80, slide 77]:

```
fun main() {
  i32 i = 0;
  while (1) {
    i = i + 1;
    if (i == 10) {
      i = 0;
    }
    __voblint_check(i <= 9);
  }
}
```

VIMP

The equality test removes no bound from an interval, so nothing stops the widened bound at 9. Both checks are `UNKNOWN` in Voblint (claims `flavors-blackcurrant` and `flavors-reset-eq`).

The literature repairs this loss in several ways. A delayed widening joins for a fixed number of rounds before it widens [26, §7.1.3], [50, §4.7.4]. Goblint's top-down solver can delay widening at each widening point. A delay of 10 proves both checks (Table 13.2) and a delay of 9 does not (`results.tsv`), so the delay must exceed the loop bound and does not repair loops with large or unknown bounds. Threshold widening stops at constants of the program before it jumps to the limit [26, §7.1.2]. Goblint proves both checks with thresholds only when they include every constant of the program, here including the 9 of the check itself, and not with the constants of comparisons or with the check written as `i < 10`. Finally, some methods avoid the decreasing iteration. Policy iteration descends through the fixpoints of simpler systems [84] but need not reach the least one [80, slide 73], and max-strategy iteration reaches the least solution in its domain [85, Lemma 6.5]. Restarting the iteration from a fixpoint projected onto chosen seed points recovers the bound lost to a self-loop [86, §4.1.2], and guided static analysis of a sequence of restricted versions of the program [87] yields $[0, 9]$ on the reset at equality [80, slide 78]. Each repair changes how the solver iterates, so in Voblint each would be a change to the vendored solver and its partial-correctness proof. The tutorial's last flavor, completeness relative to a domain, concerns arrays and maps [80, slide 81], which VIMP lacks.

Voblint therefore proves the checks that non-relational domains with a standard widening and narrowing prove, and it loses those on which these techniques are known to lose precision. Goblint recovers three of them only through options that Voblint lacks, and neither analyzer proves the absolute-value example with the domains we ran. Each missing technique is a separate soundness obligation, on a domain for partitioning and on the solver for the iteration strategies.

13.5. Tests of the unverified parts

Evidence: executable, and repository measurement. For the handwritten code around the generated analyzer (Section 12.5), tests are the only project-local evidence. The theories under `src/` contain 45,206 physical lines in 153 files, including comments and document text but excluding generated theories. The analyzer that runs is the 19,139-line generated OCaml module, surrounded by 5,615 lines of handwritten OCaml under `cli/`, not counting the lexer, parser, and printer generated from a grammar description.

A regression corpus runs the analyzer end to end. Part of it is adapted from Goblint's regression tests, whose origin the fixture header records. The corpus holds 443 VIMP fixtures in 29 groups, each stating its command-line flags and the verdict expected at each check. Cases in `precision/` pin the exact, usually definite, verdict. In `soundness/` the program has executions on both sides of the check, so `UNKNOWN` is the only sound answer. In `known-imprecision/` the concrete result is fixed but the abstraction cannot establish it, and the header names the mechanism that loses the information. Of the remaining 66 fixtures, the 28 in `26-machine-kinds` pin verdicts, diagnostics, and rejections at machine kinds, and most others pin snapshots of the rendered graph or state, rejection of malformed input, or solves expected not to finish within their time limit. The runner matches results by source line, which tests the position bookkeeping of Section 12.3, and distinguishes a missing report row from a `DEAD` one, so a check the compiler dropped cannot pass as proved unreachable. Some fixtures pin the conventions of Table 3.1 at the analyzer's output, for instance `10-arithmetic/precision/07-signed_division_totalization.vimp` (truncating di-

vision and remainder). They check verdicts, not concrete runs, since the development has no executable form of `pstep`. Golden files pin the solver trace (Section 12.4) of three programs step by step and expose changes in the generated solver's execution order.

The parser is trusted to hand the analyzer the program the user wrote, and a parser bug could make the analyzer report verdicts for a different program. Property tests therefore print random syntax trees, built from the constructors Isabelle exports, and require the parser to read back the same tree, and they require it to finish without a crash on mutated programs. A round trip cannot detect a misreading of the grammar that parser and printer share, and no test relates the parsed tree to the program a user meant to write. CI runs the corpus and the property tests on every pull request and push to the main branch.

Together, these results show that the end-to-end theorem applies to a concrete run, that three selected coverage conditions cannot simply be dropped, and that one domain obligation excludes a real defect. Each precision example concerns one program, the tests reach only what the corpus covers, and none of this shows that `pstep` is the intended semantics of VIMP (Section 3.4).

14. Engineering and Tooling

Every theorem statement, Isabelle name, and analyzer result quoted in this thesis is derived from the sources and rechecked on every change (Section 14.1). This chapter describes the repository and the tools that make this hold, and the changes made to the vendored solver. Isabelle's batch build checks every proof, but it does not check that the prose cites existing names, that every theory belongs to a *session*, a group of theories that Isabelle builds and checks as a unit, or that a locale assumption still mentions the constants its author meant.

The need grows with AI assistance. Agents wrote many proofs and a large share of the surrounding code of this development (see the statement on the use of generative AI). An agent produces a plausible change quickly but cannot be trusted to keep the cross-references of a large development consistent, and a reviewer cannot read every line it writes. An agent's change therefore counts as done only when deterministic checks pass, so the consistency of the repository does not depend on the agent's judgment. The repository, the tools, and the upstream changes made for this thesis are engineering contributions, and nothing in them is proved.

14.1. The Voblint repository

One repository holds the Isabelle theories, a submodule reference to the vendored solver, whose fork is private, the exported analyzer with its command-line tool and playground, the regression corpus, and the sources of this thesis. Each release is archived on Zenodo with a citable DOI [49]. The archive, like the repository, refers to the solver as a submodule and does not contain its theories (Section 14.3). The project site hosts the playground, the explainer, and the rendered theories that every Isabelle name in this thesis links to.

A fact written down in two places drifts as the development changes, so the repository keeps a few sources and derives everything else from them (Figure 14.1). One grammar file yields the Isabelle syntax of VIMP, the analyzer's parser, and its printer, so the unverified parser (Section 12.5) and the Isabelle syntax come from one language description. `run_voblint` reaches OCaml through `export_code` (Section 12.1). Because the derived files are checked against their sources, a stale name, statement, or quoted output fails the build instead of reaching the reader.

For the thesis, each Isabelle name goes through a helper that a check resolves against the theories and links to the rendered theories. Theorem statements are lifted from the theories and compared with what the built session proves. Each analyzer result the text quotes is registered as a named run of the analyzer, which the build repeats and compares with the quoted output. A tag such as (claim `intro-answers`) in a caption or sentence names such a run and links to it in the playground. These tags are unrelated to the claims `cover v c` of Chapter 4. A fixture is a program of the regression corpus (Section 13.5). Each VIMP listing's playground link is decoded from the PDF and compared with the listing, and repository figures come from a measurement script.

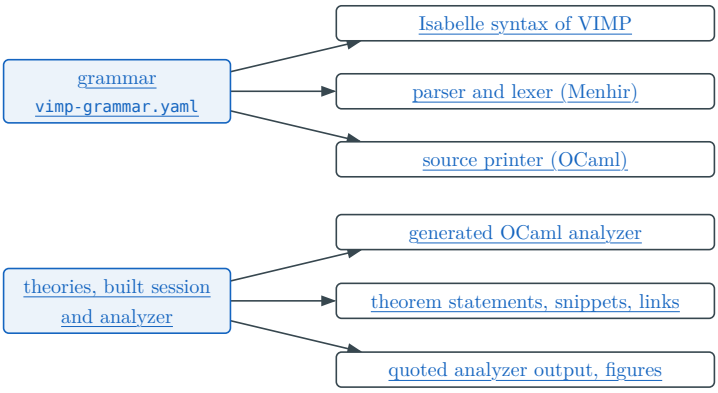


Figure 14.1: The sources of the repository (blue) and the files derived from them, each generated during the build or committed with a check that regenerates it. Each box links to its file.

The checks run in layers ordered by cost (Figure 14.2), from hooks on the staged files to continuous integration, which reruns them on a fresh runner together with the tests of Section 13.5. Only the main branch is deployed, and only after the rendered theories, the formalization PDF, and this thesis build.

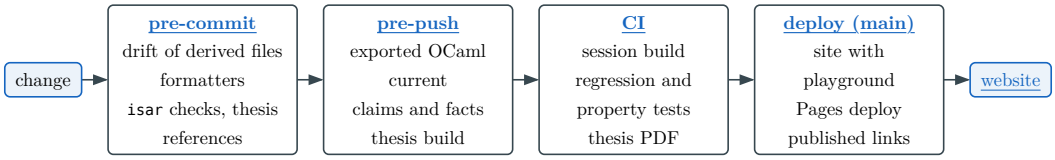


Figure 14.2: The layers of checks between a change and the published site, with representative checks. The published links are checked after deployment. Each box links to its configuration.

14.2. `isar-tools`: source tooling for Isabelle

Isabelle's build does not check project properties such as whether every theory is reached by a session, whether a proof was left unfinished, whether two lemmas state the same fact, or whether a locale still says what it said before a rename. `isar-tools` [88], written for this thesis and published separately, answers such questions from the sources alone. It reads sources, build logs, and rendered HTML as text and never runs Isabelle, so it also reads theories that do not yet check and runs in a pre-commit hook.

Besides a formatter that changes only layout, it reports unfinished and unclosed proofs, theories no session reaches, malformed `ROOT` files, and, on request, unused or duplicated lemmas, unused imports, leftover proof-search commands, and broken links. Its extraction layer lifts the theorem statements, the notation tables, and the anchors into the rendered theories that this thesis and the website use. Among related tools, the Isabelle Linter [89] runs inside Isabelle and lints proof style, and `isabelle-query` [90] also parses the sources without Isabelle and answers queries about entries, call graphs, and `sorry`s. What `isar-tools` adds is the checks that guard a project against silent failures, such as the locale check below, and the extraction that feeds this thesis.

14.2.1. A silent failure in locale headers

Inside the header of a `locale`, an identifier Isabelle does not know is read as a free variable and generalized. An assumption that cites a renamed or misspelled constant therefore still builds, and the batch build reports nothing. The assumption must then hold for every function in place of the intended constant, which is far stronger than intended. In Figure 14.3, choosing the function that maps every argument to the set of all integers forces `step` to be constant. An interpretation for an intended instance then fails late, and theorems proved inside the locale may hold only vacuously, the risk that the non-vacuity instances of Section 13.1 rule out for the main theorem. `isar check locales` flags identifiers in a locale header that appear to be neither parameters nor locally bound and have no matching declaration (Figure 14.3). It approximates Isabelle's inner syntax lexically and is therefore a heuristic.

```
1 definition gamma_sign :: "int ⇒ int set" where                    Isabelle
2   "gamma_sign x = {x}"
3 locale sound_domain =
4   fixes step :: "int ⇒ int"
5   assumes sound: "x ∈ gamma_sgn y ⟹ step x ∈ gamma_sign (step y)"
```

```
Demo.thy:8:27: locale-free-variable: gamma_sgn in the header of locale sound_domain is no parameter, bound
variable, or name used elsewhere; Isabelle generalizes over it as a free variable
```

Figure 14.3: A locale assumption citing a misspelled constant, which Isabelle accepts, and the report of `isar check locales` on this theory, which a test reruns. The listing is an excerpt, lines 4 to 8 of the theory, and the report counts lines in the whole file.

14.2.2. A hidden cost: rechecked theories

A session reuses only the prebuilt images of its parent chain, so a library theory outside that chain is checked again in every session that imports it. The build stays green, but in this repository this once cost 19% of a clean build. `isar stats build` finds such theories in a build log, and continuous integration enforces a budget per library session.

14.3. Changes to the verified solver

Voblint vendors the top-down solver formalization of Tilscher et al. [4] as a submodule. Four changes made for this thesis were proposed upstream.

- The first restores the build under Isabelle2025-2 [91].
- The second moves the well-foundedness assumptions, which say that widening and narrowing stabilize, out of the solver's `widening` and `narrowing` classes into the locales of the termination proofs [92]. Partial correctness needs only the four order laws, so a domain need not provide a stabilizing widening (Section 5.1, Section 9.6).
- The third renames short record fields and constructors, such as `c`, `W` and `N`, that took these names away from downstream theories, and adds one interface theory per solver family [93].
- The fourth adds facts about the solver's own definitions that Voblint had proved on its side [94], among them that a returned run of the executable solver lies in the solver's domain (`solve_dom_of_solve_c`) and that a terminating solve returns a finite set of

stable unknowns (`finite_stabl_solve`), both used by `td_certified_solver` (Section 9.3.3).

The vendored copy applies the second to fourth change, together with its own build fix for Isabelle2025-2, to the upstream revision `af78761` of March 2026. The pull requests port the changes to the current upstream revision, whose build the first restores. At the time of writing, all four await review. The partial-correctness theorem Voblint uses is therefore that of this modified copy, which every build checks but whose changes upstream has not yet reviewed. The fork that holds the vendored copy is private and accessible only by invitation, so the development cannot yet be built from public sources alone. Publishing the solver in the Archive of Formal Proofs, as planned, would make the whole development buildable by anyone.

PART VI

Assessment

15. Related Work

Section 1.2 compared what Voblint's theorem covers with unverified and with mechanized analyzers (Table 1.1, Table 1.2). Here we compare its mechanisms with the closest lines of work. The comparisons rest on a web and OpenAlex search with full-text reads of the closest papers, not on a systematic review.

15.1. Verified analyzers: Verasco and the CompCert line

CompCert and Verasco state their theorems about the tool that users run, and Voblint follows their pattern for a language with recursive procedures and context-sensitive calls.

CompCert is a compiler from Clight, a large subset of C, to PowerPC assembly, programmed and proved correct in Coq, now Rocq [33]. Its theorem is semantic preservation, so safety properties proved of the source hold for the executable. The compiler is extracted to Caml and linked with unverified parts, and Leroy lists what remains trusted, from the semantics of Clight and PowerPC assembly and the CIL-based parser to the assembler, the linker, and Coq's extraction with the Caml toolchain [33, §5]. Voblint's guarantee has the same shape, a theorem about the function the tool runs, code generated from the prover, and an explicit list of trusted components (Section 12.5). The two theorems constrain different things, however. Compiler correctness requires the executable to preserve the behavior of the source. Analyzer soundness requires only that the result over-approximate the behavior, so the theorem also accepts a less precise result, and precision must be assessed separately (Section 13.3).

Verasco is a Coq-verified analyzer for most of ISO C99, excluding recursion and dynamic allocation, that proves the absence of run-time errors [34]. Its abstract interpreter iterates over the structure of the CompCert intermediate language C#minor and combines numerical domains through modular interfaces, and CompCert's semantic preservation extends the guarantee to compiled code. Verasco covers a far larger language, so the comparison concerns design only. Verasco reanalyzes functions at every call site and raises an alarm at a possible recursive call (Table 1.2, [34, §4]), whereas Voblint turns calls into equations over (program point, context) unknowns that a generic solver computes. Its extracted analyzer runs with unbounded fuel, and termination is proved only on paper [95, §§3.1.2, 4.3.1], so its theorem, like Voblint's (Section 9.6), covers only the results it returns.

The earlier value analysis of Blazy et al. [37] does not verify its fixpoint iterator. An untrusted OCaml implementation of Bourdoncle's iteration strategy [62] computes a candidate, and a verified checker accepts it as a post-fixpoint or falls back to top [37, §4]. This leaves the iteration heuristics free at the cost of a test of every result at run time, whereas the proof of Voblint's solver [4] needs no such test but holds for that one algorithm.

Witness validation also checks results instead of the analyzer. A verifier that claims a program correct exports invariants at program locations, a *correctness witness*, from which an independent validator re-establishes the result. The need is real, since in the 2016

software verification competition ten of 13 verifiers claimed correctness for some programs known to contain a bug [96, §1]. The validator Saan et al. [97] build on Goblint's abstract interpreter confirms one result at a time and, unlike the checker of Blazy et al., is itself unverified.

CompCert hides its dataflow solvers behind a common Coq module type whose clients rely only on fixpoint properties of the result [98]. La Spina et al. [99] verify two solvers based on weak topological orderings against it, and CompCert's dataflow analyses then run with these solvers. Both works are intraprocedural. Voblint consumes its solver through a fixpoint guarantee in the same way, for side-effecting, context-indexed systems, where the certificate `part_post_solution` also accounts for contributions to global unknowns.

15.2. Mechanized abstract interpretation

Abstract interpreters have been mechanized in several provers, with different ways of factoring their proofs for reuse. Nipkow and Klein develop abstract interpretation in Isabelle/HOL over annotated commands of the While language IMP [7], [54]. Their syntax-directed, intraprocedural interpreter is parametric in a domain of abstract values, which supplies abstract and inverse operations with their soundness laws, and derives forward evaluation, the backward filtering of guards, and the step of each command generically [7, Sects. 13.5, 13.7]. Voblint's non-relational domains follow this pattern (Section 5.7). Voblint also keeps their separation between a collecting semantics and its abstraction, but its collecting semantics ranges over activation traces of a procedure-aware control-flow graph (Chapter 4), because a return must know which caller resumes.

Michelland et al. [100], Keidel et al. [101], and Darais et al. [102] make soundness proofs reusable by sharing one interpreter between the concrete and the abstract semantics, through monadic handlers, an arrow-based interface, or monad transformers. They reuse the interpreter, and a new analysis replaces the abstract operations it calls. Voblint's analyzer is constraint-based instead, so the iteration belongs to a generic solver, and the proof splits along analysis, context policy, and solver. `activation_collect_dg_sound` composes an analysis with a context policy for any post-solution, Section 9.3.3 attaches the solver, and `run_voblint_source_sound` states the composed result for every analysis configuration (Section 10.4).

Syntax-directed interpreters for While languages without procedures have also been verified in Coq, through an intermediate collecting semantics and with extraction to OCaml [39, §§1, 6], and in F* with SMT automation, running in a browser [42]. An executable or interactive verified analyzer is therefore not new with this thesis.

Isabelle verifications of interprocedural analyses include a conflict analysis for programs with recursive procedures, threads, and monitors, proved sound and, for every program, precise [41], an interprocedural slicer over an abstract control-flow graph with matched calls and returns [103], and Shivers' control-flow analysis in Isabelle/HOLCF, parametric in its contours through a type class that plays the role a context policy plays here [104]. None of these entries claims an extracted analyzer, and none of their semantics represents one activation with its caller chain, the object Voblint's context relation reads. Code extracted from an Isabelle proof has, however, already replaced an unverified component of a larger tool. Buchwald et al. [105] replace the SSA construction of CompCertSSA by OCaml code extracted from their locale-based construction.

15.3. Verified fixpoint solvers

A verified solver guarantees a property of its result for arbitrary equations, and the analyzer must give the equations their meaning. Voblint reuses the solver of Tilscher et al. [4] and supplies that meaning.

The bytecode verifier of Klein and Nipkow is an early Isabelle instance of a verified solver consumed through a specification. It must report no error exactly when a post-fixpoint of the flow functions without errors exists above the initial state, and Kildall's algorithm meets this specification for every semilattice with the ascending chain condition and monotone, bounded flow functions [106, §3, Thm. 1]. Their system is finite and intraprocedural, and the specification also demands completeness, whereas Voblint's solver explores a context-indexed system with side effects on demand and Voblint uses only the soundness direction of its certificate.

Hofmann et al. [63] verify the local generic solver RLD in Coq, and De Vilhena et al. [107] verify in Iris a local generic solver that discovers dependencies dynamically and computes the least fixpoint of a monotone system. Stade et al. [3] prove the top-down solver partially correct in Isabelle/HOL, first for a simpler recursive formulation and then for the optimized solver. Tilscher et al. [4] verify $\mathrm{TD}_{\mathrm{side}}$, which accepts contributions to global unknowns during iteration, against a generic interface for update rules, prove five update rules of Stemmler et al. [45] sound, and refine the solver to executable code. For the solver variant without widening and narrowing, which joins every contribution into its target, they also prove that the solver returns the least partial post-solution, provided it terminates and the equation system satisfies their monotonicity conditions. Tilscher et al. [76] treat the termination of the solver without side effects, and Section 9.6 explains why these results do not apply to Voblint's equations.

The algorithm, the update rules, and their proofs belong to that work, of which Voblint includes a locally refactored copy (Section 14.3). Voblint contributes the meaning of the equations, since its certificate `part_post_solution` implies coverage of the concrete traces and compiler correctness transfers the coverage to source executions. The solver Voblint runs warrows every local unknown at a widening point, whichever update rule merges the global contributions, so the least-solution theorem does not apply. Voblint uses only partial correctness, for all five rules (Section 9.4), and claims no optimality.

15.4. Context sensitivity and trace partitioning

Calling contexts have been defined by call history, by procedure summaries, and as an index of traces. Voblint's contexts index a collecting semantics, as trace partitioning does, and one activation may belong to several of them.

M. Sharir and A. Pnueli [57] introduce the two classical approaches to interprocedural analysis. The functional approach computes input-output relations of procedures, and the call-string approach tags propagated information with an encoded call history, so that a return propagates only information whose history matches. For recursive programs, both may fail to yield an effective solution for problems such as constant propagation [57, §1]. J. Knoop and B. Steffen [108] extend the coincidence theorem to recursive procedures with local variables, with a return function that combines the information before the call with the callee's exit information [108, §4.1], as Voblint's return step combines the caller's

pre-call state with the callee's result (Section 6.1). Reps et al. [109] solve interprocedural problems with finite fact sets and distributive transfer functions precisely, as reachability over interprocedurally valid paths, and state that semantic correctness is an orthogonal issue [109, §2]. Voblint needs neither finite nor distributive domains and proves no precision result.

Trace partitioning abstracts a set of traces by indexing it with control history before abstracting states [82]. Rival and Mauborgne define both partitions and coverings, in which one trace may belong to several indices, present calling-context sensitivity as one such indexing, and note that a relation may replace the function mapping tokens to tokens in a covering [82, Rem. 3.2.4]. Voblint's activation collecting semantics is a covering of this relational kind, since a context policy may admit one activation in several contexts. Voblint mechanizes the indexing over activation traces and proves that the solved, routed result bounds the activation collecting semantics of every admitted context.

Contexts also determine how many unknowns a solve creates. The context lifters of Erhard et al. [68] bound the number of contexts on the fly and guarantee finitely many contexts whenever the solver updates each unknown finitely often [68, Abstract]. Voblint's entry-state policy has no such bound, which is one reason the termination of the solve is not proved (Section 9.6).

15.5. Goblint, local traces, and thread-modular analysis

Voblint follows Goblint's architecture and adapts the local-trace semantics of Schwarz et al.

15.5.1. Architecture

Side-effecting constraint systems let one equation both define a local unknown and publish contributions to global unknowns [43], [44]. Seidl et al. [2] describe how Goblint uses them to decouple a mixed flow-sensitive analysis from the solver, with digests on the analysis side and update rules on the solver side recovering precision. Voblint adopts this split between transfer behavior and the organization of its interaction.

The correspondence with Goblint's implementation is architectural. We compared Voblint with the framework interface of Goblint's constraint generator by source inspection, at revision `5320a6b7`, which the flavors comparison of Section 13.4 also runs. There, a call runs the analysis's `enter`, selects the callee context from the entered state, publishes the callee entry, and combines the callee's exit with the caller. Voblint's equations have the same four steps, with enter and combine supplied by the analysis (Section 6.1) and the context chosen by routing (Section 8.2). The project site's side-by-side comparison (site) lists 19 Goblint constructs, of which 3 are modeled, 13 simplified, and 3 not modeled. The constructs not modeled are Goblint's `sync` hook, threads, and C itself, which Goblint reads through CIL. No row claims that the two compute the same fixpoint.

The two differ in several places. Voblint publishes the callee entry to a global seed that the entry's local unknown reads back, where Goblint writes the callee's entry unknown directly, and under the warrowing rules the seed itself can be widened. Widening thus acts one step later than in Goblint, and we have not compared the precision of the two placements. Call targets are resolved statically. The analyses share one combined state and one query kind without globals, where Goblint's MCP also passes events, spawns, and

per-analysis globals. No agreement rate between the two analyzers' verdicts is reported. Apart from the flavors programs of Section 13.4, whose C versions we ran through Goblint, the corpus has not been run through Goblint, and the adapted fixtures record Goblint's annotations only as prose.

15.5.2. Combining analyses

Goblint combines its analyses at run time in its MCP (Section 7.3), which also answers a query cycle with the top element, caches answers per transfer, and raises `Deadcode` when one analysis finds a point unreachable (`mCP.ml` at the same revision). Voblint's combination follows this design for one query kind and for analyses without globals, and adds a proof obligation per analysis that quantifies over every sound query channel. Unlike the reduced product [17, §5.1.2], [47], which combines domains through a reduction on the product carrier as the Int domain of Section 11.3 does, the combined state keeps separate carriers and exchanges facts only through answers, so no reduction operator has to be defined or proved for a pair of analyses. Verasco, whose channels Voblint's follow (Section 7.2), threads them so that the second domain can query the state the first has just computed [34, §7]. Voblint's answers describe the predecessor state only, as Goblint's do.

15.5.3. Local traces and contexts

Local traces give each thread of a concurrent program a semantics against which thread-modular analyses are proved sound and compared [6], later extended to relational analyses [35], with detailed proofs in [36, Ch. 6]. Voblint adapts the local view to sequential procedure activations. A trace covers one activation and records its suspended caller, so a return reads its caller from the trace instead of choosing one, and a calling context becomes a projection of the trace instead of a component of the state. Context policies can therefore be proved against one fixed concrete semantics. VIMP's activations run one at a time, so no concurrency result of that work transfers. P. Sotin and B. Jeannet [110] also replace a stack semantics by a local one, in which every instruction acts on the top activation record only, prove the two equivalent with respect to reachable stacks [110, §3], and derive a relational interprocedural analysis from the local semantics. Their motive is pointers into the stack, which VIMP lacks. Voblint proves for activation traces only that every graph run is represented by a valid trace (Section 4.2.3).

The digest framework uses abstractions of execution histories to decide which observations may interact [111]. A digest is a total function on local traces that splits the unknowns $[u]$ into $[u, A]$ already in the concrete semantics [36, §2.3]. That local-trace semantics has no procedures yet, although Goblint's analyzer handles procedure calls [36, §8, p. 277]. In Voblint, the context relation `activation_context_rel` (Section 4.3) plays a similar role for sequential activations, with two differences. A digest is a function, so every trace has exactly one digest and the traces split into disjoint groups [36, §2.3, Def. 4, Thm. 4]. One activation may instead carry several contexts, so Voblint's groups may overlap, and together they still contain every collected store (`node_collect_eq_Union_activation_collect`). A digest is also updated at every step of a trace, whereas an activation's contexts are fixed when it is called, so Voblint distinguishes activations only by how they were entered.

15.6. Where Voblint sits

Two mechanized analyzers come closest to Voblint's combination of mixed flow sensitivity and calling contexts. Cachera et al. [38] prove in Coq that a constraint-based analysis of Carmel, an intermediate representation of Java Card bytecode, is sound with respect to a small-step operational semantics. It keeps one flow-insensitive heap and flow-sensitive local variables and operand stacks per program point, is context-insensitive, and solves ordinary inequations with a verified round-robin solver extracted to OCaml [38, §§2, 3.2]. F. Dabrowski and D. Pichardie [40] prove in Coq that a context-sensitive points-to analysis with a flow-insensitive heap, a component of their certified data race analyzer, is sound. They instrument the concrete semantics with contexts through a Coq functor over a module type of contexts, whose call-context function computes a callee's context from the call site, the caller's context, and the receiver, and instantiate it with, among others, k-object sensitivity. By the authors' account the specification is not executable, since the analyses are only sets of constraints that no solver computes [40, §7]. This concrete semantics, which carries contexts and is parameterized by a context policy, is the closest precedent for Voblint's. Voblint's differs in that a relation admits its contexts, the totality condition `call_context_total_on` makes the context-indexed collection exhaustive (`node_collect_eq_Union_activation_collect`), and the indexing is connected to an executable solver.

Voblint thus takes its trust pattern from CompCert and Verasco, its domain interface from Nipkow and Klein, its solver from Tilscher et al., and its architecture from Goblint. It combines these with what we did not find mechanized elsewhere, a context-indexed collecting semantics over activation traces, covered by equations that a verified side-effecting solver computes.

16. Conclusion

The thesis shows that the soundness of a Goblint-style analyzer with contexts, side effects, and a verified solver can be machine-checked from source executions to the verdicts of the exported function. The result holds for a scalar language with recursive procedures and is a partial-correctness result, so it covers every answer the analyzer returns. Graph runs simulate source executions, valid activation traces represent graph runs, and the activation collecting semantics groups the traces by the contexts their policy admits, possibly several per trace. Context-indexed equations with routed calls cover this semantics at every context, provided the analysis and the context policy meet their separate obligations, and the solver enters only through a post-solution certificate. The central contribution is this composition of an activation semantics that stores no calling context, routing to contexts admitted by a relation, analysis-local soundness obligations, and a verified side-effecting solver into one guarantee about the analyzer that the command-line tool and the browser playground run (Chapter 12).

16.1. Discussion

Two design choices shape what the theorem says. A context policy is a relation, so it may place one call in several contexts, and entry-state routing can read contexts off the analysis's own result. The price is the totality obligation TOTAL, which gives every call at least one context. Without it, a call that gets no context meets CALL and RETURN vacuously, and a claim may miss the stores after the call (`total_dropped_unsound`). Policies that compute one context per call from the call site and the entered store, such as call strings, meet it directly (`call_context_total_on_of_fun`). The solver, in turn, enters only through its post-solution certificate. Soundness is therefore independent of the update rule (Section 9.4), but the theorem says nothing about termination or about which post-solution the solver returns (Section 9.3).

How far the results carry beyond VIMP, toward the C that Goblint analyzes, depends on the layer. The certificate mentions no VIMP construct, and the coverage contract refers to VIMP only through the graph, its stores, and three step functions, the edge transfer `edge_step`, the call entry `call_enter`, and the return combination `combine_collect` (Section 4.4). We therefore expect the shape of the coverage contract, the theorem that the contexts exhaust the node collecting semantics, and the composition of Section 8.3 to carry over to a language whose activations do not interfere, that is, in which a callee cannot change its caller's locals. The step functions and every proof that unfolds them would have to be redone.

Machine integers show this split. Because elaboration makes the kind of every operation and conversion explicit (Section 3.2), adding them changed the step functions and the domains but neither routing nor the certificate. Signed overflow wraps in the semantics and division by zero yields a value, where C leaves both undefined, and the analyzer

reports where they may occur instead (`run_voblint_arithmetic_safe`), as Goblint reports overflow [32, §3.3.2]. Treating them as undefined behavior would need an error outcome in the semantics and a meaning for it in the verdicts of Section 10.5.

Pointers into the stack break the assumption that activations do not interfere. A return keeps the caller's locals (Section 6.1), which fails once a callee can write them through a pointer, the case for which P. Sotin and B. Jeannet [110] introduce their local semantics. A heap adds a store component that caller and callee share, so RETURN, the entry pairs, and the one value per variable in the domains (Section 3.1) would all change. Threads interleave activations, which `valid_activation_trace` cannot express, and the local traces that give threads a semantics [6] have no procedures [36, §8, p. 277]. The interface would also need synchronization. A C front end such as CIL would join the parser in the trust boundary unless it is verified.

16.2. Limitations

The theorem is a partial-correctness result. Termination is proved neither for the solve nor for the reduction of Int, and the regression corpus contains programs whose solve gives no answer within its time limit (Section 9.6, Section 11.3). The theorem concerns VIMP, a scalar language without pointers, heap, or memory model, whose integers have fixed widths and wrap, whose division by zero is defined, and whose locals declared without an initializer hold some value of their kind. A call additionally resets the callee's locals to 0, which no program the parser accepts observes, because every local is declared and written before it is read. A verdict about a VIMP program therefore does not transfer to a C program with the same text. The arithmetic diagnostics cover the operations whose result C leaves undefined or implementation-defined, and a point without diagnostics is still not shown free of every undefined behavior of C (Section 3.4). We argue, but do not prove, that elaboration and the core semantics, `pstep` among them, model the intended language, and the parser's resolution of block scopes and the kind declarations it builds are trusted. The simulation from source runs to graph runs and the representation of graph runs by valid traces are proved in the forward direction only (Section 3.7, Section 4.2.3). Within the analyzer, only the combined state as a whole publishes to analysis globals, since its components cannot (Section 7.4). Even when program globals are flow-insensitive, the order analysis therefore keeps all its relations in the local state, and one update rule serves every global unknown of a run (Section 6.4, Section 11.4, Section 9.4).

The delivered tools rest on the trusted components of Section 12.5, and every theorem proved by evaluation trusts the code generator. No test runs VIMP programs concretely, since the development has no executable form of `pstep`. No systematic time or memory measurement is reported. The executable keeps proof-oriented data representations without a data refinement to efficient sets and maps (Section 12.2). Refining the solver's state would require the proof of the vendored solver to be redone, whereas the rest of the chain uses the solver only through its certificate (Section 9.3) and would be unaffected. The vendored solver is itself a modified copy whose changes await upstream review, and its repository is private, so the development cannot yet be built from public sources alone (Section 14.3).

No general precision, optimality, or completeness theorem is proved, where completeness would mean that every check that holds in all executions is reported as proved. Each precision example concerns one program with fixed analysis settings (Section 13.3). The

regression corpus is small and was written or adapted for this work, and its expected verdicts were set by hand (Section 13.5). Only the C versions of the flavors programs were also run through Goblint (Section 13.4), and the rest of the corpus was not checked against an independent analyzer. The Goblint defect of Section 13.2 was not re-run. The correspondence with Goblint is architectural. No theorem transfers to its OCaml implementation, and no agreement rate between the two analyzers is measured (Section 15.5). The comparison with prior work rests on a targeted search rather than a systematic review (Chapter 15).

16.3. Outlook

16.3.1. Extending Voblint

A richer source language, up to a subset of C, needs the new obligations that Section 16.1 names for signed overflow, pointers, a heap, and threads, and a semantics and a preservation argument per construct at every layer. A C front end would extend the elaboration into core VIMP (Section 3.2) to C's types and expressions, and the meaning of the result would be argued as for VIMP.

That argument is checked against example programs only (Section 3.4). A fuel-bounded evaluator whose runs are proved to be `pstep` runs would make this check systematic. A run that violates a check reported as `PROVED` would then point to a fault in the trusted base, and comparing the evaluator's runs with a C compiler on the common subset would test the adequacy argument itself.

Richer analyses need their own soundness proofs. A relational domain with stronger transfer functions than the order analysis could also answer the numeric analyses at branches (Section 11.4). Further query kinds would each need a truth relation, and analyses with their own globals would need the combination to keep these globals apart, as Goblint's MCP does (Section 7.4). Every shipped analysis answers a single entry pair per call, although the interface admits several (Section 6.1), so path-sensitive analyses that split a call into cases remain future work. Since the contexts of an activation depend only on how it was entered, generalizing the context relation of Section 4.3 to other abstractions of activation traces, as digests are for local traces (Section 15.5), would allow path- or history-sensitive unknowns, each with its own admissibility conditions.

A new update rule that meets the solver's interface changes little in Voblint, since only `update_rule_update_global_of` splits on the rule. Other extensions change the vendored solver itself, and with it its partial-correctness proof. Goblint publishes into the callee's entry unknown directly, and its per-origin narrowing rule for globals joins the writes of one evaluation per target. A vendored solver that did both would let callers publish into callee entries directly and make activation seeds and buffering unnecessary (Section 9.1). Of the losses in Section 13.4, those not repaired by a partitioned local state come from how the solver iterates. Goblint delays widening per widening point (`solvers.td3.widen_gas`) and can widen to thresholds (`ana.int.interval_threshold_widening`). A delay needs a counter per widening point in the vendored solver, and thresholds need the widening to know the constants of the program, which a parameterless `warrowing` operation cannot. Either change also touches the solver's traced executable form (Section 12.4).

A termination theorem would guarantee an answer for every program. The total-correctness result of Tilscher et al. for the top-down solver without side effects assumes finitely many unknowns, monotonic right-hand sides with monotonic dependencies, and a precise widening [76, Cor. 1] that stabilizes every widening sequence by definition [76, Def. 2]. The vendored class `warrowing` states no stabilization law (Section 2.5), so it does not bound how long the values of even finitely many unknowns keep increasing (Section 9.6).

16.3.2. Agent-assisted development

Each new analysis only has to meet its own obligations, which mention neither the other analyses nor contexts, the solver, or the equations (Section 7.4). This suits development with AI agents (see the statement on the use of generative AI). An agent may propose definitions and proofs, but Isabelle checks every proof, and the batch build and the drift checks of Chapter 14 reject anything that does not check. A build does not detect a theorem that holds only vacuously, however. An agent once added a premise to the end-to-end theorem that no program satisfied, and an audit of the theorem statements, not the build, exposed it. The counterexample theorems for weakened obligations (Section 13.2) therefore act as negative tests, and the non-vacuity instances of Section 13.1 as positive tests that the premises of `run_voblint_source_sound` and `run_voblint_check_sound` can be met. The arithmetic theorems have no such instance. Human review can then concentrate on what no proof settles, namely whether the base definitions and the obligations state the intended meaning (Section 3.4) and whether the trusted base of Section 12.5 is acceptable. The same division applies to extensions that change definitions at several layers, such as a construct from C or a Goblint feature like path-sensitive entry alternatives. Isabelle reports every proof the change breaks, so an agent can work through the failures while a reviewer checks the new definitions, above all the extended source semantics. Whether this division carries a larger extension remains to be tried.

16.3.3. Voblint and Goblint

Within the architectural correspondence of Section 15.5, Voblint can serve as an executable specification of the architecture with proved soundness obligations. In one direction, a Goblint feature gives the design that a Voblint extension follows, as the query mechanism did for Chapter 7. Goblint's regression programs can be adapted as fixtures, as part of the corpus already is (Section 13.5), and a documented Goblint defect can be replayed as a counterexample theorem, as the congruence remainder was (Section 13.2). The same obligations could be consulted before a change to Goblint lands, by transliterating a proposed domain operation by hand and checking it against the obligation its Voblint counterpart meets, such as `congruence_mod_sound` for the remainder. Operations on pointers or threads have no such counterpart.

In the other direction, a verified component exported as OCaml could replace its unverified counterpart in Goblint, as extracted code replaced the SSA construction of CompCertSSA [105]. Besides the export, that needs the data refinement of Section 16.2, a correspondence between the component's interface and Goblint's, and soundness for C's integers, since the component's proofs assume VIMP's wrapping integers and defined division by zero (Section 3.4).

Voblint verifies neither Goblint nor an analyzer for C. Its remaining gaps are a richer language, termination, efficient representations, and the handwritten code around the analyzer. The certificate and the shape of the coverage contract would carry over to a language whose activations do not interfere (Section 16.1). Pointers into the stack and threads break that assumption, termination needs a new theorem about the solver, and efficient representations need the solver's proof redone. Within these limits, Voblint connects, for a Goblint-style architecture, source executions to the verdicts of executable code by one machine-checked chain.

Bibliography

[1] V. Vojdani, K. Apinis, V. Rõtov, H. Seidl, V. Vene, and R. Vogler, "Static Race Detection for Device Drivers: The Goblint Approach," in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, ACM, 2016, pp. 391–402. doi: 10.1145/2970276.2970337.

[2] H. Seidl, V. Vojdani, J. Erhard, and M. Schwarz, "Mixed Flow-Sensitive Static Analysis: Engineering Modularity," in *FM 2026: Formal Methods*, 2026, pp. 446–470. doi: 10.1007/978-3-032-26220-2_22.

[3] Y. Stade, S. Tilscher, and H. Seidl, "The Top-Down Solver Verified: Building Confidence in Static Analyzers," in *Computer Aided Verification*, 2024, pp. 303–324. doi: 10.1007/978-3-031-65627-9_15.

[4] S. Tilscher, A. Graß, and H. Seidl, "Verifying a Solver for Mixed Flow-Sensitive Analyses," in *NASA Formal Methods (NFM 2026)*, in LNCS. Springer, 2026, pp. 48–69. doi: 10.1007/978-3-032-28079-4_3.

[5] G. C. Necula, S. McPeak, S. P. Rahul, and W. Weimer, "CIL: Intermediate Language and Tools for Analysis and Transformation of C Programs," in *Compiler Construction (CC 2002)*, in LNCS, vol. 2304. Springer, 2002, pp. 213–228. doi: 10.1007/3-540-45937-5_16.

[6] M. Schwarz, S. Saan, H. Seidl, K. Apinis, J. Erhard, and V. Vojdani, "Improving Thread-Modular Abstract Interpretation," in *Static Analysis*, in LNCS, vol. 12913. 2021, pp. 359–383. doi: 10.1007/978-3-030-88806-0_18.

[7] T. Nipkow and G. Klein, *Concrete Semantics with Isabelle/HOL*. Springer, 2014. doi: 10.1007/978-3-319-10542-0.

[8] ISO/IEC JTC 1/SC 22/WG 14, "ISO/IEC 9899:2011, Information technology – Programming languages – C." Accessed: Sep. 21, 2026. [Online]. Available: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf

[9] Leiden Declaration Working Group, "Leiden Declaration on Artificial Intelligence and Mathematics." [Online]. Available: https://leidendeclaration.ai/

[10] AutoCorrode contributors, "AutoCorrode: Verification Infrastructure for the Isabelle/HOL Interactive Proof Assistant." Accessed: Sep. 21, 2026. [Online]. Available: https://github.com/awslabs/AutoCorrode

[11] K. Kappelmann, M. Schäffeler, L. Stevens, M. Abdulaziz, A. Popescu, and D. Traytel, "Just Type It in Isabelle! AI Agents Drafting, Mechanizing, and Generalizing from Human Hints." [Online]. Available: https://arxiv.org/abs/2604.15713v3

[12] D. Bryant, J. J. H. y Munive, C. Kaliszyk, and J. Urban, "Munkres' General Topology Autoformalized in Isabelle/HOL." Accessed: Sep. 22, 2026. [Online]. Available: https://arxiv.org/abs/2604.07455v1

[13] M. Schwarz, "Improve Precision for %." Accessed: Sep. 22, 2026. [Online]. Available: https://github.com/goblint/analyzer/issues/1156

[14] M. Schwarz, “Congruences: Make % Sound by Restricting Cases Where We Return a Constant.” Accessed: Sep. 21, 2026. [Online]. Available: https://github.com/goblint/analyzer/pull/1161

[15] L. C. Paulson, T. Nipkow, and M. Wenzel, “From LCF to Isabelle/HOL,” *Formal Aspects of Computing*, vol. 31, no. 6, pp. 675–698, 2019, doi: 10.1007/s00165-019-00492-1.

[16] G. Klein *et al.*, “seL4: Formal Verification of an OS Kernel,” in *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP 2009)*, ACM, 2009, pp. 207–220. doi: 10.1145/1629575.1629596.

[17] X. Rival and K. Yi, *Introduction to Static Analysis: An Abstract Interpretation Perspective*. MIT Press, 2020. [Online]. Available: https://mitpress.mit.edu/9780262043410/introduction-to-static-analysis/

[18] H. G. Rice, “Classes of Recursively Enumerable Sets and Their Decision Problems,” *Transactions of the American Mathematical Society*, vol. 74, no. 2, pp. 358–366, 1953, doi: 10.1090/S0002-9947-1953-0053041-6.

[19] B. Livshits *et al.*, “In Defense of Soundiness: A Manifesto,” *Communications of the ACM*, vol. 58, no. 2, pp. 44–46, 2015, doi: 10.1145/2644805.

[20] P. Cousot *et al.*, “Combination of Abstractions in the ASTRÉE Static Analyzer,” in *Advances in Computer Science – ASIAN 2006. Secure Software and Related Issues*, in LNCS, vol. 4435. Springer, 2007, pp. 272–300. doi: 10.1007/978-3-540-77505-8_23.

[21] D. Bühler, P. Cuoq, and B. Yakobowski, “The Eva Plug-in, for Frama-C 33.0 (Arsenic).” Accessed: Sep. 29, 2026. [Online]. Available: https://www.frama-c.com/download/frama-c-eva-manual.pdf

[22] M. Journault, A. Miné, R. Monat, and A. Ouadjaout, “Combinations of Reusable Abstract Domains for a Multilingual Static Analyzer,” in *Verified Software. Theories, Tools, and Experiments (VSTTE 2019)*, in LNCS. Springer, 2020, pp. 1–18. doi: 10.1007/978-3-030-41600-3_1.

[23] G. Brat, J. A. Navas, N. Shi, and A. Venet, “IKOS: A Framework for Static Analysis Based on Abstract Interpretation,” in *Software Engineering and Formal Methods (SEFM 2014)*, in LNCS. Springer, 2014, pp. 271–277. doi: 10.1007/978-3-319-10431-7_20.

[24] Q. L. Le, A. Raad, J. Villard, J. Berdine, D. Dreyer, and P. W. O'Hearn, “Finding Real Bugs in Big Programs with Incorrectness Logic,” *Proceedings of the ACM on Programming Languages*, vol. 6, no. OOPSLA1, p. 81:1–81:27, 2022, doi: 10.1145/3527325.

[25] S. Saan *et al.*, “Goblint: Autotuning Thread-Modular Abstract Interpretation (Competition Contribution),” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2023)*, in LNCS, vol. 13994. Springer, 2023, pp. 547–552. doi: 10.1007/978-3-031-30820-8_34.

[26] B. Blanchet *et al.*, “A Static Analyzer for Large Safety-Critical Software,” in *Proceedings of the ACM SIGPLAN 2003 Conference on Programming Language Design and Implementation (PLDI)*, ACM, 2003, pp. 196–207. doi: 10.1145/781131.781153.

[27] Mopsa developers, “Mopsa User Manual.” Accessed: Sep. 29, 2026. [Online]. Available: https://mopsa.gitlab.io/mopsa-analyzer/user-manual/

[28] IKOS developers, "IKOS Analyzer README." Accessed: Sep. 29, 2026. [Online]. Available: https://github.com/NASA-SW-VnV/ikos/blob/ac7f7c1738976cabc58c6a53413df6e458995c38/analyzer/README.md

[29] P. Cuoq *et al.*, "Testing Static Analyzers with Randomly Generated Programs," in *NASA Formal Methods (NFM 2012)*, in LNCS. Springer, 2012, pp. 120–125. doi: 10.1007/978-3-642-28891-3_12.

[30] C. Klinger, M. Christakis, and V. Wüstholz, "Differentially Testing Soundness and Precision of Program Analyzers," in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, ACM, 2019, pp. 239–250. doi: 10.1145/3293882.3330553.

[31] D. Kaindlstorfer, A. Isychev, V. Wüstholz, and M. Christakis, "Interrogation Testing of Program Analyzers for Soundness and Precision Issues," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, ACM, 2024, pp. 319–330. doi: 10.1145/3691620.3695034.

[32] S. Saan, "Correctness Witnesses for Thread-Modular Program Analysis," Doctoral dissertation, 2026. [Online]. Available: https://dspace.ut.ee/server/api/core/bitstreams/00ec3799-947c-48eb-99e3-f43c9646bbcb/content

[33] X. Leroy, "Formal Verification of a Realistic Compiler," *Communications of the ACM*, vol. 52, no. 7, pp. 107–115, 2009, doi: 10.1145/1538788.1538814.

[34] J.-H. Jourdan, V. Laporte, S. Blazy, X. Leroy, and D. Pichardie, "A Formally-Verified C Static Analyzer," in *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, ACM, 2015, pp. 247–259. doi: 10.1145/2676726.2676966.

[35] M. Schwarz, S. Saan, H. Seidl, J. Erhard, and V. Vojdani, "Clustered Relational Thread-Modular Abstract Interpretation with Local Traces," in *Programming Languages and Systems (ESOP 2023)*, in LNCS, vol. 13990. 2023, pp. 28–58. doi: 10.1007/978-3-031-30044-8_2.

[36] M. B. Schwarz, "Thread-Modular Abstract Interpretation: The Local Perspective," Doctoral dissertation, Technische Universität München, 2025. [Online]. Available: https://mediatum.ub.tum.de/doc/1765098/n8cop1l4luxwzq7hmzpv376a6.tmai-local-perspective-schwarz.pdf

[37] S. Blazy, V. Laporte, A. Maroneze, and D. Pichardie, "Formal Verification of a C Value Analysis Based on Abstract Interpretation," in *Static Analysis*, 2013, pp. 324–344. doi: 10.1007/978-3-642-38856-9_18.

[38] D. Cachera, T. Jensen, D. Pichardie, and V. Rusu, "Extracting a Data Flow Analyser in Constructive Logic," *Theoretical Computer Science*, vol. 342, no. 1, pp. 56–78, 2005, doi: 10.1016/j.tcs.2005.06.004.

[39] D. Cachera and D. Pichardie, "A Certified Denotational Abstract Interpreter," in *Interactive Theorem Proving (ITP 2010)*, in LNCS. Springer, 2010, pp. 9–24. doi: 10.1007/978-3-642-14052-5_3.

[40] F. Dabrowski and D. Pichardie, "A Certified Data Race Analysis for a Java-like Language," in *Theorem Proving in Higher Order Logics (TPHOLs 2009)*, in LNCS. 2009, pp. 212–227. doi: 10.1007/978-3-642-03359-9_16.

[41] P. Lammich and M. Müller-Olm, "Formalization of Conflict Analysis of Programs with Procedures, Thread Creation, and Monitors," *Archive of Formal Proofs*, Dec.

2007, Accessed: Sep. 22, 2026. [Online]. Available: https://isa-afp.org/entries/Program-Conflict-Analysis.html

[42] L. Franceschino, D. Pichardie, and J.-P. Talpin, "Verified Functional Programming of an Abstract Interpreter," in *Static Analysis (SAS 2021)*, in LNCS. Springer, 2021, pp. 124–143. doi: 10.1007/978-3-030-88806-0_6.

[43] H. Seidl, V. Vene, and M. Müller-Olm, "Global Invariants for Analysing Multi-threaded Applications," *Proceedings of the Estonian Academy of Sciences. Physics. Mathematics*, vol. 52, no. 4, pp. 413–436, 2003, doi: 10.3176/phys.math.2003.4.05.

[44] K. Apinis, H. Seidl, and V. Vojdani, "Side-Effecting Constraint Systems: A Swiss Army Knife for Program Analysis," in *Programming Languages and Systems (APLAS 2012)*, in LNCS, vol. 7705. Springer, 2012, pp. 157–172. doi: 10.1007/978-3-642-35182-2_12.

[45] F. Stemmler, M. Schwarz, J. Erhard, S. Tilscher, and H. Seidl, "Taking Out the Toxic Trash: Recovering Precision in Mixed Flow-Sensitive Static Analyses," *Proceedings of the ACM on Programming Languages*, vol. 9, no. PLDI, pp. 1167–1193, 2025, doi: 10.1145/3729297.

[46] P. Cousot and R. Cousot, "Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints," in *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, ACM, 1977, pp. 238–252. doi: 10.1145/512950.512973.

[47] P. Cousot and R. Cousot, "Systematic Design of Program Analysis Frameworks," in *Proceedings of the 6th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, ACM, 1979, pp. 269–282. doi: 10.1145/567752.567778.

[48] F. Haftmann and T. Nipkow, "Code Generation via Higher-Order Rewrite Systems," in *Functional and Logic Programming (FLOPS 2010)*, in LNCS, vol. 6009. Springer, 2010, pp. 103–117. doi: 10.1007/978-3-642-12251-4_9.

[49] M. Lerchner, "Voblint: A Verified Goblint-Style Static Analysis Pipeline in Isabelle/HOL." Zenodo, 2026. doi: 10.5281/zenodo.23260696.

[50] A. Miné, "Tutorial on Static Inference of Numeric Invariants by Abstract Interpretation," *Foundations and Trends in Programming Languages*, vol. 4, no. 3–4, pp. 120–372, 2017, doi: 10.1561/2500000034.

[51] A. Tarski, "A Lattice-Theoretical Fixpoint Theorem and Its Applications," *Pacific Journal of Mathematics*, vol. 5, no. 2, pp. 285–309, 1955, doi: 10.2140/pjm.1955.5.285.

[52] H. Seidl, R. Wilhelm, and S. Hack, *Compiler Design: Analysis and Transformation*. Berlin, Heidelberg: Springer, 2012. doi: 10.1007/978-3-642-17548-0.

[53] P. Cousot and R. Cousot, "Comparing the Galois Connection and Widening/Narrowing Approaches to Abstract Interpretation," in *Programming Language Implementation and Logic Programming (PLILP 1992)*, in LNCS, vol. 631. Springer, 1992, pp. 269–295. doi: 10.1007/3-540-55844-6_142.

[54] T. Nipkow, "Abstract Interpretation of Annotated Commands," in *Interactive Theorem Proving (ITP 2012)*, in LNCS. Springer, 2012, pp. 116–132. doi: 10.1007/978-3-642-32347-8_9.

[55] G. A. Kildall, “A Unified Approach to Global Program Optimization,” in *Proceedings of the 1st Annual ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, ACM, 1973, pp. 194–206. doi: 10.1145/512927.512945.

[56] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006.

[57] M. Sharir and A. Pnueli, “Two Approaches to Interprocedural Data Flow Analysis,” in *Program Flow Analysis: Theory and Applications*, S. S. Muchnick and N. D. Jones, Eds., Prentice-Hall, 1981, ch. 7.

[58] K. Apinis, H. Seidl, and V. Vojdani, “How to Combine Widening and Narrowing for Non-Monotonic Systems of Equations,” in *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, ACM, 2013, pp. 377–386. doi: 10.1145/2491956.2462190.

[59] K. Apinis, H. Seidl, and V. Vojdani, “Enhancing Top-Down Solving with Widening and Narrowing,” in *Semantics, Logics, and Calculi*, vol. 9560, in LNCS, vol. 9560., Springer, 2016, pp. 272–288. doi: 10.1007/978-3-319-27810-0_14.

[60] A. Graß, “Towards the Verification of Top-Down Solvers,” Master's thesis, Technische Universität München, 2024. Accessed: Sep. 22, 2026. [Online]. Available: https://www.tcs.ifi.lmu.de/staff/alexandra-grass/master-thesis-alexandra-grass.pdf

[61] H. Seidl and R. Vogler, “Three improvements to the top-down solver,” *Mathematical Structures in Computer Science*, vol. 31, no. 9, pp. 1090–1134, 2021, doi: 10.1017/S0960129521000499.

[62] F. Bourdoncle, “Efficient Chaotic Iteration Strategies with Widenings,” in *Formal Methods in Programming and Their Applications*, in LNCS, vol. 735. Springer, 1993, pp. 128–141. doi: 10.1007/BFb0039704.

[63] M. Hofmann, A. Karbyshev, and H. Seidl, “Verifying a Local Generic Solver in Coq,” in *Static Analysis (SAS 2010)*, in LNCS. Springer, 2010, pp. 340–355. doi: 10.1007/978-3-642-15769-1_21.

[64] P. Lammich and S. Wimmer, “IMP2 – Simple Program Verification in Isabelle/HOL,” *Archive of Formal Proofs*, Jan. 2019, [Online]. Available: https://isa-afp.org/entries/IMP2.html

[65] N. Schirmer, “A Sequential Imperative Programming Language Syntax, Semantics, Hoare Logics and Verification Environment,” *Archive of Formal Proofs*, Feb. 2008, Accessed: Sep. 22, 2026. [Online]. Available: https://isa-afp.org/entries/Simpl.html

[66] G. D. Plotkin, “A Structural Approach to Operational Semantics,” *Journal of Logic and Algebraic Programming*, vol. 60–61, pp. 17–139, 2004, doi: 10.1016/j.jlap.2004.05.001.

[67] M. Brecknell *et al.*, “AutoCorres2,” *Archive of Formal Proofs*, Apr. 2024, Accessed: Oct. 04, 2026. [Online]. Available: https://isa-afp.org/entries/AutoCorres2.html

[68] J. Erhard, J. F. Schinabeck, M. Schwarz, and H. Seidl, “Context Gas and friends: taming context-sensitivity on the fly,” *International Journal on Software Tools for Technology Transfer*, vol. 27, no. 2, pp. 289–307, 2025, doi: 10.1007/s10009-025-00803-3.

[69] C. Ballarin, “Locales: A Module System for Mathematical Theories,” *Journal of Automated Reasoning*, vol. 52, no. 2, pp. 123–153, 2014, doi: 10.1007/s10817-013-9284-7.

[70] F. Haftmann and M. Wenzel, “Constructive Type Classes in Isabelle,” in *Types for Proofs and Programs (TYPES 2006)*, in LNCS. Springer, 2007, pp. 160–174. doi: 10.1007/978-3-540-74464-1_11.

[71] P. Granger, “Static Analysis of Arithmetical Congruences,” *International Journal of Computer Mathematics*, vol. 30, no. 3–4, pp. 165–190, 1989, doi: 10.1080/00207168908803778.

[72] P. Cousot, “The Calculational Design of a Generic Abstract Interpreter,” in *Calculational System Design*, vol. 173, M. Broy and R. Steinbrüggen, Eds., in NATO ASI Series F, vol. 173., IOS Press, 1999, pp. 421–505.

[73] K. Apinis, “Frameworks for Analyzing Multi-threaded C,” Doctoral dissertation, 2014. [Online]. Available: https://nbn-resolving.org/urn:nbn:de:bvb:91-diss-20140606-1189191-0-9

[74] B. Huffman and O. Kunčar, “Lifting and Transfer: A Modular Design for Quotients in Isabelle/HOL,” in *Certified Programs and Proofs (CPP 2013)*, in LNCS. Springer, 2013, pp. 131–146. doi: 10.1007/978-3-319-03545-1_9.

[75] S. Schulze Frielinghaus, H. Seidl, and R. Vogler, “Enforcing Termination of Interprocedural Analysis,” *Formal Methods in System Design*, vol. 53, no. 2, pp. 313–338, 2018, doi: 10.1007/s10703-017-0288-5.

[76] S. Tilscher, A. Graß, H. Seidl, and Y. Stade, “Proving Total Correctness of Top-Down Solvers with Widening and Narrowing,” *Journal of Automated Reasoning*, vol. 70, no. 2, p. 15, 2026, doi: 10.1007/s10817-026-09756-x.

[77] J. Kim, “Mechanizing Typed Regulatory Actions for Security Tokens: Semantics, Falsification, and Bounded EVM Evidence.” [Online]. Available: https://arxiv.org/abs/2608.29134v3

[78] D. Marmsoler, “Isabelle/STARK: A Formalization of zk-STARK in Isabelle/HOL.” [Online]. Available: https://arxiv.org/abs/2608.01965v1

[79] L. C. Paulson, “Broken Proofs and Broken Provers.” Accessed: Sep. 22, 2026. [Online]. Available: https://lawrencecpaulson.github.io/2026/01/15/Broken_proofs.html

[80] D. Monniaux, “Flavors of Abstract Interpretation.” [Online]. Available: https://hal.science/hal-04750222

[81] P. Cousot and N. Halbwachs, “Automatic Discovery of Linear Restraints Among Variables of a Program,” in *Proceedings of the 5th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, ACM, 1978, pp. 84–96. doi: 10.1145/512760.512770.

[82] X. Rival and L. Mauborgne, “The Trace Partitioning Abstract Domain,” *ACM Transactions on Programming Languages and Systems*, vol. 29, no. 5, 2007, doi: 10.1145/1275497.1275501.

[83] N. Halbwachs and J. Henry, “When the Decreasing Sequence Fails,” in *Static Analysis (SAS 2012)*, in LNCS, vol. 7460. Springer, 2012, pp. 198–213. doi: 10.1007/978-3-642-33125-1_15.

[84] A. Costan, S. Gaubert, É. Goubault, M. Martel, and S. Putot, “A Policy Iteration Algorithm for Computing Fixed Points in Static Analysis of Programs,” in *Computer Aided Verification (CAV 2005)*, in LNCS, vol. 3576. Springer, 2005, pp. 462–475. doi: 10.1007/11513988_46.

[85] T. M. Gawlitza and D. Monniaux, “Invariant Generation through Strategy Iteration in Succinctly Represented Control Flow Graphs,” *Logical Methods in Computer Science*, vol. 8, no. 3, 2012, doi: 10.2168/LMCS-8(3:29)2012.

[86] J. Henry, “Static Analysis by Abstract Interpretation and Decision Procedures,” Doctoral dissertation, 2014. [Online]. Available: https://www-verimag.imag.fr/PUBLIS/uploads/ipogm3582.pdf

[87] D. Gopan and T. Reps, “Guided Static Analysis,” in *Static Analysis (SAS 2007)*, in LNCS, vol. 4634. Springer, 2007, pp. 349–365. doi: 10.1007/978-3-540-74061-2_22.

[88] M. Lerchner, “isar-tools: Source Tooling for Isabelle/Isar Projects.” Accessed: Sep. 27, 2026. [Online]. Available: https://github.com/ManuelLerchner/isar-tools

[89] Y. Megdiche and others, “Isabelle Linter.” Accessed: Oct. 08, 2026. [Online]. Available: https://github.com/isabelle-prover/isabelle-linter

[90] A. Salamon, “isabelle-query: Querying Isabelle/Isar Projects.” Accessed: Oct. 08, 2026. [Online]. Available: https://github.com/ott2/isabelle-query

[91] M. Lerchner, “Fix Isabelle2025-2 compatibility after phase-state refactor.” Accessed: Oct. 04, 2026. [Online]. Available: https://github.com/stilscher/td-verification/pull/15

[92] M. Lerchner, “refactor: separate operator laws from termination assumptions.” Accessed: Oct. 04, 2026. [Online]. Available: https://github.com/stilscher/td-verification/pull/13

[93] M. Lerchner, “Rename short constructors and record fields that leak into downstream theories.” Accessed: Oct. 04, 2026. [Online]. Available: https://github.com/stilscher/td-verification/pull/14

[94] M. Lerchner, “Add solver facts downstream users had to prove themselves.” Accessed: Oct. 04, 2026. [Online]. Available: https://github.com/stilscher/td-verification/pull/16

[95] J.-H. Jourdan, “Verasco: a Formally Verified C Static Analyzer,” Doctoral dissertation, 2016. [Online]. Available: https://theses.hal.science/tel-01327023

[96] D. Beyer, M. Dangl, D. Dietsch, and M. Heizmann, “Correctness Witnesses: Exchanging Verification Results between Verifiers,” in *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE 2016)*, ACM, 2016, pp. 326–337. doi: 10.1145/2950290.2950351.

[97] S. Saan, M. Schwarz, J. Erhard, H. Seidl, S. Tilscher, and V. Vojdani, “Correctness Witness Validation by Abstract Interpretation,” in *Verification, Model Checking, and Abstract Interpretation (VMCAI 2024)*, in LNCS, vol. 14499. Springer, 2024, pp. 74–97. doi: 10.1007/978-3-031-50524-9_4.

[98] CompCert developers, “CompCert Module Kildall: Solvers for Dataflow Inequations.” Accessed: Sep. 22, 2026. [Online]. Available: https://compcert.org/doc/html/compcert.backend.Kildall.html

[99] R. L. Spina, D. Demange, and S. Blazy, “Formal Verification of WTO-based Dataflow Solvers,” in *Programming Languages and Systems (ESOP 2025)*, in LNCS. Springer, 2025, pp. 91–118. doi: 10.1007/978-3-031-91121-7_4.

[100] S. Michelland, Y. Zakowski, and L. Gonnord, “Abstract Interpreters: A Monadic Approach to Modular Verification,” *Proceedings of the ACM on Programming Languages*, vol. 8, no. ICFP, 2024, doi: 10.1145/3674646.

[101] S. Keidel, C. B. Poulsen, and S. Erdweg, “Compositional Soundness Proofs of Abstract Interpreters,” *Proceedings of the ACM on Programming Languages*, vol. 2, no. ICFP, 2018, doi: 10.1145/3236767.

[102] D. Darais, M. Might, and D. V. Horn, “Galois Transformers and Modular Abstract Interpreters: Reusable Metatheory for Program Analysis,” in *Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA 2015)*, ACM, 2015, pp. 552–571. doi: 10.1145/2814270.2814308.

[103] D. Wasserrab, “Backing up Slicing: Verifying the Interprocedural Two-Phase Horwitz-Reps-Binkley Slicer,” *Archive of Formal Proofs*, Nov. 2009, Accessed: Sep. 22, 2026. [Online]. Available: https://isa-afp.org/entries/HRB-Slicing.html

[104] J. Breitner, “Shivers' Control Flow Analysis,” *Archive of Formal Proofs*, Nov. 2010, Accessed: Sep. 22, 2026. [Online]. Available: https://isa-afp.org/entries/Shivers-CFA.html

[105] S. Buchwald, D. Lohner, and S. Ullrich, “Verified Construction of Static Single Assignment Form,” in *Proceedings of the 25th International Conference on Compiler Construction (CC 2016)*, ACM, 2016, pp. 67–76. doi: 10.1145/2892208.2892211.

[106] G. Klein and T. Nipkow, “Verified bytecode verifiers,” *Theoretical Computer Science*, vol. 298, no. 3, pp. 583–626, 2003, doi: 10.1016/S0304-3975(02)00869-1.

[107] P. E. de Vilhena, F. Pottier, and J.-H. Jourdan, “Spy Game: Verifying a Local Generic Solver in Iris,” *Proceedings of the ACM on Programming Languages*, vol. 4, no. POPL, pp. 1–28, 2020, doi: 10.1145/3371101.

[108] J. Knoop and B. Steffen, “The Interprocedural Coincidence Theorem,” in *Compiler Construction (CC '92)*, in LNCS, vol. 641. Springer, 1992, pp. 125–140. doi: 10.1007/3-540-55984-1_13.

[109] T. Reps, S. Horwitz, and M. Sagiv, “Precise Interprocedural Dataflow Analysis via Graph Reachability,” in *Proceedings of the 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, ACM, 1995, pp. 49–61. doi: 10.1145/199448.199462.

[110] P. Sotin and B. Jeannet, “Precise Interprocedural Analysis in the Presence of Pointers to the Stack,” in *Programming Languages and Systems (ESOP 2011)*, in LNCS. Springer, 2011, pp. 459–479. doi: 10.1007/978-3-642-19718-5_24.

[111] M. Schwarz and J. Erhard, “The digest framework: concurrency-sensitivity for abstract interpretation,” *International Journal on Software Tools for Technology Transfer*, vol. 26, no. 6, pp. 727–746, 2024, doi: 10.1007/s10009-024-00773-y.